

QATAR AIRWAYS

Interview Q&A — Part 2 (Company Series)

CTK Project · Azure · Angular · .NET Core · 7 Questions with Full Answers

Qatar Airways — CTK Project Overview

The CTK (Customer Incentive) project is an internal system for managing customer incentive programs — FAM trips, flight tickets, and loyalty rewards. The tech stack includes Azure cloud, Angular UI (v11–15), .NET Core APIs, SQL Server, Azure DevOps CI/CD, and Repository pattern with Dependency Injection.

Q1. Project architecture with Azure, Angular, .NET Core

✓ Answer:

The CTK system follows a clean N-tier architecture with full cloud deployment on Azure.

Layer	Technology	Responsibility
Frontend	Angular 11–15	SPA — FAM trip booking UI, dashboards, forms
API Layer	.NET Core 6 Web API	RESTful endpoints — business logic, auth
Data Layer	EF Core + SQL Server	ORM queries, stored procedures, migrations
Auth	JWT + Azure AD	Token-based authentication & role-based access
Cloud Infra	Azure App Service + Azure SQL	Hosting, scaling, managed DB
CI/CD	Azure DevOps Pipelines	Build, test, deploy on every merge
Monitoring	Azure Application Insights	Logging, performance, exceptions

```
// Typical .NET Core Web API controller in CTK project
[ApiController]
[Route("api/[controller]")]
[Authorize(Roles = "Admin,Manager")]
public class IncentiveController : ControllerBase
{
    private readonly IIncentiveRepository _repo;
    private readonly ILogger<IncentiveController> _logger;

    public IncentiveController(IIncentiveRepository repo,
                              ILogger<IncentiveController> logger)
    {
        _repo = repo;
        _logger = logger;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> GetIncentive(int id)
    {
        var result = await _repo.GetByIdAsync(id);
        return result is null ? NotFound() : Ok(result);
    }
}
```

Q2. RESTful API development with .NET Framework 4.5 and ASP.NET Core

✓ Answer:

The CTK project migrated from .NET Framework 4.5 to ASP.NET Core for better performance, cross-platform support, and cloud-native features.

Aspect	.NET Framework 4.5	ASP.NET Core (.NET 6)
Hosting	IIS only	Kestrel, IIS, Docker, Azure App Service
DI Container	Third-party (Autofac)	Built-in (IServiceCollection)
Middleware	HTTP Modules/Handlers	Middleware pipeline
Performance	Moderate	Significantly faster (Kestrel)
Cross-platform	Windows only	Windows, Linux, macOS
Configuration	Web.config	appsettings.json + environment variables

```
// .NET Framework 4.5 – Global.asax registration
// protected void Application_Start() {
//     GlobalConfiguration.Configure(WebApiConfig.Register);
// }

// ASP.NET Core – Program.cs (minimal hosting model .NET 6+)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddScoped<IIncentiveRepository, IncentiveRepository>();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])),
            ValidateIssuer = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidateAudience = true
        };
    });

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Q3. Angular UI implementation (v11–15)

✓ Answer:

The Angular front-end handles the full incentive management lifecycle — browsing FAM trips, submitting requests, admin approvals, and tracking statuses.

```
// Angular service for API calls (CTK project pattern)
@Injectable({ providedIn: 'root' })
export class IncentiveService {
    private apiUrl = environment.apiUrl + '/incentive';
```

```

constructor(private http: HttpClient) {}

getIncentives(): Observable<Incentive[]> {
  return this.http.get<Incentive[]>(this.apiUrl).pipe(
    catchError(this.handleError)
  );
}

applyForTrip(tripId: number): Observable<any> {
  return this.http.post(`${this.apiUrl}/apply/${tripId}`, {});
}

private handleError(err: HttpErrorResponse) {
  return throwError(() => new Error(err.message));
}
}

// HTTP Interceptor — attach JWT token to every request
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.authService.getToken();
    const cloned = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;
    return next.handle(cloned);
  }
}

```

Key Angular features used in CTK:

- Lazy loading — each module (Admin, User, Reports) loaded on demand
- Reactive Forms — complex validation for trip booking and approval forms
- HTTP Interceptors — auto-attach JWT token to API calls
- Route Guards (CanActivate) — protect admin routes from unauthorized access
- Angular Material / PrimeNG — UI component library for tables, dialogs, forms

Q4. SQL Server and stored procedures

✓ **Answer:**

SQL Server is used for all relational data — employee records, trip inventory, approval workflows, and audit logs. Stored procedures handle complex batch operations.

```

-- Stored procedure: Get incentives with eligibility check
CREATE PROCEDURE usp_GetEligibleTrips
  @EmployeeID INT,
  @DepartmentID INT
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
    t.TripID,
    t.Destination,
    t.TripDate,
    t.SlotsAvailable,
    CASE
      WHEN e.YearsOfService >= 3 THEN 'Eligible'
      ELSE 'Not Eligible'
    END AS EligibilityStatus
  FROM Trips t

```

```

    CROSS JOIN Employees e
    WHERE e.EmployeeID = @EmployeeID
        AND e.DepartmentID = @DepartmentID
        AND t.SlotsAvailable > 0
        AND t.TripDate > GETDATE()
    ORDER BY t.TripDate;
END

-- Call from .NET using EF Core:
-- var trips = await _context.EligibleTrips
--     .FromSqlRaw("EXEC usp_GetEligibleTrips @p0, @p1", empId, deptId)
--     .ToListAsync();

```

Q5. Azure DevOps for tracking and CI/CD

✓ Answer:

Azure DevOps is used end-to-end: Boards for sprint tracking, Repos for Git source control, Pipelines for automated CI/CD.

```

# azure-pipelines.yml - CI/CD for CTK .NET Core API
trigger:
  branches:
    include: [ main, develop ]

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'

steps:
- task: UseDotNet@2
  inputs:
    packageType: sdk
    version: '6.0.x'

- script: dotnet restore
  displayName: 'Restore NuGet packages'

- script: dotnet build --configuration $(buildConfiguration)
  displayName: 'Build solution'

- script: dotnet test --configuration $(buildConfiguration) --logger trx
  displayName: 'Run unit tests'

- task: DotNetCoreCLI@2
  displayName: 'Publish'
  inputs:
    command: publish
    publishWebProjects: true
    arguments: '--configuration $(buildConfiguration) --output
$(Build.ArtifactStagingDirectory) '

- task: AzureRmWebAppDeployment@4
  displayName: 'Deploy to Azure App Service'
  inputs:
    azureSubscription: 'CTK-ServiceConnection'
    appName: 'ctk-api-prod'

```

Q6. Repository pattern with Dependency Injection

✓ Answer:

The Repository pattern abstracts the data access layer. Combined with DI, it makes controllers testable and decoupled from EF Core directly.

```
// 1. Interface – defines contract
public interface IIncentiveRepository
{
    Task<Incentive?> GetByIdAsync(int id);
    Task<IEnumerable<Incentive>> GetAllAsync();
    Task AddAsync(Incentive incentive);
    Task UpdateAsync(Incentive incentive);
    Task DeleteAsync(int id);
}

// 2. Implementation – uses EF Core
public class IncentiveRepository : IIncentiveRepository
{
    private readonly AppDbContext _context;
    public IncentiveRepository(AppDbContext context) => _context = context;

    public async Task<Incentive?> GetByIdAsync(int id) =>
        await _context.Incentives.FindAsync(id);

    public async Task<IEnumerable<Incentive>> GetAllAsync() =>
        await _context.Incentives.AsNoTracking().ToListAsync();

    public async Task AddAsync(Incentive incentive) {
        await _context.Incentives.AddAsync(incentive);
        await _context.SaveChangesAsync();
    }
}

// 3. Register in DI (Program.cs)
builder.Services.AddScoped<IIncentiveRepository, IncentiveRepository>();

// 4. Inject into controller – fully testable
public class IncentiveController : ControllerBase {
    public IncentiveController(IIncentiveRepository repo) => _repo = repo;
    // ... actions
}
```

 *Scoped lifetime = one instance per HTTP request. Perfect for EF Core DbContext which should not be shared across requests.*

Q7. GitHub Copilot usage for accelerated development

✓ Answer:

GitHub Copilot was used throughout the CTK project to speed up repetitive coding tasks while maintaining code quality standards.

Use Case	How Copilot Helped	Outcome
Boilerplate CRUD	Auto-generated controller actions and repository methods	~40% less manual typing
Unit Test stubs	Generated Arrange/Act/Assert test method skeletons	Faster test coverage
LINQ queries	Suggested complex GroupBy, Join, projection queries	Reduced syntax errors
Angular components	Scaffolded component + service + model together	Consistent patterns

SQL queries	Helped write stored procedure templates	Fewer syntax mistakes
Documentation	Generated XML doc comments for interfaces and methods	Improved code docs

 *Copilot suggestions were always reviewed and tested — not blindly accepted. It works best as an accelerator, not a replacement for understanding.*

TCS

Interview Q&A — Full Stack .NET + AI (20+ LPA)

27 Questions with Clear Explanations & Code Examples

Section A: Dependency Injection & OOP (Q1–Q6)

Q1. Explain Singleton lifetime in Dependency Injection with a real-time example

✓ **Answer:**

Singleton = one instance for the entire application lifetime. The same object is shared by all requests and all users.

Lifetime	Instances	Use When
Singleton	1 — entire app lifetime	Configuration, caching, logging, HttpClient
Scoped	1 per HTTP request	DbContext, repositories, business services
Transient	New every time injected	Stateless, lightweight services

```
// Real-time example: ApplicationSettings loaded once at startup
public class AppSettings {
    public string ApiBaseUrl { get; set; }
    public int MaxRetries { get; set; }
}

// Register as Singleton — loaded once, reused everywhere
builder.Services.AddSingleton<AppSettings>(sp => {
    var cfg = sp.GetRequiredService<IConfiguration>();
    return cfg.GetSection("AppSettings").Get<AppSettings>!();
});

// ⚠ Captive dependency trap:
// WRONG — Singleton depending on Scoped causes issues
// builder.Services.AddSingleton<MyService>(); // if MyService uses IDbContext (Scoped)

// CORRECT — inject IServiceScopeFactory into Singleton instead
public class MySingletonService {
    private readonly IServiceScopeFactory _factory;
    public MySingletonService(IServiceScopeFactory factory) => _factory = factory;

    public async Task DoWork() {
        using var scope = _factory.CreateScope();
        var repo = scope.ServiceProvider.GetRequiredService<IMyRepository>();
        await repo.DoSomethingAsync();
    }
}
```

Q2. Explain SOLID principles

✓ **Answer:**

Principle	Full Name	One-Line Definition
-----------	-----------	---------------------

S	Single Responsibility	A class should have only ONE reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes must be substitutable for base types
I	Interface Segregation	No client should depend on methods it doesn't use
D	Dependency Inversion	Depend on abstractions, not concrete implementations

```
// S - Single Responsibility: separate concerns
// Bad: OrderService handles business logic + email sending + DB
// Good: OrderService, EmailService, OrderRepository - each one job

// O - Open/Closed: extend without modifying existing code
public abstract class DiscountStrategy { public abstract decimal Apply(decimal price); }
public class FestivalDiscount : DiscountStrategy {
    public override decimal Apply(decimal p) => p * 0.8m; // 20% off
}

// L - Liskov Substitution
// Square should NOT inherit Rectangle if it breaks area contract

// I - Interface Segregation
public interface IPrinter { void Print(); }
public interface IScanner { void Scan(); }
// Don't force a Printer class to implement Scan()

// D - Dependency Inversion: program to interface, not class
public class OrderService {
    private readonly IOrderRepository _repo; // interface, not SqlOrderRepository
    public OrderService(IOrderRepository repo) => _repo = repo;
}
```

Q3. Difference between Hashtable and Dictionary

✓ Answer:

Feature	Hashtable	Dictionary<TKey,TValue>
Type safety	Non-generic (object keys/values)	Generic — compile-time type safety
Null keys	Allows null key	Does NOT allow null key
Performance	Boxing/unboxing for value types	No boxing — faster for value types
Thread safety	Partially thread-safe	Not thread-safe (use ConcurrentDictionary)
Namespace	System.Collections	System.Collections.Generic
Recommended	Legacy code only	Always prefer Dictionary<>

```
// Hashtable - old, avoid in new code
Hashtable ht = new Hashtable();
ht.Add("name", "Alice");
string name = (string)ht["name"]; // requires cast
```



```
// Dictionary<TKey,TValue> – modern, preferred
var dict = new Dictionary<int, string>();
dict[1] = "Alice";
dict[2] = "Bob";

if (dict.TryGetValue(1, out string val))
    Console.WriteLine(val); // Alice

// Thread-safe: use ConcurrentDictionary
var safe = new ConcurrentDictionary<int, string>();
safe.TryAdd(1, "Alice");
safe.AddOrUpdate(1, "Bob", (k, old) => "Bob");
```

Q4. Primary vs Secondary Constructor in C#

✓ **Answer:**

C# 12 (.NET 8) introduced primary constructors — a compact syntax to declare constructor parameters directly on the class/struct declaration.

```
// Traditional constructor (before C# 12)
public class UserService
{
    private readonly IUserRepository _repo;
    private readonly ILogger<UserService> _logger;

    public UserService(IUserRepository repo, ILogger<UserService> logger)
    {
        _repo = repo;
        _logger = logger;
    }
}

// Primary constructor (C# 12 / .NET 8)
public class UserService(IUserRepository repo, ILogger<UserService> logger)
{
    // Parameters 'repo' and 'logger' are directly accessible in the class body
    public async Task<User?> GetUserAsync(int id)
    {
        logger.LogInformation("Fetching user {Id}", id);
        return await repo.GetByIdAsync(id);
    }
}

// Records use primary constructors (since C# 9)
public record Point(int X, int Y); // X and Y are init-only properties
```

Q5. Convert.ToString() vs ToString()

✓ **Answer:**

Feature	Convert.ToString()	.ToString()
Null handling	Returns empty string ""	Throws NullReferenceException
Null input	Safe — handles null gracefully	Must call on non-null object
Use case	Unknown/nullable objects	Known non-null objects
Format support	No format overload	Can pass IFormatProvider
Example	Convert.ToString(null) => ""	null.ToString() => ✗ Exception

```
string? name = null;
```

```
string a = Convert.ToString(name); // "" - safe, no exception
// string b = name.ToString(); // ✗ NullReferenceException

// For value types - both work the same:
int num = 42;
string s1 = num.ToString(); // "42"
string s2 = Convert.ToString(num); // "42"

// ToString with format
double pi = 3.14159;
Console.WriteLine(pi.ToString("F2")); // 3.14 - format specifier supported
```

Q6. First(), FirstOrDefault(), Single(), SingleOrDefault() differences

✓ **Answer:**

Method	Returns	No match	Multiple matches
First()	First element	✗ InvalidOperationException	✓ Returns first
FirstOrDefault()	First or default(T)	✓ Returns null/0/false	✓ Returns first
Single()	Exactly one element	✗ InvalidOperationException	✗ InvalidOperationException
SingleOrDefault()	One or default(T)	✓ Returns null/0/false	✗ InvalidOperationException

```
var users = new List<User> {
    new User { Id=1, Name="Alice" },
    new User { Id=2, Name="Bob" }
};

// First - use when you expect results, want first one
var u1 = users.First(u => u.Id == 1); // Alice
// users.First(u => u.Id == 99); // ✗ Exception - no match

// FirstOrDefault - safe, returns null if not found
var u2 = users.FirstOrDefault(u => u.Id == 99); // null - no exception

// Single - use when exactly ONE record expected (e.g., by unique ID)
var u3 = users.Single(u => u.Id == 1); // Alice
// users.Single(u => u.Id > 0); // ✗ Exception - multiple matches

// SingleOrDefault - safe version of Single
var u4 = users.SingleOrDefault(u => u.Id == 99); // null - no exception
```

💡 Use `Single()` when you want to ASSERT uniqueness. Use `First()` when duplicates are possible and you just want one result.

Section B: ASP.NET Core & Middleware (Q7–Q13)

Q7. Middleware + write custom middleware

✓ **Answer:**

Middleware forms a pipeline — each component processes the request and passes it to the next, or short-circuits. Order matters!

```
// Custom Middleware - API Key validation
```

```

public class ApiKeyMiddleware
{
    private readonly RequestDelegate _next;
    private const string API_KEY_HEADER = "X-API-Key";
    private const string VALID_KEY = "my-secret-api-key";

    public ApiKeyMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        if (!context.Request.Headers.TryGetValue(API_KEY_HEADER, out var key)
            || key != VALID_KEY)
        {
            context.Response.StatusCode = 401;
            await context.Response.WriteAsync("Invalid or missing API key");
            return; // short-circuit — don't call next
        }
        await _next(context); // valid key — proceed
    }
}

// Register in Program.cs
app.UseMiddleware<ApiKeyMiddleware>();

```

Q8. GET vs POST — which is better and why?

✓ Answer:

Feature	GET	POST
Data location	URL query string	Request body
Security	Visible in URL/logs/history	Not visible in URL (but not encrypted by itself)
Data size	Limited (~2048 chars)	Unlimited
Idempotent	Yes — safe to repeat	No — creates new resource each time
Caching	Can be cached	Not cached by default
Use case	Read/search/filter data	Create resource, send sensitive data, file upload

💡 Neither is universally 'better' — use GET for reads, POST for writes. Use HTTPS for both to ensure transport security.

Q9. API performance optimization

✓ Answer:

Key strategies to optimize .NET Core API performance:

- Use async/await throughout — never block threads on I/O
- Enable response caching (IMemoryCache, IDistributedCache/Redis)
- Use AsNoTracking() in EF Core for read-only queries
- Add database indexes on frequently queried columns
- Implement pagination — never return unbounded result sets
- Use output caching / response compression middleware
- Use IHttpClientFactory — avoids socket exhaustion
- Profile with Application Insights / MiniProfiler to find bottlenecks

```
// 1. Async/await — never block
```

```

public async Task<IActionResult> GetOrders() =>
    Ok(await _repo.GetAllAsync()); // non-blocking

// 2. Response caching with IMemoryCache
public async Task<IActionResult> GetProducts([FromServices] IMemoryCache cache) {
    if (!cache.TryGetValue("products", out List<Product> products)) {
        products = await _repo.GetAllAsync();
        cache.Set("products", products,
            new MemoryCacheEntryOptions { AbsoluteExpirationRelativeToNow =
                TimeSpan.FromMinutes(10) });
    }
    return Ok(products);
}

// 3. AsNoTracking for reads
var users = await _context.Users.AsNoTracking().ToListAsync();

// 4. Response compression (Program.cs)
builder.Services.AddResponseCompression(o => o.EnableForHttps = true);
app.UseResponseCompression();

```

Q10. Pagination in .NET Core

✓ Answer:

```

// Pagination request model
public class PaginationParams {
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 10;
}

// API endpoint
[HttpGet]
public async Task<IActionResult> GetEmployees([FromQuery] PaginationParams p)
{
    var query = _context.Employees.AsNoTracking();
    int total = await query.CountAsync();
    var items = await query
        .Skip((p.Page - 1) * p.PageSize)
        .Take(p.PageSize)
        .ToListAsync();

    return Ok(new {
        TotalRecords = total,
        TotalPages = (int)Math.Ceiling(total / (double)p.PageSize),
        Page = p.Page,
        PageSize = p.PageSize,
        Data = items
    });
}

// SQL equivalent:
-- SELECT * FROM Employees ORDER BY EmpID
-- OFFSET (@Page-1)*@PageSize ROWS FETCH NEXT @PageSize ROWS ONLY;

```

Q11. Filters in MVC

✓ Answer:

Filter Type	Interface	Runs When
Authorization	IAuthorizationFilter	Before anything else — check JWT/roles

Resource	IResourceFilter	After auth, wrap model binding — caching
Action	IActionFilter	Before/after controller action executes
Result	IResultFilter	Before/after result (view/JSON) is produced
Exception	IExceptionHandler	When an unhandled exception is thrown

```
// Custom Action Filter – validate model + log
public class ValidateModelFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        if (!context.ModelState.IsValid)
        {
            context.Result = new BadRequestObjectResult(context.ModelState);
            return; // short-circuit – action won't run
        }
    }
    public void OnActionExecuted(ActionExecutedContext context) { }
}

// Register globally
builder.Services.AddControllers(o => o.Filters.Add<ValidateModelFilter>());
```

Q12. IActionResult vs ActionResult

✓ **Answer:**

Feature	IActionResult	ActionResult<T>
Type	Interface	Generic class (wraps IActionResult + T)
Return type	Any HTTP result	Typed result OR HTTP result
Swagger/OpenAPI	No automatic schema	Auto-generates response schema for T
Use when	Multiple unrelated types	Prefer — gives both typed + HTTP results
Example	return Ok(data) or NotFound()	return data; OR return NotFound();

```
// IActionResult – flexible but no type info for OpenAPI
[HttpGet("{id}")]
public async Task<IActionResult> GetOld(int id) {
    var user = await _repo.GetByIdAsync(id);
    return user is null ? NotFound() : Ok(user);
}

// ActionResult<T> – preferred (generates Swagger schema automatically)
[HttpGet("{id}")]
[ProducesResponseType(typeof(User), 200)]
[ProducesResponseType(404)]
public async Task<ActionResult<User>> Get(int id) {
    var user = await _repo.GetByIdAsync(id);
    return user is null ? NotFound() : user; // implicit conversion!
}
```

Q13. Why API is slow and how to fix it

✓ Answer:

Diagnose first, then fix. Common causes and solutions:

Root Cause	Symptom	Fix
N+1 query problem	100 DB calls per request	Use Include() / JOIN in EF Core
No indexes	Full table scans in SQL	Add indexes on WHERE/JOIN columns
Synchronous I/O	Thread pool exhaustion	Use async/await throughout
No caching	Same data fetched every time	IMemoryCache or Redis
Unbounded results	Returning millions of rows	Add pagination (Skip/Take)
Heavy serialization	Slow JSON responses	Use System.Text.Json, projection
Missing AsNoTracking	EF tracking unneeded objects	Add AsNoTracking() on reads
No CDN/compression	Large payload to browser	Response compression + CDN

Section C: AI Integration (Q14–Q18)

Q14. Explain AI integration in your project

✓ Answer:

Modern .NET projects integrate Azure OpenAI or Semantic Kernel for intelligent features — document summarization, chatbots, semantic search, and anomaly detection.

```
// Azure OpenAI integration in .NET Core
// Install: dotnet add package Azure.AI.OpenAI

var client = new OpenAIClient(
    new Uri("https://my-resource.openai.azure.com"),
    new AzureKeyCredential(configuration["AzureOpenAI:Key"]));

var chatCompletionsOptions = new ChatCompletionsOptions {
    Messages = {
        new ChatRequestSystemMessage("You are a helpful HR assistant."),
        new ChatRequestUserMessage("Summarize the employee leave policy."),
    },
    DeploymentName = "gpt-4"
};

var response = await client.GetChatCompletionsAsync(chatCompletionsOptions);
string answer = response.Value.Choices[0].Message.Content;
Console.WriteLine(answer);
```

Q15. How to call an AI API in .NET

✓ Answer:

```
// Option 1: Azure OpenAI SDK (recommended for Azure-hosted models)
builder.Services.AddSingleton(sp => {
    var config = sp.GetRequiredService<IConfiguration>();
    return new OpenAIClient(
        new Uri(config["AzureOpenAI:Endpoint"]!),
        new AzureKeyCredential(config["AzureOpenAI:Key"]!));
});
```

```
// Option 2: Semantic Kernel (abstraction layer – works with OpenAI, Azure, Hugging Face)
// Install: dotnet add package Microsoft.SemanticKernel
var kernel = Kernel.CreateBuilder()
    .AddAzureOpenAIChatCompletion(
        deploymentName: "gpt-4",
        endpoint: config["AzureOpenAI:Endpoint"]!,
        apiKey: config["AzureOpenAI:Key"]!)
    .Build();

var result = await kernel.InvokePromptAsync("Summarize: {{$input}}",
    new KernelArguments {{ "input", employeeReport }});
Console.WriteLine(result);
```

Q16. What is Prompt Engineering

✓ Answer:

Prompt engineering is the practice of designing input text (prompts) to guide an LLM to produce desired outputs — improving accuracy, format, tone, and relevance.

Technique	Description	Example
Zero-shot	No examples given — rely on model knowledge	"Translate to French: Hello"
Few-shot	Give 2–3 examples before the question	Show 2 examples then ask similar
Chain-of-thought	Ask model to reason step-by-step	"Think step by step: ..."
Role prompting	Give model a persona	"You are a senior .NET architect..."
Constrained output	Specify exact output format	"Respond ONLY in JSON format: {...}"
RAG	Inject relevant documents into context	Include policy docs in prompt

```
// Structured prompt engineering in C#
string BuildPrompt(string employeeData) => $"""
    You are an HR assistant. Analyze the following employee data and:
    1. Identify any policy violations
    2. Suggest corrective actions
    3. Respond ONLY in JSON format: {{ "violations": [], "actions": [] }}

    Employee Data:
    {employeeData}
    """;
```

Q17. Design AI-powered chatbot in .NET

✓ Answer:

Architecture: Angular/React frontend → SignalR or REST → .NET Core API → Azure OpenAI / Semantic Kernel → Optional Vector DB for RAG.

```
// ChatController – maintains conversation history
[ApiController]
[Route("api/[controller]")]
public class ChatController : ControllerBase
{
    private readonly OpenAIClient _client;
    private static readonly List<ChatRequestMessage> _history = new();
```

```

public ChatController(OpenAIClient client) => _client = client;

[HttpPost]
public async Task<IActionResult> Chat([FromBody] ChatRequest req)
{
    _history.Add(new ChatRequestUserMessage(req.Message));

    var options = new ChatCompletionsOptions {
        Messages = {
            new ChatRequestSystemMessage("You are a helpful assistant."),
            .._history
        },
        DeploymentName = "gpt-4"
    };

    var response = await _client.GetChatCompletionsAsync(options);
    var reply = response.Value.Choices[0].Message.Content;

    _history.Add(new ChatRequestAssistantMessage(reply));
    return Ok(new { reply });
}

public record ChatRequest(string Message);

```

💡 In production: store conversation history in a distributed cache (Redis) keyed by session ID, not in static memory.

Q18. Chatbot giving wrong answers — how to fix

✓ Answer:

Wrong answers (hallucinations) are a known LLM issue. Fix with a layered approach:

Problem	Solution
Hallucination (making up facts)	Use RAG — inject real documents as context into prompt
Out-of-date knowledge	Connect to live data sources / vector database
Wrong format	Prompt: 'Respond ONLY in JSON format' + JSON schema validation
Confidently wrong	Prompt: 'If unsure, say I don't know'
Off-topic answers	Add system prompt: 'Only answer questions about X'
Inconsistent answers	Lower temperature to 0–0.2 for factual Q&A
Long context issues	Use chunking + semantic search (RAG) to include only relevant docs

Section D: Advanced Scenarios (Q19–Q27)

Q19. API has authorization — but bypass it for specific condition inside the method

✓ Answer:

Use AllowAnonymous attribute, `IAuthorizationService.AuthorizeAsync()` inside the method, or a custom policy.

```

// Option 1: AllowAnonymous on a specific action within an authorized controller
[Authorize]
public class OrderController : ControllerBase

```



```

{
    [HttpGet("{id}")]
    public IActionResult GetOrder(int id) => Ok(/* needs auth */);

    [AllowAnonymous] // override - no auth for this action
    [HttpGet("public-status/{id}")]
    public IActionResult GetPublicStatus(int id) => Ok(/* public */);
}

// Option 2: Programmatic check inside action
[Authorize]
[HttpGet("{id}")]
public async Task<IActionResult> GetOrder(int id,
    [FromServices] IAuthorizationService authSvc)
{
    var authResult = await authSvc.AuthorizeAsync(User, "AdminPolicy");
    if (!authResult.Succeeded)
    {
        // bypass for specific condition - e.g. return limited data
        return Ok(GetPublicOrderView(id));
    }
    return Ok(GetFullOrderDetails(id));
}

```

Q20. int a=1; at runtime assign dynamically like var e = a — which data type?

✓ Answer:

Use dynamic. The dynamic type bypasses compile-time type checking and resolves type at runtime via the DLR (Dynamic Language Runtime).

```

int a = 1;
dynamic e = a; // dynamic resolves at runtime
Console.WriteLine(e); // 1
Console.WriteLine(e.GetType()); // System.Int32

e = "Now I'm a string"; // reassign to different type - allowed with dynamic
Console.WriteLine(e); // Now I'm a string

// Why NOT var?
// var is statically typed at compile time - inferred as int
// var e = a; => e IS int at compile time, cannot reassign to string

// Why NOT object?
// object o = a; - works for storage, but needs explicit cast to use methods
// dynamic gives you IntelliSense-free runtime dispatch without casts

```

Q21. MVC API returns JSON with null values — how to ignore nulls in response

✓ Answer:

```

// Option 1: Global setting in Program.cs (applies to all controllers)
builder.Services.AddControllers().AddJsonOptions(options => {
    options.JsonSerializerOptions.DefaultIgnoreCondition =
        JsonIgnoreCondition.WhenWritingNull;
});
// { "id": 1, "name": null, "age": 25 } => { "id": 1, "age": 25 }

// Option 2: Per-property using [JsonIgnore(Condition = ...)]
public class UserDto {
    public int Id { get; set; }

    [JsonIgnore(Condition = JsonIgnoreCondition.WhenWritingNull)]
    public string? Name { get; set; } // omitted if null
}

```

```

    public int Age { get; set; }
}

// Option 3: Newtonsoft.Json (if using Newtonsoft instead of System.Text.Json)
services.AddControllers().AddNewtonsoftJson(o =>
    o.SerializerSettings.NullValueHandling = NullValueHandling.Ignore);

```

Q22. SELECT * INTO newtable FROM oldtable WHERE 1 = 0

✓ Answer:

WHERE 1=0 is always FALSE — so NO rows are copied. But the table structure (schema) IS created. This is a common trick to create an empty copy of a table.

```

-- Creates 'newtable' with same columns as 'oldtable' but ZERO rows
SELECT * INTO newtable FROM oldtable WHERE 1 = 0;

-- Useful for: staging tables, temp copies for bulk insert testing
-- Result: newtable exists with correct schema, 0 records

```

Q23. SELECT * INTO newtable FROM oldtable WHERE 1 = 1

✓ Answer:

WHERE 1=1 is always TRUE — ALL rows are copied along with the schema.

```

-- Creates 'newtable' as a full copy of 'oldtable' (schema + all data)
SELECT * INTO newtable FROM oldtable WHERE 1 = 1;

-- Equivalent to: SELECT * INTO newtable FROM oldtable;
-- Useful for: quick table backup before running destructive queries

```

Q24. Can you write pagination in SQL?

✓ Answer:

```

-- SQL Server pagination using OFFSET / FETCH
DECLARE @Page      INT = 2;  -- page number (1-based)
DECLARE @PageSize  INT = 10; -- rows per page

SELECT EmpID, EmpName, Salary
FROM Employees
ORDER BY EmpID
OFFSET (@Page - 1) * @PageSize ROWS -- ORDER BY required for pagination
FETCH NEXT @PageSize ROWS ONLY;    -- skip rows from previous pages
                                   -- take only PageSize rows

-- Get total count (for frontend to show total pages)
SELECT COUNT(*) AS TotalRecords FROM Employees;

-- Older SQL Server (before 2012) - use ROW_NUMBER
WITH Paged AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY EmpID) AS RowNum
    FROM Employees
)
SELECT * FROM Paged
WHERE RowNum BETWEEN ((@Page-1)*@PageSize + 1) AND (@Page*@PageSize);

```

Q25. Insert Male='M', Female='F', Other='O' — write SQL query

✓ Answer:

```

-- Using CHECK constraint to enforce valid values
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY,
    Name   VARCHAR(100) NOT NULL,

```

```

    Gender CHAR(1) NOT NULL CHECK (Gender IN ('M', 'F', 'O'))
);

-- Insert data
INSERT INTO Users (Name, Gender) VALUES ('Alice', 'F');
INSERT INTO Users (Name, Gender) VALUES ('Bob', 'M');
INSERT INTO Users (Name, Gender) VALUES ('Alex', 'O');

-- Insert multiple rows at once
INSERT INTO Users (Name, Gender)
VALUES ('Carol', 'F'), ('Dave', 'M'), ('Sam', 'O');

-- Using CASE WHEN to insert based on full-word input
DECLARE @GenderInput VARCHAR(10) = 'Female';
INSERT INTO Users (Name, Gender)
VALUES ('Emma', CASE @GenderInput
                  WHEN 'Male' THEN 'M'
                  WHEN 'Female' THEN 'F'
                  ELSE 'O'
                END);

```

Q26. While creating UserDetails table, set default age = 18

✓ Answer:

```

-- Set DEFAULT constraint on Age column
CREATE TABLE UserDetails (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    FirstName VARCHAR(100) NOT NULL,
    LastName VARCHAR(100) NOT NULL,
    Email VARCHAR(200) UNIQUE NOT NULL,
    Age INT DEFAULT 18, -- default age = 18
    CreatedAt DATETIME DEFAULT GETDATE()
);

-- Insert without specifying Age - defaults to 18
INSERT INTO UserDetails (FirstName, LastName, Email)
VALUES ('Alice', 'Smith', 'alice@email.com');

-- Insert with explicit Age - overrides default
INSERT INTO UserDetails (FirstName, LastName, Email, Age)
VALUES ('Bob', 'Jones', 'bob@email.com', 25);

-- Add default to existing column (ALTER)
ALTER TABLE UserDetails ADD CONSTRAINT DF_Age DEFAULT 18 FOR Age;

```

Q27. Find the 3rd highest tanker level of water + find duplicate tanker names

✓ Answer:

Two queries: dense rank for Nth highest, GROUP BY for duplicates.

```

-- Table: Tankers (TankerID, TankerName, WaterLevel)

-- 1. Find 3rd highest water level
-- Method A: DENSE_RANK (handles ties correctly)
WITH RankedTankers AS (
    SELECT *,
           DENSE_RANK() OVER (ORDER BY WaterLevel DESC) AS Rnk
    FROM Tankers
)
SELECT TankerID, TankerName, WaterLevel
FROM RankedTankers
WHERE Rnk = 3;

```

```
-- Method B: Simple subquery
SELECT TOP 1 WaterLevel
FROM Tankers
WHERE WaterLevel < (
    SELECT TOP 1 WaterLevel
    FROM Tankers
    WHERE WaterLevel < (SELECT MAX(WaterLevel) FROM Tankers)
    ORDER BY WaterLevel DESC)
ORDER BY WaterLevel DESC;

-- 2. Find duplicate tanker names
SELECT TankerName, COUNT(*) AS Count
FROM Tankers
GROUP BY TankerName
HAVING COUNT(*) > 1
ORDER BY Count DESC;
```

Qatar Airways + TCS | Interview Q&A Document

34 Questions · Full Code Examples · Ready for Interview

QBURST

Interview Q&A — Microservices Architecture Focus

22 Questions · Authentication · Azure Service Bus · Kubernetes · JWT · OAuth 2.0

Section A: Project & Microservices Architecture (Q1–Q8)

Q1. Brief your technical exposure and recent projects at a high level

✓ Answer:

A structured answer covering your stack, scale, and architecture. Tailor this to your actual project.

```
// Sample answer structure (adapt to your project):  
/*  
  I have X years of experience in full-stack .NET development.  
  My recent project is [Name] – a microservices-based platform built on:  
  
  Backend   : ASP.NET Core 6 Web APIs (~8 microservices)  
  Frontend  : Angular 15 SPA  
  Database  : SQL Server (per-service), Redis for caching  
  Messaging: Azure Service Bus (async inter-service communication)  
  Auth      : Azure AD + JWT (OAuth 2.0 / OpenID Connect)  
  Cloud     : Azure App Service, AKS, Azure Key Vault, App Insights  
  CI/CD     : Azure DevOps pipelines with YAML  
  Patterns  : Repository + CQRS + MediatR + Clean Architecture  
  
  My responsibilities include API design, EF Core data modeling,  
  performance optimization, and code reviews.  
*/
```

Q2. How many microservices do you have and what does each one handle?

✓ Answer:

Each microservice owns a specific bounded context — one business capability.

Microservice	Responsibility	DB / Storage
Identity Service	Login, JWT issuance, role management	SQL Server — Users/Roles
Customer Service	Customer profile CRUD, preferences	SQL Server — Customers
Order Service	Order lifecycle — create, update, cancel	SQL Server — Orders
Payment Service	Payment processing, refunds, receipts	SQL Server — Payments
Notification Service	Email/SMS triggers from Service Bus	No DB — stateless consumer
Catalog Service	Products, pricing, inventory levels	SQL Server — Products
Reporting Service	Aggregated dashboards, exports	Redis + read replica DB
API Gateway	Routing, rate limiting, auth validation	Stateless — Azure APIM

Q3. Does each microservice have its own database or do they share one?

✓ Answer:

Each microservice has its own dedicated database — the Database-per-Service pattern. This is a core microservices principle.

Approach	Shared DB	Database-per-Service
Coupling	Tight — schema changes affect all services	Loose — each service owns its schema
Scalability	Single DB is a bottleneck	Each DB scales independently
Deployment	All services deploy together	Independent deployments
Transactions	Easy — local ACID transactions	Needs SAGA pattern for distributed tx
Recommended	✗ Anti-pattern in microservices	✓ Correct approach

💡 Each service exposes data via API, not by allowing other services to query its DB directly.

Q4. How do your services share data across service boundaries — APIs or events?

✓ Answer:

Both — use synchronous REST/gRPC for real-time reads and asynchronous events (Service Bus) for writes/notifications.

Pattern	When to Use	Example
Synchronous REST	Real-time data needed immediately	Order Service calls Customer Service to validate customer
Async Events	Non-blocking — eventual consistency	Order placed → publish event → Notification Service sends email
gRPC	High-performance internal service calls	Catalog Service → Pricing Service (internal, binary protocol)
Shared Cache	Read-heavy master data	Redis cache for product catalog shared by Order + Cart services

```
// Publishing an event to Azure Service Bus (Order Service)
public class OrderService
{
    private readonly ServiceBusSender _sender;

    public OrderService(ServiceBusClient client)
        => _sender = client.CreateSender("order-events");

    public async Task PlaceOrderAsync(Order order)
    {
        // 1. Save to own DB
        await _orderRepo.AddAsync(order);

        // 2. Publish event — other services react asynchronously
        var payload = JsonSerializer.Serialize(new {
            EventType = "OrderPlaced",
            OrderId    = order.Id,
            CustomerId = order.CustomerId,
            Amount     = order.Total
        });
        await _sender.SendMessageAsync(new ServiceBusMessage(payload));
    }
}
```

```
}  
}
```

Q5. Why are you directly querying the DB instead of calling the API of that service?

✓ Answer:

Direct cross-service DB queries are an anti-pattern in microservices. The correct approach is to call the owning service's API or use events.

Issue with cross-DB query	Why it's a problem
Tight coupling	Schema changes in Service A break Service B immediately
Bypasses business logic	Direct DB query skips validation, events, and caching in Service A
No contract	No versioning, no SLA — just raw SQL
Hard to scale	Can't independently scale or migrate databases
Security risk	Service B has DB credentials for Service A's database

💡 If you need data from another service: call its API, subscribe to its events, or replicate relevant data into your own service's cache/DB.

Q6. Could you replicate or cache the master data in the dependent service?

✓ Answer:

Yes — this is called the Data Replication pattern. Replicate slow-changing master data (e.g. product catalog) into the consuming service's own store.

```
// Consuming service subscribes to catalog-updated events  
// and stores a local copy to avoid cross-service API calls  
  
// Notification Service keeps its own copy of customer names  
public class CustomerCacheService : IHostedService  
{  
    private readonly ServiceBusProcessor _processor;  
    private readonly ICustomerCacheRepo _cache;  
  
    public async Task StartAsync(CancellationToken ct)  
    {  
        _processor.ProcessMessageAsync += async args =>  
        {  
            var evt = JsonSerializer.Deserialize<CustomerUpdatedEvent>(  
                args.Message.Body.ToString());  
  
            // Update local replica  
            await _cache.UpsertAsync(new CachedCustomer {  
                Id = evt.CustomerId,  
                Name = evt.Name,  
                Email = evt.Email  
            });  
        };  
        await _processor.StartProcessingAsync(ct);  
    }  
}
```

Q7. What about using Redis cache for master data instead of direct DB joins?

✓ **Answer:**

Redis is the preferred solution for caching master/reference data — fast, distributed, TTL-based expiry.

```
// Install: dotnet add package StackExchange.Redis
// or: dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis

// Program.cs - register Redis distributed cache
builder.Services.AddStackExchangeRedisCache(options => {
    options.Configuration = builder.Configuration["Redis:ConnectionString"];
    options.InstanceName = "OrderService_";
});

// Cache-aside pattern in a service
public class ProductCacheService
{
    private readonly IDistributedCache _cache;
    private readonly IProductApiClient _api;

    public async Task<Product?> GetProductAsync(int productId)
    {
        string key = $"product:{productId}";
        var cached = await _cache.GetStringAsync(key);
        if (cached is not null)
            return JsonSerializer.Deserialize<Product>(cached);

        // Not in cache - call the Product Service API
        var product = await _api.GetByIdAsync(productId);
        if (product is not null)
            await _cache.SetStringAsync(key,
                JsonSerializer.Serialize(product),
                new DistributedCacheEntryOptions {
                    AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
                });
        return product;
    }
}
```

Q8. Which message broker are you using for inter-service communication?

✓ **Answer:**

Azure Service Bus is the enterprise-grade choice for .NET + Azure projects. For open-source stacks, RabbitMQ or Kafka are common.

Broker	Best For	Key Feature
Azure Service Bus	Azure-native .NET apps, enterprise	Queues + Topics, dead-letter, sessions, ordering
RabbitMQ	On-prem, open-source, AMQP protocol	Flexible routing, exchanges, binding keys
Apache Kafka	High-throughput event streaming	Log retention, consumer groups, replay
Azure Event Hub	Telemetry, IoT, millions of events/sec	Partitioned consumers, Kafka-compatible API

Section B: Azure Service Bus Deep Dive (Q9–Q10)

Q9. Explain how Azure Service Bus works — publisher, consumer, retry, and dead-letter queue

✓ Answer:

Concept	Explanation
Queue	Point-to-point — one publisher, one consumer. FIFO order within a session.
Topic/Subscription	Pub-Sub — one publisher, multiple subscribers each get their own copy
Publisher	Sends ServiceBusMessage to queue or topic
Consumer	Reads and processes messages; must complete (ack) or abandon
Retry	Failed messages are retried automatically (configurable max delivery count)
Dead-Letter Queue	After max retries, message moves to DLQ for investigation
Session	Guarantees ordered processing for messages with same SessionId
Peek-Lock	Default receive mode — message is locked while processing, released on Complete

```
// Producer - send message to topic
await using var client = new ServiceBusClient(connectionString);
var sender = client.CreateSender("order-events");

var message = new ServiceBusMessage(JsonSerializer.Serialize(orderEvent))
{
    Subject          = "OrderPlaced",
    MessageId        = Guid.NewGuid().ToString(),
    ApplicationProperties = { ["EventType"] = "OrderPlaced" }
};
await sender.SendMessageAsync(message);

// Consumer - process with auto retry + dead-letter
var processor = client.CreateProcessor("order-events", "notification-sub",
    new ServiceBusProcessorOptions {
        MaxConcurrentCalls = 5,
        AutoCompleteMessages = false // manual complete for reliability
    });

processor.ProcessMessageAsync += async args => {
    try {
        var order = JsonSerializer.Deserialize<OrderEvent>(args.Message.Body.ToString());
        await SendNotificationAsync(order);
        await args.CompleteMessageAsync(args.Message); // ack - remove from queue
    }
    catch (Exception ex) {
        // abandon - message goes back for retry
        await args.AbandonMessageAsync(args.Message);
        // After max delivery count (default 10) → moves to DLQ automatically
    }
};

// Process dead-letter queue manually
var dlqProcessor = client.CreateProcessor(
    ServiceBusAdministrationClient.FormatDeadLetterPath("order-events"));
```

Q10. Do you have experience with other message brokers like RabbitMQ or Kafka?

✓ Answer:

Feature	RabbitMQ	Apache Kafka	Azure Service Bus
Protocol	AMQP	Custom TCP	AMQP / HTTP
Model	Push-based	Pull-based	Push + Pull
Retention	Until consumed	Time-based (days)	Until consumed / TTL
Replay	No (once consumed)	Yes — rewind offset	No (DLQ only)
Throughput	Medium	Very High	High
Ordering	Per-queue	Per-partition	Per-session
Best for	Task queues, RPC	Event streaming, logs	Enterprise .NET + Azure

```
// RabbitMQ with MassTransit in .NET
// Install: dotnet add package MassTransit.RabbitMQ
builder.Services.AddMassTransit(x => {
    x.AddConsumer<OrderPlacedConsumer>();
    x.UsingRabbitMq((ctx, cfg) => {
        cfg.Host("rabbitmq://localhost", h => {
            h.Username("guest"); h.Password("guest");
        });
        cfg.ConfigureEndpoints(ctx);
    });
});

// Consumer
public class OrderPlacedConsumer : IConsumer<OrderPlacedEvent>
{
    public async Task Consume(ConsumeContext<OrderPlacedEvent> context)
    {
        var order = context.Message;
        await SendEmailAsync(order.CustomerEmail, order.OrderId);
    }
}
```

Section C: Authentication & Authorization (Q11–Q15)

Q11. What authentication mechanism does your application use?

✓ Answer:

JWT (JSON Web Tokens) issued via Azure AD (Entra ID) using OAuth 2.0 / OpenID Connect. Services validate tokens on every request.

Component	Role
Azure AD (Entra ID)	Identity Provider — issues access tokens after login
OAuth 2.0	Authorization framework — defines the token flow
OpenID Connect (OIDC)	Identity layer on top of OAuth — provides ID token with user info
JWT Access Token	Passed in Authorization header — Bearer {token}

Q12. What is your experience working directly with Azure AD / Entra ID?**✓ Answer:**

- App Registration — registered client app and API app in Azure AD
- Expose API / Add Scope — defined custom scopes (e.g. api://ctk-api/read)
- App Roles — assigned Admin, Manager, User roles via manifest
- Client Credentials Flow — for service-to-service auth (no user)
- Authorization Code + PKCE — for SPA (Angular) login
- Managed Identity — used to access Key Vault and SQL without storing secrets

```
// .NET Core API - validate Azure AD JWT tokens
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

// appsettings.json
// {
//   "AzureAd": {
//     "Instance": "https://login.microsoftonline.com/",
//     "TenantId": "your-tenant-id",
//     "ClientId": "your-api-client-id",
//     "Audience": "api://your-api-client-id"
//   }
// }

// Protect endpoint with role
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id) { ... }
```

Q13. Walk through the authentication flow — how does a user log in and get a token?**✓ Answer:**

Authorization Code Flow with PKCE (for SPA/Angular):

Step	Actor	Action
1	Angular App	User clicks Login → redirect to Azure AD login page with code_challenge (PKCE)
2	Azure AD	User enters credentials → Azure AD verifies → redirects back with authorization code
3	Angular App	Exchanges code + code_verifier for access token + ID token
4	Angular App	Stores token in memory (not localStorage), attaches to API calls via interceptor
5	.NET Core API	Validates JWT: signature, issuer, audience, expiry, scopes
6	.NET Core API	Extracts claims (roles, user ID) → processes request

🔒 *Never store tokens in localStorage — XSS attacks can steal them. Use in-memory storage or HttpOnly cookies.*

Q14. Do you have an API Gateway in place for cross-cutting concerns?

✓ **Answer:**

Yes — Azure API Management (APIM) acts as the single entry point for all microservices.

Cross-Cutting Concern	How APIM Handles It
Authentication	Validate JWT policy — rejects invalid tokens before hitting services
Rate Limiting	Quota + rate-limit policies per subscription/user
Routing	Maps external URLs to internal microservice endpoints
Load Balancing	Backend pool configuration with health probes
CORS	Centralized CORS policy — not needed in each microservice
Logging	Request/response logging to App Insights
Transformation	Rewrite headers, body transformations, mock responses

Q15. What is a DDoS attack and how would you mitigate it at the API level?

✓ **Answer:**

DDoS (Distributed Denial of Service) — flood the API with massive traffic to exhaust resources and make it unavailable.

Mitigation Layer	Tool / Technique
Azure DDoS Protection	Azure DDoS Standard plan — absorbs volumetric attacks at network level
API Gateway Rate Limiting	APIM quotas — limit requests per IP/subscription
WAF (Web App Firewall)	Azure Application Gateway WAF — blocks SQLi, XSS, bot traffic
.NET Rate Limiting	ASP.NET Core built-in rate limiting middleware (fixed/sliding window)
IP Allow/Block Lists	APIM or Azure Front Door policies
CAPTCHA / Bot Detection	Azure Bot Protection, reCAPTCHA on login endpoints

```
// .NET Core 7+ built-in rate limiting middleware
builder.Services.AddRateLimiter(options => {
    options.AddFixedWindowLimiter("fixed", opt => {
        opt.Window = TimeSpan.FromMinutes(1);
        opt.PermitLimit = 100; // max 100 requests per minute
        opt.QueueLimit = 10; // queue up to 10 overflow requests
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });
    options.RejectionStatusCode = 429; // Too Many Requests
});

app.UseRateLimiter();
```

```
// Apply to specific endpoint
[EnableRateLimiting("fixed")]
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest req) { ... }
```

Section D: Deployment & Kubernetes (Q16–Q18)

Q16. How is your application deployed — AKS, App Service, or something else?

✓ **Answer:**

Microservices deployed to Azure Kubernetes Service (AKS) — containers managed via Helm charts, CI/CD through Azure DevOps.

Component	Deployment Target	Why
Microservices APIs	AKS (pods)	Independent scaling, container isolation
Angular SPA	Azure Static Web App	CDN-backed, automatic HTTPS
SQL Databases	Azure SQL Managed Instance	PaaS — managed backups, HA
Redis Cache	Azure Cache for Redis	Managed, high-availability
Service Bus	Azure Service Bus (PaaS)	Fully managed messaging
Secrets	Azure Key Vault	No secrets in code or config files

Q17. Do you understand how Kubernetes works — pods, nodes, services, ingress controller?

✓ **Answer:**

K8s Concept	Definition	Real-World Analogy
Pod	Smallest deployable unit — 1+ containers	A single running app instance
Node	VM that runs pods	Physical/virtual server
Deployment	Desired state spec — replicas, image, strategy	"I want 3 copies of order-api running"
Service	Stable network endpoint for a set of pods	Load balancer for pods
Ingress	HTTP routing rules — external → internal services	Nginx/reverse proxy rules
ConfigMap	Non-sensitive config injected into pods	appsettings for the cluster
Secret	Base64-encoded sensitive values	Connection strings, API keys
HPA	Horizontal Pod Autoscaler — scales pod count	Auto scale-out under load
Namespace	Logical isolation within cluster	Separate environments: dev/staging/prod

Q18. In a Kubernetes deployment how does service discovery work?

✓ **Answer:**

Kubernetes uses DNS-based service discovery. Every Service gets a DNS name: <service-name>.<namespace>.svc.cluster.local

```
# Kubernetes Service manifest for Order Service
apiVersion: v1
kind: Service
metadata:
  name: order-service
  namespace: production
spec:
  selector:
    app: order-api          # routes to pods with this label
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

# Other services reach order-service via DNS:
# http://order-service          (same namespace)
# http://order-service.production.svc.cluster.local (cross-namespace)

# In .NET appsettings.json inside the cluster:
# "OrderServiceUrl": "http://order-service/api"

# Ingress - external traffic routing
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
spec:
  rules:
    - host: api.myapp.com
      http:
        paths:
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: order-service
                port: { number: 80 }
```

Section E: OAuth 2.0, JWT & Token Validation (Q19–Q22)

Q19. OAuth 2.0 Authorization Code Flow vs PKCE Flow — what is the difference?

✓ **Answer:**

Feature	Authorization Code Flow	Authorization Code + PKCE
Used by	Confidential clients (server apps)	Public clients (SPA, mobile apps)
Client Secret	Required — stored on server	Not used — no secret to store/expose
PKCE	Optional	Required — prevents auth code interception
code_verifier	Not needed	Random string generated by client
code_challenge	Not needed	SHA256(code_verifier) sent with auth request

Security	Secure for server-side apps	Secure for browser/mobile — no secret leakage
----------	-----------------------------	---

🔑 PKCE (Proof Key for Code Exchange) prevents a malicious app from exchanging a stolen authorization code for tokens, since it doesn't have the code_verifier.

Q20. What are the minimum configurations needed to register a client application with Azure AD?

✓ Answer:

- Application Name — display name in Azure AD
- Supported Account Types — single tenant, multi-tenant, or personal accounts
- Redirect URI — where Azure AD sends the auth code after login
- Client ID (auto-generated) — public identifier of the app
- Tenant ID — your Azure AD directory identifier
- For API access: Expose an API → Add a Scope, and client app adds API Permission
- For confidential clients: generate a Client Secret or Certificate

Q21. What are the 3 parts of a JWT token and what is the role of each part?

✓ Answer:

Part	Encoding	Contains	Purpose
Header	Base64URL	Algorithm (HS256/RS256), token type (JWT)	Tells receiver how to verify signature
Payload	Base64URL	Claims: sub, iss, aud, exp, roles, email	The actual data — who, what, when
Signature	Base64URL	HMAC or RSA signature of Header + Payload	Tamper detection — ensures token is unmodified

```
// JWT structure: Header.Payload.Signature
// eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9 <- Header
// .eyJzdWIiOiIxMjMONTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIn0 <- Payload
// .SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c <- Signature

// Decoded Payload example:
// {
//   "sub": "user-id-123", // subject (user ID)
//   "iss": "https://login.microsoftonline.com/tenant-id", // issuer
//   "aud": "api://my-api-id", // audience (your API)
//   "exp": 1735689600, // expiry (Unix timestamp)
//   "iat": 1735686000, // issued at
//   "roles": ["Admin"], // custom claim
//   "name": "Alice Smith"
// }
```

Q22. From the API side how do you validate a JWT token — what specific claims do you check?

✓ Answer:

Claim	What to Validate	Why
-------	------------------	-----

iss (Issuer)	Matches your Azure AD tenant URL	Ensures token came from your identity provider
aud (Audience)	Matches your API's client ID	Token intended for YOUR API, not another
exp (Expiry)	Current time < exp	Token not expired
nbf (Not Before)	Current time > nbf	Token is valid from this time
sub (Subject)	Present — identifies the user	Know who made the request
signature	Verify using public key from JWKS URI	Token was not tampered with
roles / scp	Match required roles/scopes	Authorization — user has permission

```
// .NET Core JWT validation — these checks happen automatically
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true,
            ValidIssuer =
                "https://login.microsoftonline.com/{tenantId}/v2.0",
            ValidateAudience = true,
            ValidAudience = "api://your-api-client-id",
            ValidateLifetime = true, // checks exp + nbf
            ValidateIssuerSigningKey = true, // verifies signature
            // Azure AD public keys fetched automatically from JWKS endpoint
            IssuerSigningKeyResolver = ...
        };
    });

// Access claims in controller
var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
var userEmail = User.FindFirst("email")?.Value;
bool isAdmin = User.IsInRole("Admin");
```


ENVESTNET

Interview Q&A — Senior .NET Developer (Design Focus)

9 Questions · System Design · DI · SOLID · Azure Blob · FileUploader

Section A: Project & Architecture (Q1–Q2)

Q1. Self Intro — Tell me about yourself

✓ Answer:

Structure: Current role → Years of experience → Tech stack → Recent achievement → Why this role.

```
// Sample intro structure
/*
I am a Senior .NET Full Stack Developer with X years of experience
building enterprise applications using ASP.NET Core, Angular, SQL Server,
and Azure cloud services.

In my current role at [Company], I work on a microservices-based platform
handling [domain]. I lead API design, mentor junior developers, and
drive architecture decisions – recently migrated from .NET Framework to
.NET 6 which improved performance by ~35%.

I specialize in clean architecture, SOLID principles, and Azure-native
solutions. I'm excited about this Senior .NET Developer role at Envestnet
because [specific reason – fintech domain, scale, tech stack].
*/
```

Q2. Describe current project + architecture

✓ Answer:

Cover: business domain → architecture style → tech stack → your specific contributions → scale.

Layer	Technology	Your Contribution
API	ASP.NET Core 6 Web API	Designed RESTful endpoints, implemented CQRS with MediatR
Frontend	Angular 15	Built reactive forms, HTTP interceptors, lazy loading
Data	EF Core + SQL Server	Code-first migrations, query optimization, stored procs
Messaging	Azure Service Bus	Pub-sub for async notifications
Auth	Azure AD + JWT	App registration, RBAC, token validation
DevOps	Azure DevOps	YAML CI/CD pipelines, release gates
Monitoring	App Insights + Serilog	Structured logging, performance dashboards

Section B: System Design — Parking Lot (Q3)

Q3. Design parking lot system: Multiple regions, 2-wheeler + 4-wheeler, Pluggable allocation strategies

✓ Answer:

This tests OOP, SOLID, and design pattern knowledge. Use Strategy pattern for pluggable allocation.

```
// — Domain Models —————
public enum VehicleType { TwoWheeler, FourWheeler }

public class Vehicle
{
    public string LicensePlate { get; set; }
    public VehicleType Type { get; set; }
}

public class ParkingSpot
{
    public int SpotId { get; set; }
    public VehicleType Type { get; set; }
    public bool IsOccupied { get; set; }
    public string? VehiclePlate { get; set; }
}

public class ParkingFloor
{
    public int FloorNumber { get; set; }
    public List<ParkingSpot> Spots { get; set; } = new();
}

public class ParkingLot
{
    public string Region { get; set; } // North, South, etc.
    public List<ParkingFloor> Floors { get; set; } = new();
}

// — Strategy Pattern – pluggable allocation —————
public interface IAllocationStrategy
{
    ParkingSpot? Allocate(ParkingLot lot, Vehicle vehicle);
}

// Strategy 1: Nearest available spot
public class NearestSpotStrategy : IAllocationStrategy
{
    public ParkingSpot? Allocate(ParkingLot lot, Vehicle vehicle)
    => lot.Floors
        .SelectMany(f => f.Spots)
        .FirstOrDefault(s => !s.IsOccupied && s.Type == vehicle.Type);
}

// Strategy 2: Even distribution across floors
public class EvenDistributionStrategy : IAllocationStrategy
{
    public ParkingSpot? Allocate(ParkingLot lot, Vehicle vehicle)
    {
        var floorWithLeastOccupancy = lot.Floors
            .OrderBy(f => f.Spots.Count(s => s.IsOccupied))
            .FirstOrDefault();
        return floorWithLeastOccupancy?.Spots
            .FirstOrDefault(s => !s.IsOccupied && s.Type == vehicle.Type);
    }
}
```

```
// — Parking Service – uses injected strategy —————
public class ParkingService
{
    private readonly IAllocationStrategy _strategy;
    private readonly IParkingRepository _repo;

    // Strategy injected – swap strategies without changing service
    public ParkingService(IAllocationStrategy strategy, IParkingRepository repo)
    {
        _strategy = strategy;
        _repo = repo;
    }

    public async Task<ParkingSpot?> ParkVehicleAsync(string region, Vehicle vehicle)
    {
        var lot = await _repo.GetLotByRegionAsync(region);
        var spot = _strategy.Allocate(lot, vehicle);
        if (spot is null) return null;

        spot.IsOccupied = true;
        spot.VehiclePlate = vehicle.LicensePlate;
        await _repo.SaveAsync();
        return spot;
    }
}

// — DI Registration —————
// Swap strategy without changing any other code:
builder.Services.AddScoped<IAllocationStrategy, NearestSpotStrategy>();
// or: builder.Services.AddScoped<IAllocationStrategy, EvenDistributionStrategy>();
```

💡 **SOLID applied:** S (ParkingService does parking only), O (add new strategy without modifying), D (depends on IAllocationStrategy interface).

Section C: Middleware, DI & Static (Q4–Q6)

Q4. What is middleware in .NET Core? Built custom middleware in project?

✓ Answer:

Middleware = components in the HTTP pipeline. Each can process requests/responses and call the next component or short-circuit.

```
// Custom Middleware – Correlation ID tracking (used in distributed systems)
public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;
    private const string HEADER = "X-Correlation-Id";

    public CorrelationIdMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        // Check if correlation ID was passed by caller
        if (!context.Request.Headers.TryGetValue(HEADER, out var correlationId))
            correlationId = Guid.NewGuid().ToString();

        // Make it available to the entire request pipeline
        context.Items[HEADER] = correlationId.ToString();
        context.Response.Headers[HEADER] = correlationId.ToString();

        using (Serilog.Context.LogContext.PushProperty("CorrelationId", correlationId))
        {
            await _next(context);
        }
    }
}
```

```

        await _next(context); // all logs within this request carry CorrelationId
    }
}

// Program.cs
app.UseMiddleware<CorrelationIdMiddleware>(); // must be early in pipeline

```

Q5. How DI works? Three lifetimes — explain each

✓ Answer:

Lifetime	New Instance Created	Shared Across	Use For
Singleton	Once per app startup	All requests, all users	Config, cache, HttpClient, logging
Scoped	Once per HTTP request	Same request only	DbContext, repositories, unit-of-work
Transient	Every time it is requested	Never — always fresh	Lightweight, stateless services

```

// Registration
builder.Services.AddSingleton<IConfigService, ConfigService>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddTransient<IEmailFormatter, EmailFormatter>();

// ⚠ Common mistake: Captive dependency
// A Singleton holding a Scoped service is a bug —
// the Scoped service lives longer than its intended lifetime

// WRONG:
// services.AddSingleton<MySingleton>(); // MySingleton injects IOrderRepository (Scoped)

// CORRECT: Use IServiceScopeFactory inside Singleton to create scope on demand
public class MySingleton
{
    private readonly IServiceScopeFactory _factory;
    public MySingleton(IServiceScopeFactory factory) => _factory = factory;

    public async Task DoWorkAsync()
    {
        using var scope = _factory.CreateScope();
        var repo = scope.ServiceProvider.GetRequiredService<IOrderRepository>();
        await repo.DoSomethingAsync();
    }
}

```

Q6. Why use Singleton instead of Static class? What's the difference?

✓ Answer:

Feature	Static Class	Singleton (DI)
Testability	Cannot be mocked/replaced in tests	Can be mocked via interface in unit tests
DI integration	Cannot be injected — global access	Injected via constructor — no global state
Interface	Cannot implement interface	Can implement interface — abstraction possible

Lifetime control	Lives for entire app — no control	DI container controls lifetime
Thread safety	Developer must handle locking	Same — developer must handle thread safety
Lazy init	Class loads on first use	Lazy<T> or Lazy registration available

```
// Static class — hard to test, global, no DI
public static class TaxCalculator {
    public static decimal Calculate(decimal amount) => amount * 0.18m;
}
// Usage: TaxCalculator.Calculate(1000); — cannot mock in tests

// Singleton via DI — testable, injectable
public interface ITaxCalculator { decimal Calculate(decimal amount); }

public class TaxCalculator : ITaxCalculator {
    public decimal Calculate(decimal amount) => amount * 0.18m;
}

builder.Services.AddSingleton<ITaxCalculator, TaxCalculator>();

// In test — easy to mock:
var mockCalc = new Mock<ITaxCalculator>();
mockCalc.Setup(c => c.Calculate(It.IsAny<decimal>())) .Returns(100m);
```

Section D: Performance & System Design (Q7)

Q7. Website load time very high — steps to investigate and fix

✓ Answer:

Systematic debugging: measure → profile → identify bottleneck → fix → verify.

Step	Action	Tool
1. Measure	Check browser Network tab — find slowest requests	Chrome DevTools / Lighthouse
2. API profiling	Check Application Insights — slow requests, dependencies	Azure App Insights / MiniProfiler
3. DB queries	Find N+1, missing indexes, full table scans	SQL Profiler / Execution Plan
4. Frontend bundle	Check JS bundle size — lazy loading applied?	webpack-bundle-analyzer
5. Caching	Is repeated data being fetched every request?	Redis / IMemoryCache
6. CDN	Static assets served from origin instead of CDN?	Azure Front Door / CDN
7. Network	TTFB (Time to First Byte) high? Server location far?	Lighthouse / PageSpeed

```
// Quick wins in .NET Core API:
// 1. Add response compression
```

```

builder.Services.AddResponseCompression();
app.UseResponseCompression();

// 2. Add output caching for GET endpoints
builder.Services.AddOutputCache();
app.UseOutputCache();

[HttpGet]
[OutputCache(Duration = 60)] // cache for 60 seconds
public async Task<IActionResult> GetProducts()
    => Ok(await _repo.GetAllAsync());

// 3. Fix N+1 with Include() in EF Core
// BAD - N+1 queries
var orders = await _context.Orders.ToListAsync();
foreach (var o in orders) { var c = o.Customer; } // extra query per order!

// GOOD - single JOIN query
var orders = await _context.Orders.Include(o => o.Customer).ToListAsync();

```

Section E: FileUploader System Design + SOLID (Q8–Q9)

Q8. Design FileUploader class: Support JSON + CSV now, Extensible for future formats, Upload to Azure Blob

✓ Answer:

Apply SOLID principles: Strategy pattern for parsers, Dependency Inversion for storage.

```

// — Abstractions —————
public interface IFileParser<T>
{
    IEnumerable<T> Parse(Stream fileStream);
}

public interface IBlobStorageService
{
    Task<string> UploadAsync(Stream data, string fileName, string contentType);
}

// — Concrete Parsers —————
public class JsonFileParser<T> : IFileParser<T>
{
    public IEnumerable<T> Parse(Stream fileStream)
        => JsonSerializer.Deserialize<List<T>>(fileStream) ?? new List<T>();
}

public class CsvFileParser<T> : IFileParser<T>
{
    public IEnumerable<T> Parse(Stream fileStream)
    {
        using var reader = new StreamReader(fileStream);
        using var csv = new CsvReader(reader, CultureInfo.InvariantCulture);
        return csv.GetRecords<T>().ToList();
    }
}

// Future: XmlFileParser<T>, ExcelFileParser<T> – no existing code changes!

// — Azure Blob Storage Service —————
public class AzureBlobStorageService : IBlobStorageService

```

```

{
    private readonly BlobContainerClient _container;

    public AzureBlobStorageService(IConfiguration config)
    {
        var blobService = new
BlobServiceClient(config["Azure:BlobStorage:ConnectionString"]);
        _container = blobService.GetBlobContainerClient("uploads");
    }

    public async Task<string> UploadAsync(Stream data, string fileName, string
contentType)
    {
        var blob = _container.GetBlobClient(fileName);
        await blob.UploadAsync(data,
            new BlobHttpHeaders { ContentType = contentType });
        return blob.Uri.ToString();
    }
}

// — FileUploader — orchestrates parsing + upload —————
public class FileUploader<T>
{
    private readonly IFileParser<T> _parser;
    private readonly IBlobStorageService _storage;
    private readonly ILogger<FileUploader<T>> _logger;

    public FileUploader(
        IFileParser<T> parser,
        IBlobStorageService storage,
        ILogger<FileUploader<T>> logger)
    {
        _parser = parser;
        _storage = storage;
        _logger = logger;
    }

    public async Task<FileUploadResult<T>> UploadAndParseAsync(
        IFormFile file)
    {
        // 1. Parse file contents
        await using var stream = file.OpenReadStream();
        var records = _parser.Parse(stream).ToList();
        _logger.LogInformation("Parsed {Count} records from {File}",
            records.Count, file.FileName);

        // 2. Upload original file to Azure Blob
        await using var uploadStream = file.OpenReadStream();
        var blobUrl = await _storage.UploadAsync(
            uploadStream, file.FileName, file.ContentType);

        return new FileUploadResult<T>
        {
            Records = records,
            BlobUrl = blobUrl,
            FileName = file.FileName
        };
    }
}

public record FileUploadResult<T>
{
    public List<T> Records { get; init; } = new();
    public string BlobUrl { get; init; } = string.Empty;
    public string FileName { get; init; } = string.Empty;
}

```

```
// — DI Registration —————
// Register based on content type at runtime:
builder.Services.AddScoped<IFileParser<Employee>, JsonFileParser<Employee>>>();
builder.Services.AddScoped<IBlobStorageService, AzureBlobStorageService>();
builder.Services.AddScoped<FileUploader<Employee>>>();
```

Q9. Which SOLID principles does your FileUploader design follow? Explain each

✓ Answer:

Principle	How FileUploader Applies It
S — Single Responsibility	FileUploader only orchestrates; parsing is IFileParser's job; storage is IBlobStorageService's job
O — Open/Closed	Add XmlFileParser<T> or ExcelFileParser<T> without modifying FileUploader — open for extension
L — Liskov Substitution	JsonFileParser and CsvFileParser are interchangeable — both fulfill IFileParser<T> contract
I — Interface Segregation	IFileParser only has Parse(). IBlobStorageService only has UploadAsync(). No fat interfaces
D — Dependency Inversion	FileUploader depends on IFileParser + IBlobStorageService abstractions, not concrete classes

💡 This design means: to support Excel uploads, just implement ExcelFileParser<T> : IFileParser<T> and register it — zero changes to FileUploader, controller, or storage.

QBurst + Envestnet | Interview Q&A Document

31 Questions · Microservices · Kubernetes · OAuth 2.0 · SOLID · System Design · Full Code Examples

VIRTUSA

Interview Q&A — Part 6

Lead Software Engineer (10+ YOY) | All 38 Questions with Code Examples

SECTION A: SQL Questions (Q1–Q5)

Q1. Find employees who have worked on at least 2 projects

✓ **Answer:**

Use GROUP BY on employee-project relationship table and filter with HAVING COUNT >= 2.

```
-- Table: EmployeeProjects (EmpID, ProjectID)

SELECT e.EmpID, e.EmpName, COUNT(ep.ProjectID) AS ProjectCount
FROM Employees e
JOIN EmployeeProjects ep ON e.EmpID = ep.EmpID
GROUP BY e.EmpID, e.EmpName
HAVING COUNT(ep.ProjectID) >= 2
ORDER BY ProjectCount DESC;
```

💡 *HAVING filters on aggregated data (after GROUP BY), while WHERE filters rows before grouping.*

Q2. Identify the longest duration project

✓ **Answer:**

Use DATEDIFF to compute duration and ORDER BY descending, taking TOP 1.

```
-- Table: Projects (ProjectID, ProjectName, StartDate, EndDate)

SELECT TOP 1
    ProjectID,
    ProjectName,
    StartDate,
    EndDate,
    DATEDIFF(DAY, StartDate, EndDate) AS DurationInDays
FROM Projects
ORDER BY DATEDIFF(DAY, StartDate, EndDate) DESC;
```

Q3. Return employee names along with the longest project they worked on

✓ **Answer:**

Join Employees + EmployeeProjects + Projects, then use ROW_NUMBER() OVER (PARTITION BY EmpID ...) to pick the longest project per employee.

```
WITH RankedProjects AS (
    SELECT
        e.EmpID, e.EmpName,
        p.ProjectName,
        DATEDIFF(DAY, p.StartDate, p.EndDate) AS Duration,
        ROW_NUMBER() OVER (
            PARTITION BY e.EmpID
            ORDER BY DATEDIFF(DAY, p.StartDate, p.EndDate) DESC
        ) AS rn
    FROM Employees e
    JOIN EmployeeProjects ep ON e.EmpID = ep.EmpID
    JOIN Projects p ON ep.ProjectID = p.ProjectID
)
SELECT EmpID, EmpName, ProjectName, Duration
```

```
FROM RankedProjects
WHERE rn = 1;
```

Q4. Find CustId, CustName, OrderCount for customers having more than 3 orders

✓ Answer:

```
-- Tables: Customers (CustId, CustName), Orders (OrderId, CustId)

SELECT c.CustId, c.CustName, COUNT(o.OrderId) AS OrderCount
FROM Customers c
JOIN Orders o ON c.CustId = o.CustId
GROUP BY c.CustId, c.CustName
HAVING COUNT(o.OrderId) > 3
ORDER BY OrderCount DESC;
```

Q5. Brand, TotalQuantity sold (include brands with zero sales)

✓ Answer:

Use LEFT JOIN from Brands to Sales so brands with no sales are still included (NULL becomes 0 via ISNULL).

```
-- Tables: Brands (BrandID, BrandName), Sales (SaleID, BrandID, Qty)

SELECT b.BrandName,
       ISNULL(SUM(s.Qty), 0) AS TotalQuantity
FROM Brands b
LEFT JOIN Sales s ON b.BrandID = s.BrandID
GROUP BY b.BrandName
ORDER BY TotalQuantity DESC;
```

💡 Use LEFT JOIN (not INNER JOIN) to preserve brands with zero sales.

SECTION B: C# Questions (Q6–Q11)

Q6. Generate all combinations and permutations of 'ABC'

✓ Answer:

Use recursion. Permutations = all orderings; Combinations = subset selections without order.

```
// ---- PERMUTATIONS of 'ABC' ----
void Permute(string str, string prefix = "")
{
    if (str.Length == 0)
    {
        Console.WriteLine(prefix);
        return;
    }
    for (int i = 0; i < str.Length; i++)
    {
        char c = str[i];
        string remaining = str.Remove(i, 1);
        Permute(remaining, prefix + c);
    }
}
Permute("ABC");
// Output: ABC, ACB, BAC, BCA, CAB, CBA

// ---- COMBINATIONS (subsets) of 'ABC' ----
void Combine(string str, int start, string current = "")
{
    if (current.Length > 0) Console.WriteLine(current);
    for (int i = start; i < str.Length; i++)
```

```

        Combine(str, i + 1, current + str[i]);
    }
    Combine("ABC", 0);
    // Output: A, AB, ABC, AC, B, BC, C

```

Q7. Output of overriding vs hiding (virtual methods) — code output question

✓ Answer:

The key concept: virtual + override = runtime polymorphism (late binding). new keyword = method hiding (compile-time / early binding).

```

class A {
    public virtual void Show() => Console.WriteLine("A.Show()");
    public void Display() => Console.WriteLine("A.Display()");
}
class B : A {
    public override void Show() => Console.WriteLine("B.Show()"); // override
    public new void Display() => Console.WriteLine("B.Display()"); // hiding
}

A obj = new B();
obj.Show();      // Output: B.Show()    <- runtime polymorphism
obj.Display();   // Output: A.Display() <- method hiding uses compile-time type

B obj2 = new B();
obj2.Show();     // Output: B.Show()
obj2.Display();  // Output: B.Display()

```

Feature	override (virtual)	new (hiding)
Binding	Runtime (late binding)	Compile-time (early binding)
Keyword on base	virtual / abstract	Any method
Keyword on derived	override	new
Base ref calls	Derived method	Base method

Q8. Sort array {4,0,7,5,2,8,0,0,0} — move all zeros to front, keep rest in original order

✓ Answer:

Two-pointer or stable partition approach: collect zeros, then non-zeros.

```

int[] arr = { 4, 0, 7, 5, 2, 8, 0, 0, 0 };

// Two-pass approach: zeros first, preserve original order of non-zeros
var zeros      = arr.Where(x => x == 0).ToList();
var nonZeros   = arr.Where(x => x != 0).ToList();
int[] result = zeros.Concat(nonZeros).ToArray();

Console.WriteLine(string.Join(", ", result));
// Output: 0, 0, 0, 0, 4, 7, 5, 2, 8

// In-place approach (two pointers):
int pos = 0;
for (int i = 0; i < arr.Length; i++)
    if (arr[i] == 0) { int tmp = arr[pos]; arr[pos++] = arr[i]; arr[i] = tmp; }

```

💡 The original order within non-zero elements must be preserved — do not use `Array.Sort()`.

Q9. Find duplicate characters in a string

✓ Answer:

```

string input = "programming";

```

```
// Method 1: Using Dictionary
var freq = new Dictionary<char, int>();
foreach (char c in input)
    freq[c] = freq.ContainsKey(c) ? freq[c] + 1 : 1;

var duplicates = freq.Where(kv => kv.Value > 1).Select(kv => kv.Key);
Console.WriteLine("Duplicates: " + string.Join(", ", duplicates));
// Output: Duplicates: r, g, m

// Method 2: LINQ one-liner
var dups = input.GroupBy(c => c)
    .Where(g => g.Count() > 1)
    .Select(g => g.Key);
Console.WriteLine(string.Join(", ", dups));
```

Q10. What is Concurrent Programming?

✓ Answer:

Concurrent programming means multiple tasks make progress simultaneously — not necessarily at the exact same instant (that is parallelism). In C#, this is achieved via threads, `async/await`, Task Parallel Library (TPL), and `ConcurrentCollections`.

Concept	Definition	C# Example
Concurrency	Multiple tasks in progress at the same time (may interleave)	<code>async/await</code> , <code>Task.WhenAll</code>
Parallelism	Tasks truly running simultaneously (multi-core CPU)	<code>Parallel.ForEach</code> , <code>PLINQ</code>
Thread Safety	Data stays consistent under concurrent access	<code>lock</code> , <code>Interlocked</code> , <code>ConcurrentDictionary</code>

```
// Concurrent tasks with async/await
async Task RunConcurrentAsync()
{
    var t1 = FetchDataAsync("API1");
    var t2 = FetchDataAsync("API2");
    var t3 = FetchDataAsync("API3");
    var results = await Task.WhenAll(t1, t2, t3); // all run concurrently
    foreach (var r in results) Console.WriteLine(r);
}

// Thread-safe counter using Interlocked
int counter = 0;
Parallel.For(0, 1000, i => Interlocked.Increment(ref counter));
Console.WriteLine(counter); // Always 1000, not a race condition
```

Q11. What is GraphQL and why is it used?

✓ Answer:

GraphQL is a query language for APIs that lets the client specify exactly what data it needs — unlike REST which returns fixed shapes per endpoint. Created by Facebook, it reduces over-fetching and under-fetching.

Feature	REST	GraphQL
Endpoints	Multiple endpoints per resource	Single endpoint (<code>/graphql</code>)
Data shape	Fixed by server	Specified by client
Over-fetching	Common	Eliminated

Under-fetching	Multiple calls needed	Single query
Versioning	URL-based (/v1, /v2)	Schema evolution
Real-time	Polling/WebSockets manually	Built-in Subscriptions

```
// GraphQL in .NET using Hot Chocolate
// Install: dotnet add package HotChocolate.AspNetCore

// Query type
public class Query {
    public Employee GetEmployee(int id, [Service] IEmployeeRepo repo)
        => repo.GetById(id);
}

// Program.cs registration
builder.Services
    .AddGraphQLServer()
    .AddQueryType<Query>();

// Client GraphQL query (only asks for needed fields):
// {
//   employee(id: 1) {
//     name
//     department { name }
//   }
// }
```

SECTION C: .NET Core Questions (Q12–Q25)

Q12. .NET 6 vs .NET 8 differences

✓ Answer:

Area	.NET 6 (LTS)	.NET 8 (LTS)
Minimal APIs	Introduced	Enhanced with filters, OpenAPI, route groups
C# Version	C# 10	C# 12
Primary Constructors	Not available	Available on all classes
Blazor	WASM + Server	Unified Blazor (Auto render mode)
Native AOT	Limited preview	Full support for APIs
Performance	Fast	Faster (JIT, GC improvements)
JSON	System.Text.Json	Source generators mature, JsonSerializerOptions
LTS Until	November 2024	November 2026

```
// C# 12 Primary Constructors (.NET 8)
public class UserService(IUserRepository repo, ILogger<UserService> logger)
{
    public async Task<User> GetUserAsync(int id)
    {
        logger.LogInformation("Fetching user {Id}", id);
        return await repo.GetByIdAsync(id);
    }
}
```

```
}
```

Q13. var vs dynamic differences

✓ Answer:

Feature	var	dynamic
Type resolution	Compile-time (statically typed)	Runtime (dynamically typed)
IntelliSense	Full support	No IntelliSense
Performance	Same as explicit type	Slower (DLR overhead)
Errors caught	At compile time	At runtime only
Reassignment	Same type only	Any type allowed

```
var x = 10;           // Compiler infers int - x IS int at compile time
// x = "hello";      // ERROR: cannot assign string to int

dynamic d = 10;       // Runtime type
d = "hello";          // OK at compile time - fails or works at runtime
d = new List<int>();   // Also OK

// Use dynamic when: COM interop, reflection-heavy code, duck typing
// Use var when: reducing verbose type declarations
```

Q14. Task vs Thread differences

✓ Answer:

Feature	Thread	Task
Level	Low-level OS thread	High-level abstraction over ThreadPool
Return value	Cannot return value	Task<T> returns a value
Cancellation	Manual via bool flag	Built-in CancellationToken
Exception handling	Must handle in thread	Can await and catch
Continuation	Manual join/callback	ContinueWith / await
Use when	Fine-grained thread control	Async operations, I/O, CPU tasks

```
// Thread - low level
Thread t = new Thread(() => Console.WriteLine("Thread running"));
t.Start();
t.Join(); // wait

// Task - high level, preferred
Task<int> task = Task.Run(() => {
    Thread.Sleep(1000);
    return 42;
});
int result = await task;
Console.WriteLine(result); // 42

// Task.WhenAll - concurrency
await Task.WhenAll(Task.Delay(1000), Task.Delay(1000)); // takes ~1s not 2s
```

Q15. async vs await differences

✓ Answer:

async marks a method as asynchronous — it does NOT run it on a separate thread. await suspends execution of the async method until the awaited Task completes, freeing the calling thread meanwhile.

```
// async keyword: tells compiler to create state machine for this method
public async Task<string> FetchDataAsync(string url)
{
    // await: suspends HERE, thread is free for other work
    var response = await httpClient.GetAsync(url);        // I/O wait
    var content  = await response.Content.ReadAsStringAsync();
    return content; // resumes here after I/O completes
}

// Calling code:
string data = await FetchDataAsync("https://api.example.com");

// WRONG: .Result blocks the thread (can cause deadlock in UI/ASP.NET)
// string data = FetchDataAsync("...").Result; // Avoid!
```

🔗 async/await = non-blocking I/O. Thread is returned to pool while waiting. NOT the same as running on a new thread.

Q16. Explain static class

✓ Answer:

A static class cannot be instantiated. All members must be static. Use for utility/helper methods, extension methods, and constants.

```
public static class MathHelper
{
    public static double CircleArea(double r) => Math.PI * r * r;
    public static int Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);
}

// Usage — no instantiation needed
double area = MathHelper.CircleArea(5.0);

// Static class cannot:
// - Be instantiated: new MathHelper() // ERROR
// - Be inherited: class Sub : MathHelper // ERROR
// - Have instance members

// Static class CAN:
// - Have static constructors (called once)
// - Have extension methods (must be in static class)
```

Q17. Explain middleware (ASP.NET)

✓ Answer:

Middleware is software assembled into a pipeline to handle HTTP requests and responses. Each component calls the next middleware or short-circuits the pipeline.

```
// Program.cs — middleware pipeline
var app = builder.Build();

app.UseHttpsRedirection(); // 1st
app.UseAuthentication();   // 2nd
app.UseAuthorization();    // 3rd
app.MapControllers();      // 4th — terminal

// Custom middleware using Use:
app.Use(async (context, next) => {
    Console.WriteLine("Request: " + context.Request.Path);
});
```

```

    await next(); // call next middleware
    Console.WriteLine("Response: " + context.Response.StatusCode);
});

// Custom middleware class:
public class TimingMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext context) {
        var sw = Stopwatch.StartNew();
        await next(context);
        sw.Stop();
        Console.WriteLine($"Request took {sw.ElapsedMilliseconds}ms");
    }
}

```

Q18. Explain temp table usage

✓ Answer:

Temporary tables in SQL Server store intermediate results within a session. They are stored in tempdb.

Type	Syntax	Scope
Local temp table	#TempTable	Current session only
Global temp table	##TempTable	All sessions until creator disconnects
Table variable	@TableVar	Current batch/procedure only

```

-- Create and use a local temp table
CREATE TABLE #HighSalaryEmp (
    EmpID INT,
    EmpName VARCHAR(100),
    Salary DECIMAL(10,2)
);

INSERT INTO #HighSalaryEmp
SELECT EmpID, EmpName, Salary
FROM Employees
WHERE Salary > 80000;

SELECT * FROM #HighSalaryEmp ORDER BY Salary DESC;

DROP TABLE #HighSalaryEmp; -- cleanup (optional, auto-dropped at end of session)

```

Q19. Can temp tables be used in functions?

✓ Answer:

Local temp tables (#TempTable) CANNOT be used inside scalar or inline table-valued functions in SQL Server. Table variables (@var) CAN be used. Multi-statement table-valued functions allow table variables only.

```

-- WRONG: Cannot use #TempTable in a function
-- CREATE FUNCTION dbo.GetHighSalary()
-- RETURNS TABLE AS
-- BEGIN
--     CREATE TABLE #Temp (...); -- ERROR!

-- CORRECT: Use table variable inside function
CREATE FUNCTION dbo.GetHighSalary()
RETURNS @Result TABLE (EmpID INT, EmpName VARCHAR(100))
AS
BEGIN
    INSERT INTO @Result
    SELECT EmpID, EmpName FROM Employees WHERE Salary > 80000;

```



```
RETURN;  
END
```

Q20. Can temp tables be used in triggers?

✓ Answer:

YES — temp tables (#TempTable) CAN be used inside triggers. Triggers are essentially stored procedures and run in the same session context.

```
CREATE TRIGGER trg_AfterInsertEmployee  
ON Employees  
AFTER INSERT  
AS  
BEGIN  
    -- Temp table IS allowed in triggers  
    CREATE TABLE #NewEmps (EmpID INT, EmpName VARCHAR(100));  
  
    INSERT INTO #NewEmps  
    SELECT EmpID, EmpName FROM INSERTED; -- INSERTED magic table  
  
    -- Process new employees  
    INSERT INTO AuditLog (Action, EmpID, LogDate)  
    SELECT 'INSERT', EmpID, GETDATE() FROM #NewEmps;  
  
    DROP TABLE #NewEmps;  
END
```

💡 *INSERTED and DELETED are magic tables (virtual tables) available only inside triggers.*

SECTION D: Exception Handling & API (Q21–Q25)

Q21. Exception handling in C#, React, SQL

✓ Answer:

C# — try/catch/finally:

```
// C# Exception Handling  
try {  
    int result = int.Parse("not a number");  
}  
catch (FormatException ex) {  
    Console.WriteLine($"Format error: {ex.Message}");  
}  
catch (Exception ex) {  
    Console.WriteLine($"General error: {ex.Message}");  
    throw; // re-throw preserving stack trace  
}  
finally {  
    Console.WriteLine("Always runs - close resources here");  
}  
  
// Global exception handler in .NET Core (Program.cs):  
app.UseExceptionHandler(err => err.Run(async ctx => {  
    ctx.Response.StatusCode = 500;  
    await ctx.Response.WriteAsJsonAsync(new { error = "Internal server error" });  
}));
```

React — Error Boundaries and try/catch in async:

```
// React Error Boundary (class component)  
class ErrorBoundary extends React.Component {  
    state = { hasError: false };  
}
```

```

static getDerivedStateFromError() { return { hasError: true }; }
componentDidCatch(error, info) { console.error(error, info); }
render() {
  return this.state.hasError
    ? <h2>Something went wrong.</h2>
    : this.props.children;
}
}

// Async error handling in React
const fetchData = async () => {
  try {
    const res = await fetch('/api/data');
    if (!res.ok) throw new Error('Network response was not ok');
    const data = await res.json();
    setData(data);
  } catch (err) {
    setError(err.message);
  }
};

```

SQL — TRY/CATCH:

```

-- SQL Server TRY/CATCH with transaction
BEGIN TRY
  BEGIN TRANSACTION;
  INSERT INTO Employees (Name, Salary) VALUES ('Alice', 75000);
  INSERT INTO Departments (EmpID, DeptID) VALUES (SCOPE_IDENTITY(), 5);
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  ROLLBACK TRANSACTION;
  SELECT
    ERROR_NUMBER()      AS ErrorNumber,
    ERROR_MESSAGE()     AS ErrorMessage,
    ERROR_PROCEDURE()   AS ErrorProcedure;
END CATCH

```

Q22. REST API vs traditional API

✓ **Answer:**

Feature	Traditional API (SOAP/RPC)	REST API
Protocol	SOAP (XML), RPC	HTTP (JSON/XML)
State	Can be stateful	Stateless
Format	Strict XML schema (WSDL)	Flexible JSON/XML/YAML
Coupling	Tightly coupled	Loosely coupled
Caching	Not built-in	HTTP caching headers supported
Standards	WS-Security, WS-ReliableMsg	HTTP standards (TLS, OAuth)

Q23. API versioning implementation

✓ **Answer:**

Three main strategies: URL path, query string, and header versioning.

```

// Install: dotnet add package Asp.Versioning.Mvc

// Program.cs
builder.Services.AddApiVersioning(opt => {
  opt.DefaultApiVersion = new ApiVersion(1, 0);
});

```

```

    opt.AssumeDefaultVersionWhenUnspecified = true;
    opt.ReportApiVersions = true;
    opt.ApiVersionReader = ApiVersionReader.Combine(
        new UrlSegmentApiVersionReader(),
        new HeaderApiVersionReader("X-API-Version"),
        new QueryStringApiVersionReader("api-version")
    );
});

// Controller v1
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV1Controller : ControllerBase {
    [HttpGet] public IActionResult Get() => Ok("Products V1");
}

// Controller v2
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV2Controller : ControllerBase {
    [HttpGet] public IActionResult Get() => Ok("Products V2 - Enhanced");
}

// Calls:
// GET /api/v1/products => V1
// GET /api/v2/products => V2
// GET /api/products?api-version=1.0 => V1 (query string)

```

Q24. ASP.NET Filters

✓ Answer:

Filters run at specific stages of the MVC pipeline. They allow cross-cutting concerns (logging, auth, caching) without putting code in every action.

Filter Type	When it Runs	Common Use
Authorization Filter	Before anything else	JWT validation, role checks
Resource Filter	After authorization, before model binding	Caching, short-circuit
Action Filter	Before & after action executes	Logging, validation
Result Filter	Before & after result executes	Response formatting
Exception Filter	When unhandled exception occurs	Global error handling

```

// Custom Action Filter
public class LogActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
        => Console.WriteLine($"Executing: {context.ActionDescriptor.DisplayName}");

    public void OnActionExecuted(ActionExecutedContext context)
        => Console.WriteLine($"Executed: {context.ActionDescriptor.DisplayName}");
}

// Register globally in Program.cs:
builder.Services.AddControllers(opt =>
    opt.Filters.Add<LogActionFilter>());

// Or per-controller/action:
[ServiceFilter(typeof(LogActionFilter))]

```

```
public class OrdersController : ControllerBase { }
```

Q25. Custom middleware creation

✓ Answer:

```
// Custom middleware - request logging + timing
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(RequestDelegate next,
        ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
        _logger.LogInformation("{Method} {Path} started",
            context.Request.Method, context.Request.Path);

        await _next(context); // call next middleware

        sw.Stop();
        _logger.LogInformation("{Method} {Path} => {Status} in {Ms}ms",
            context.Request.Method, context.Request.Path,
            context.Response.StatusCode, sw.ElapsedMilliseconds);
    }
}

// Extension method for clean registration:
public static class MiddlewareExtensions
{
    public static IApplicationBuilder UseRequestLogging(
        this IApplicationBuilder app) =>
        app.UseMiddleware<RequestLoggingMiddleware>();
}

// Program.cs:
app.UseRequestLogging();
```

SECTION E: React & JavaScript (Q26–Q38)

Q26. Alternative for useEffect in React

✓ Answer:

React 19 introduced the use() hook and Actions. But also consider: useLayoutEffect (sync DOM reads), useMemo (derived values), useCallback (stable refs), React Query / SWR (data fetching), and custom hooks.

```
// useEffect - side effects
useEffect(() => { fetchData(); }, [id]);

// Alternative 1: React Query (preferred for data fetching)
import { useQuery } from '@tanstack/react-query';
const { data, isLoading } = useQuery({
    queryKey: ['user', id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json())
});
```

```
// Alternative 2: useEffect (fires synchronously after DOM mutations)
useLayoutEffect(() => {
  // Measure DOM element before paint
  const width = ref.current.offsetWidth;
  setWidth(width);
}, []);

// Alternative 3: Custom hook encapsulating the effect
function useFetchUser(id) {
  const [user, setUser] = useState(null);
  useEffect(() => { fetchUser(id).then(setUser); }, [id]);
  return user;
}
```

Q27. public static string MergeStrings(string s1, string s2) — implementation

✓ **Answer:**

Interleave characters from both strings, appending the remainder of the longer string.

```
public static string MergeStrings(string s1, string s2)
{
    var sb = new StringBuilder();
    int maxLen = Math.Max(s1.Length, s2.Length);
    for (int i = 0; i < maxLen; i++)
    {
        if (i < s1.Length) sb.Append(s1[i]);
        if (i < s2.Length) sb.Append(s2[i]);
    }
    return sb.ToString();
}

// Test:
Console.WriteLine(MergeStrings("abc", "xyz")); // axbycz
Console.WriteLine(MergeStrings("ab", "xyzw")); // axbyzw
Console.WriteLine(MergeStrings("abcd", "xy")); // axbycd
```

Q28. JS object with greet() using this — output question

✓ **Answer:**

```
const object = {
  who: 'World',
  greet() { return `Hello, ${this.who}!`; },
  farwell: function() { return `Goodbye, ${this.who}!`; }
};

console.log(object.greet()); // Hello, World!
console.log(object.farwell()); // Goodbye, World!

// If destructured — 'this' loses context:
const { greet } = object;
// greet(); // Hello, undefined! — 'this' is now global/undefined in strict mode

// Fix with bind:
const boundGreet = object.greet.bind(object);
console.log(boundGreet()); // Hello, World!
```

Q29. IIFE (immediately invoked function expression) — output

✓ **Answer:**

An IIFE is a function defined and called immediately. Common use: create a private scope.

```
(function immediateA(a) {
```

```

    return (function immediateB(b) {
        return a + b;
    })(1);
})(0);

// Trace:
// immediateA is called with a = 0
// immediateB is called with b = 1
// returns 0 + 1 = 1

// IIFE use cases:
(function() {
    const privateVar = 'secret'; // not accessible outside
    console.log('IIFE executed');
})();

// Modern equivalent - block scoping with const/let:
{
    const privateVar = 'secret';
}

```

Q30. `const fruits=['jackfruit','mango','apple']` — modify array

✓ **Answer:**

`const` means the variable binding cannot be reassigned, but the array contents can be mutated.

```

const fruits = ['jackfruit', 'mango', 'apple'];

// These ALL work - mutating content is allowed with const:
fruits.push('banana'); // ['jackfruit', 'mango', 'apple', 'banana']
fruits.pop();          // removes 'banana'
fruits[0] = 'watermelon'; // change element
fruits.sort();         // ['apple', 'jackfruit', 'mango']

// This FAILS - reassignment is not allowed:
// fruits = ['durian']; // TypeError: Assignment to constant variable

// To truly freeze, use Object.freeze:
const frozen = Object.freeze(['a', 'b', 'c']);
frozen.push('d'); // silent fail in non-strict, TypeError in strict mode

```

Q31. `a = 10; var a; console.log(a);` — output?

✓ **Answer:**

Output: 10. Due to JavaScript hoisting, `'var a'` is hoisted to the top of its scope (function/global). The declaration is hoisted but NOT the assignment. So `'a = 10'` runs normally.

```

a = 10;
var a;
console.log(a); // Output: 10

// JavaScript sees this as:
var a; // hoisted to top (declaration only)
a = 10; // assignment stays here
console.log(a); // 10

// Contrast with let/const - NOT hoisted (Temporal Dead Zone):
// console.log(b); // ReferenceError: Cannot access 'b' before initialization
// let b = 5;

```

Q32. Write a function to convert camelCase to snake_case

✓ **Answer:**

```
// JavaScript
function camelToSnake(str) {
    return str
        .replace(/[A-Z]/g, '_$1') // insert _ before each uppercase letter
        .toLowerCase();           // convert all to lowercase
}

console.log(camelToSnake('helloWorld')); // hello_world
console.log(camelToSnake('getUserByIdAsync')); // get_user_by_id_async

// C# equivalent:
static string CamelToSnake(string input) =>
    Regex.Replace(input, @"(?<!(^)[A-Z])", "_$1").ToLower();
```

Q33. Flatten nested array

✓ Answer:

```
// JavaScript
const nested = [1, [2, [3, [4]], 5]];

// Method 1: flat() with Infinity depth
console.log(nested.flat(Infinity)); // [1, 2, 3, 4, 5]

// Method 2: Recursive
function flatten(arr) {
    return arr.reduce((acc, val) =>
        Array.isArray(val) ? acc.concat(flatten(val)) : acc.concat(val), []);
}

// C# - recursive flatten
IEnumerable<int> Flatten(IEnumerable<object> items) =>
    items.SelectMany(item =>
        item is IEnumerable<object> nested
            ? Flatten(nested)
            : new[] { (int)item });
```

Q34. let a = '3' + 3 + 3 = ?

✓ Answer:

```
let a = '3' + 3 + 3;
// Step 1: '3' + 3 => '33' (string + number = string concatenation)
// Step 2: '33' + 3 => '333' (string + number = string concatenation)
console.log(a); // '333' (string)
```

Q35. let b = 3 + '3' + 3 = ?

✓ Answer:

```
let b = 3 + '3' + 3;
// Step 1: 3 + '3' => '33' (number + string = string concatenation)
// Step 2: '33' + 3 => '333' (string + number = string concatenation)
console.log(b); // '333' (string)
```

Q36. let c = 3 + 3 + '3' = ?

✓ Answer:

```
let c = 3 + 3 + '3';
// Step 1: 3 + 3 => 6 (number + number = addition)
// Step 2: 6 + '3' => '63' (number + string = string concatenation)
console.log(c); // '63' (string)
```

Q37. let d = '3' + 3 - 3 = ?

✓ Answer:

```
let d = '3' + 3 - 3;
// Step 1: '3' + 3 => '33' (string concatenation)
// Step 2: '33' - 3 => 30 (- coerces string to number: 33 - 3 = 30)
console.log(d); // 30 (number)

// Key insight: + is overloaded (concat OR add), but - is always numeric
```

Q38. let e = 3 + 3 + 3 = ?

✓ Answer:

```
let e = 3 + 3 + 3;
// Step 1: 3 + 3 = 6
// Step 2: 6 + 3 = 9
console.log(e); // 9 (number)
```

Quick Reference: JS Type Coercion Summary

Expression	Result	Type	Reason
'3' + 3 + 3	'333'	string	Left-to-right: string concat first
3 + '3' + 3	'333'	string	Once string joins, stays string
3 + 3 + '3'	'63'	string	Numbers add first, then concat
'3' + 3 - 3	30	number	Concat to '33', then - coerces to 30
3 + 3 + 3	9	number	All numeric addition

End of VIRTUSA Interview Q&A — Part 6

DBIZ.AI

Interview Q&A — Factory Pattern & DI Deep Dive

15 Questions · File Upload · SignalR · Factory vs DI · EF Core · Race Conditions · Unit Testing

Section A: File Upload Progress (Q1–Q2)

Q1. File Upload Progress: Frontend wants to show processing progress as percentage. How to achieve it?

✓ **Answer:**

Three approaches in order of suitability: SignalR (real-time push), Server-Sent Events (SSE), or Polling. For a long-running backend task, SignalR is the best choice.

Approach 1 — SignalR (Recommended for real-time progress):

```
// — Backend: Hub + Background Service —————
// Install: dotnet add package Microsoft.AspNetCore.SignalR

// SignalR Hub
public class UploadProgressHub : Hub
{
    // Client connects and waits for progress events
    // Server pushes progress updates via SendAsync
}

// Upload Controller — starts background processing
[ApiController]
[Route("api/[controller]")]
public class UploadController : ControllerBase
{
    private readonly IHubContext<UploadProgressHub> _hub;
    private readonly IBackgroundTaskQueue _queue;

    public UploadController(
        IHubContext<UploadProgressHub> hub,
        IBackgroundTaskQueue queue)
    {
        _hub = hub;
        _queue = queue;
    }

    [HttpPost]
    public async Task<IActionResult> Upload(IFormFile file)
    {
        string jobId = Guid.NewGuid().ToString();

        // Queue background task — don't block HTTP response
        _queue.Enqueue(async ct =>
        {
            // Simulate file processing in stages
            for (int progress = 0; progress <= 100; progress += 10)
            {
```

```

        await Task.Delay(1500, ct); // simulate work

        // Push progress to connected client
        await _hub.Clients.Group(jobId)
            .SendAsync("ProgressUpdate", new {
                jobId,
                progress,
                status = progress == 100 ? "Completed" : "Processing",
                message = $"Processing records... {progress}%"
            }, ct);
    }
});

// Return jobId immediately - client subscribes using this
return Accepted(new { jobId });
}
}

// Program.cs
builder.Services.AddSignalR();
app.MapHub<UploadProgressHub>("/hubs/upload-progress");

```

Frontend — Angular SignalR Client:

```

// Install: npm install @microsoft/signalr

import * as signalR from '@microsoft/signalr';

@Component({ selector: 'app-upload', ... })
export class UploadComponent {
    progress = 0;
    status = '';
    private connection!: signalR.HubConnection;

    async uploadFile(file: File) {
        // 1. Upload file and get jobId
        const form = new FormData();
        form.append('file', file);
        const res = await this.http.post<{jobId:string}>('/api/upload', form).toPromise();
        const jobId = res!.jobId;

        // 2. Connect to SignalR hub
        this.connection = new signalR.HubConnectionBuilder()
            .withUrl('/hubs/upload-progress')
            .withAutomaticReconnect()
            .build();

        // 3. Listen for progress updates
        this.connection.on('ProgressUpdate', (data) => {
            this.progress = data.progress;
            this.status = data.message;
            if (data.progress === 100) {
                this.connection.stop(); // clean up after completion
            }
        });

        await this.connection.start();
        await this.connection.invoke('JoinGroup', jobId);
    }
}

```

🔔 *SignalR falls back to Long Polling automatically if WebSockets are unavailable. Always return jobId immediately so the UI is not blocked.*

Q2. Is polling every 15–20 seconds efficient for a task that takes ~15 minutes?

✓ Answer:

No — polling every 15–20 seconds for a 15-minute task is highly inefficient and wastes server resources.

Approach	Requests Made (15 min)	Efficiency	Best For
Polling every 15s	~60 requests	✗ Wasteful — most return 'still processing'	Simple, no infra requirement
Polling every 60s	~15 requests	⚠ Better but still pulls unnecessarily	Low-frequency status checks
SignalR (push)	0 poll requests	✓ Server pushes only when progress changes	Real-time progress bars
Server-Sent Events	1 connection	✓ Server streams updates over one connection	One-way server→client stream
Webhooks	1 call on done	✓ Zero waste — callback when complete	External systems, completion only

Recommended strategy for a 15-minute task:

- Use SignalR or SSE — server pushes updates only when something changes (e.g., every 10% progress)
- If polling is required (legacy/simple setup), use exponential backoff — start at 5s, increase to 30s, 60s
- Always include a job ID so clients can resume tracking after reconnection

```
// Exponential backoff polling (if SignalR not available)
async pollWithBackoff(jobId: string) {
  let interval = 5000; // start at 5 seconds
  const maxInterval = 60000; // cap at 60 seconds

  const poll = async () => {
    const res = await this.http.get<{progress:number, done:boolean}>(`
      /api/jobs/${jobId}/status`).toPromise();

    this.progress = res!.progress;
    if (!res!.done) {
      interval = Math.min(interval * 1.5, maxInterval); // backoff
      setTimeout(poll, interval);
    }
  };
  poll();
}
```

Section B: Factory Pattern vs Dependency Injection (Q3–Q6)

Q3. Factory Pattern Output: What output do you expect when you call a factory class?

✓ Answer:

A factory class returns an object instance based on logic — typically via a static or instance method. The caller does not use 'new' directly.

```
// — Factory Pattern —————
public interface INotificationService
{
  void Send(string message);
}
```

```

}

public class EmailNotification : INotificationService
{
    public void Send(string message)
        => Console.WriteLine($"[EMAIL] {message}");
}

public class SmsNotification : INotificationService
{
    public void Send(string message)
        => Console.WriteLine($"[SMS] {message}");
}

public class PushNotification : INotificationService
{
    public void Send(string message)
        => Console.WriteLine($"[PUSH] {message}");
}

// Factory – creates correct implementation based on input
public static class NotificationFactory
{
    public static INotificationService Create(string type) => type switch
    {
        "email" => new EmailNotification(),
        "sms"   => new SmsNotification(),
        "push"  => new PushNotification(),
        _       => throw new ArgumentException($"Unknown type: {type}")
    };
}

// — What you get back (output) —————
var service = NotificationFactory.Create("email");
// Returns: an instance of EmailNotification (as INotificationService)

service.Send("Your order is confirmed!");
// Console output: [EMAIL] Your order is confirmed!

var sms = NotificationFactory.Create("sms");
sms.Send("OTP: 123456");
// Console output: [SMS] OTP: 123456

```

 *The factory returns an `INotificationService` reference — the caller works with the abstraction and doesn't care about the concrete type.*

Q4. DI Constructor Output: You request `IEmployeeService` in constructor-based DI. What do you get back?

Answer:

You get back the concrete implementation that was registered in the DI container — but typed as the interface. The DI container resolves the dependency automatically.

```

// Registration in Program.cs
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
//                      ↑ interface      ↑ concrete class

// Constructor injection – DI resolves IEmployeeService
public class EmployeesController : ControllerBase
{
    private readonly IEmployeeService _employeeService;

    // DI container sees IEmployeeService, looks up registration,

```

```
// creates EmployeeService instance, and injects it here
public EmployeesController(IEmployeeService employeeService)
{
    _employeeService = employeeService;
    // _employeeService is actually an EmployeeService instance
    // but typed as IEmployeeService
}

// What you GET BACK:
// - Reference type: IEmployeeService
// - Actual object: EmployeeService (the concrete class registered)
// - Lifetime: Scoped (one per HTTP request, as registered)

// Verify at runtime:
Console.WriteLine(_employeeService.GetType().Name); // EmployeeService
Console.WriteLine(_employeeService is IEmployeeService); // True
```

Q5. Factory vs DI: Why would you choose DI over Factory? They both do the same thing.

✓ Answer:

They are NOT the same. Factory pattern creates objects manually; DI is managed by the container and solves lifetime, testability, and dependency chain problems automatically.

Aspect	Factory Pattern	Dependency Injection
Object creation	Manual — you write the creation logic	Automatic — container resolves entire graph
Dependency chains	You manage nested dependencies manually	Container resolves all transitive deps automatically
Lifetime mgmt	No concept — you manage it yourself	Singleton/Scoped/Transient handled by container
Testability	Hard to mock — factory creates concrets	Easy — inject mock via interface in tests
Coupling	Code depends on Factory class directly	Code depends only on interface
Config/startup	Logic scattered across factories	Single registration point (Program.cs)
Best used when	Runtime type selection (plugin system)	Known dependencies at startup time

```
// — Factory shortcoming: nested dependency hell —————
// If EmployeeService depends on ILogger, IRepository, ICache...
// Factory must know about all of them:
public static IEmployeeService Create()
{
    var logger = new Logger<EmployeeService>(loggerFactory);
    var repository = new EmployeeRepository(dbContext); // dbContext from where??
    var cache = new RedisCacheService(connectionString);
    return new EmployeeService(logger, repository, cache); // manual, brittle
}

// — DI: container handles the entire dependency graph —————
// You just declare what you need:
public class EmployeeService : IEmployeeService
{
    public EmployeeService(
        ILogger<EmployeeService> logger, // auto-resolved
```

```

        IEmployeeRepository repo,                // auto-resolved
        IDistributedCache cache)                // auto-resolved
    { }
}
// Registration — container wires it all up:
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
// All nested dependencies (Logger, Repo, Cache) auto-injected!

```

Q6. Parameterized DI: Have you heard of it? Doesn't that make factory pattern redundant?

✓ Answer:

Parameterized DI (runtime parameters) is when you need to pass dynamic values (not registered services) to a constructor. This is the one case where factory still has a role alongside DI.

```

// Scenario: OrderProcessor needs a dynamic 'currency' parameter at runtime
public class OrderProcessor : IOrderProcessor
{
    private readonly IOrderRepository _repo;
    private readonly string _currency; // runtime value — not a DI service

    public OrderProcessor(IOrderRepository repo, string currency)
    {
        _repo = repo;
        _currency = currency;
    }
}

// — Solution 1: Factory Function via DI (Func<string, IOrderProcessor>) —
builder.Services.AddTransient<IOrderRepository, OrderRepository>();
builder.Services.AddTransient<Func<string, IOrderProcessor>>(sp => currency =>
    new OrderProcessor(
        sp.GetRequiredService<IOrderRepository>(), // from DI
        currency // runtime param
    ));

// Consumer:
public class CheckoutService
{
    private readonly Func<string, IOrderProcessor> _processorFactory;

    public CheckoutService(Func<string, IOrderProcessor> factory)
        => _processorFactory = factory;

    public async Task Checkout(Cart cart, string currency)
    {
        // Create processor with runtime parameter, DI handles the rest
        var processor = _processorFactory(currency);
        await processor.ProcessAsync(cart);
    }
}

// — Solution 2: Options pattern for config params —————
// Better when parameters are configuration, not truly runtime:
builder.Services.Configure<OrderOptions>(o => o.DefaultCurrency = "USD");

public class OrderProcessor : IOrderProcessor
{
    private readonly IOptions<OrderOptions> _options;
    public OrderProcessor(IOrderRepository repo, IOptions<OrderOptions> options) { }
}

```

💡 *Factory pattern is NOT made redundant — it complements DI for runtime type selection and parameterized object creation. Use DI for static dependencies, factory for dynamic runtime decisions.*

Section C: EF Core Deep Dive (Q7–Q11)

Q7. EF Core Fetch by PK: DbContext, table Employees. Want employee with Id = 5. How do you do it?

✓ Answer:

Three ways — Find() is the best for primary key lookups because it checks the local cache (1st level cache) before hitting the database.

```
// — Method 1: Find() — BEST for PK lookup —————
// Checks DbContext's local identity cache FIRST
// Only goes to DB if not already tracked in memory
var emp1 = await _context.Employees.FindAsync(5);
// Returns Employee? — null if not found

// — Method 2: FirstOrDefaultAsync with Where —————
// Always goes to DB — no cache check
// Use when you need to filter on non-PK columns
var emp2 = await _context.Employees
    .FirstOrDefaultAsync(e => e.Id == 5);

// — Method 3: SingleOrDefaultAsync —————
// Goes to DB — asserts exactly 0 or 1 result
// Throws InvalidOperationException if more than 1 match
var emp3 = await _context.Employees
    .SingleOrDefaultAsync(e => e.Id == 5);

// — When to use which —————
// FindAsync → PK lookup, benefits from cache, fast
// FirstOrDefaultAsync → PK or non-PK, don't care if duplicates exist
// SingleOrDefaultAsync → Assert uniqueness (will throw if 2+ results)
```

Method	Checks Cache?	Throws if Multiple?	Best For
FindAsync(id)	✓ Yes — checks identity map	N/A — only PK match	PK lookups — fastest
FirstOrDefaultAsync()	✗ No — always hits DB	✓ No — takes first	Any column filter
SingleOrDefaultAsync()	✗ No — always hits DB	✗ Yes — throws	Assert unique result

Q8. Select vs Where: What is the difference between Select and Where in LINQ?

✓ Answer:

LINQ Method	Purpose	Returns	SQL Equivalent
Where()	Filter rows — keep only rows matching the condition	IEnumerable<T> — same type	WHERE clause
Select()	Project / transform — shape or transform each element	IEnumerable<TResult> — new type	SELECT clause

```
var employees = new List<Employee> {
    new Employee { Id=1, Name="Alice", Dept="IT", Salary=80000 },
    new Employee { Id=2, Name="Bob", Dept="HR", Salary=60000 },
}
```

```

    new Employee { Id=3, Name="Carol", Dept="IT", Salary=90000 },
};

// WHERE - filter rows (same type in, same type out)
var itEmployees = employees.Where(e => e.Dept == "IT");
// Returns: Alice, Carol (still Employee objects)

// SELECT - project/transform (change the shape)
var names = employees.Select(e => e.Name);
// Returns: ["Alice", "Bob", "Carol"] - IEnumerable<string>

// SELECT anonymous type - project to a new shape
var summary = employees.Select(e => new { e.Name, e.Salary });
// Returns: [{ Name='Alice', Salary=80000 }, ...]

// COMBINING: Where + Select (filter THEN project)
var itNames = employees
    .Where(e => e.Dept == "IT")           // filter first
    .Select(e => e.Name.ToUpper());       // then transform
// Returns: ["ALICE", "CAROL"]

// SQL equivalent:
// SELECT e.Name FROM Employees e WHERE e.Dept = 'IT'

```

Q9. Find Method: What is the Find method? When use Find vs Where?

✓ Answer:

Find() is a DbSet method (not LINQ) that looks up an entity by primary key. It checks the change tracker (in-memory cache) FIRST before going to the database — making it faster for PK lookups.

```

// Find - primary key only, checks memory cache first
var emp = await _context.Employees.FindAsync(5);
// 1. Checks DbContext identity map → if found in memory, returns immediately (no DB call)
// 2. If not in memory → executes SELECT * FROM Employees WHERE Id = 5

// Where - general purpose LINQ, ALWAYS goes to DB
var emp2 = await _context.Employees
    .Where(e => e.Id == 5)
    .FirstOrDefaultAsync();
// Always executes: SELECT TOP 1 * FROM Employees WHERE Id = 5

// — Key difference demo —
// Scenario: you already loaded the employee earlier in the same request
var emp3 = await _context.Employees.FindAsync(5); // hits DB
var emp4 = await _context.Employees.FindAsync(5); // returns from cache - NO DB call!

// With Where: ALWAYS hits DB even if already cached
var emp5 = await _context.Employees.FirstOrDefaultAsync(e => e.Id == 5); // DB call 1
var emp6 = await _context.Employees.FirstOrDefaultAsync(e => e.Id == 5); // DB call 2

// — When to use which —
// Use Find:  fetching by exact PK, may be fetched multiple times
// Use Where: filtering by non-PK columns, complex conditions, joins

```

🔗 Find() does not support AsNoTracking() — it always tracks the entity. If you need read-only fetch by PK without tracking, use FirstOrDefaultAsync with AsNoTracking().

Q10. EF Core Isolation Levels: What are isolation levels supported in EF Core?

✓ Answer:

EF Core supports all SQL Server isolation levels via IsolationLevel enum, applied when beginning a transaction.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Use When
ReadUncommitted	✓ Allowed	✓ Allowed	✓ Allowed	Max performance, reporting — dirty data OK
ReadCommitted	✗ Prevented	✓ Allowed	✓ Allowed	Default in SQL Server — balanced
RepeatableRead	✗ Prevented	✗ Prevented	✓ Allowed	Need same row to read the same twice
Serializable	✗ Prevented	✗ Prevented	✗ Prevented	Max safety — financial, banking — slowest
Snapshot	✗ Prevented	✗ Prevented	✗ Prevented	Read consistent snapshot without blocking writers

```
// EF Core — begin transaction with specific isolation level
using var transaction = await _context.Database.BeginTransactionAsync(
    System.Data.IsolationLevel.RepeatableRead);

try
{
    var emp = await _context.Employees.FindAsync(5);
    emp!.Salary += 5000; // guaranteed same value if read again

    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}

// ReadUncommitted — fastest, for reporting queries
using var readTx = await _context.Database.BeginTransactionAsync(
    System.Data.IsolationLevel.ReadUncommitted);
// SELECT with NOLOCK equivalent
```

Q11. AsNoTracking: What does AsNoTracking() mean?

✓ Answer:

AsNoTracking() tells EF Core NOT to add fetched entities to the change tracker. This improves performance for read-only queries because EF skips snapshot creation and change detection.

Aspect	With Tracking (default)	With AsNoTracking()
Change tracking	EF stores snapshot of each entity	No snapshot — entity not tracked
Memory usage	Higher — stores original values	Lower — no overhead
Performance	Slower for large result sets	Faster — especially for many rows
SaveChanges()	Detects changes and persists them	Changes ignored — not persisted

Use when	Read-write: update/delete scenarios	Read-only: reports, GET endpoints, lists
----------	-------------------------------------	--

```
// — Tracking (default) — use for update/delete —————
var emp = await _context.Employees.FindAsync(5);
emp!.Salary = 90000; // EF detects the change
await _context.SaveChangesAsync(); // generates UPDATE SQL

// — AsNoTracking — use for read-only lists/reports —————
var employees = await _context.Employees
    .AsNoTracking() // don't track these entities
    .Where(e => e.Dept == "IT")
    .ToListAsync();
// employees[0].Salary = 99999;
// await _context.SaveChangesAsync(); // NO UPDATE generated — not tracked

// — Performance impact (real-world) —————
// For 10,000 rows:
//   With tracking: ~180ms (snapshot overhead)
//   AsNoTracking: ~95ms (~47% faster)

// — AsNoTrackingWithIdentityResolution —————
// Use when: read-only + entities reference each other (avoid duplicates)
var orders = await _context.Orders
    .Include(o => o.Customer)
    .AsNoTrackingWithIdentityResolution() // deduplicates Customer objects
    .ToListAsync();
```

Section D: DI Lifetimes, LINQ GroupBy, Race Conditions (Q12–Q14)

Q12. DI Lifetimes: What are the lifetimes of an object in DI? When would you use which one?

✓ Answer:

Lifetime	Created	Disposed	Shared By	Register With
Singleton	Once at first request	App shutdown	All requests + users	AddSingleton<>()
Scoped	Once per HTTP request	End of request	Same request only	AddScoped<>()
Transient	Every .GetService() call	After each use	Nobody — always fresh	AddTransient<>()

```
// — When to use each —————

// SINGLETON: one instance, lives forever
// Use for: config, memory cache, HttpClient, thread-safe counters
builder.Services.AddSingleton<IAppSettings, AppSettings>();
builder.Services.AddSingleton<IMemoryCache, MemoryCache>();

// SCOPED: one instance per HTTP request
// Use for: EF Core DbContext, repositories, unit-of-work, current user
builder.Services.AddScoped<AppDbContext>();
builder.Services.AddScoped<IEmployeeRepository, EmployeeRepository>();
builder.Services.AddScoped<ICurrentUserService, CurrentUserService>();

// TRANSIENT: fresh instance every time
```

```
// Use for: lightweight stateless services, email formatters, validators
builder.Services.AddTransient<IEmailFormatter, HtmlEmailFormatter>();
builder.Services.AddTransient<IValidator<Employee>, EmployeeValidator>();

// — Captive Dependency Bug —————
// ✗ WRONG: Singleton holding Scoped service
// The Scoped service gets captured for the entire app lifetime!
// builder.Services.AddSingleton<MyService>();
// public class MyService { MyService(IEmployeeRepository repo) } // SCOPED!

// ✓ FIX: Use IServiceScopeFactory inside Singleton
public class MyBackgroundService : BackgroundService
{
    private readonly IServiceScopeFactory _factory;
    public MyBackgroundService(IServiceScopeFactory factory) => _factory = factory;

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using var scope = _factory.CreateScope();
        var repo = scope.ServiceProvider.GetRequiredService<IEmployeeRepository>();
        // use repo safely – proper scoped lifetime
    }
}
```

Q13. LINQ GroupBy: Write a LINQ query to get employee count grouped by department

✓ Answer:

```
// — Using LINQ method syntax —————
var deptCounts = employees
    .GroupBy(e => e.Department)
    .Select(g => new
    {
        Department      = g.Key,
        EmployeeCount    = g.Count(),
        AvgSalary        = g.Average(e => e.Salary),
        MaxSalary        = g.Max(e => e.Salary)
    })
    .OrderByDescending(x => x.EmployeeCount);

foreach (var d in deptCounts)
    Console.WriteLine($"{d.Department}: {d.EmployeeCount} employees, Avg: {d.AvgSalary:C}");

// — Output (example) —————
// IT:      45 employees, Avg: $78,500
// Finance: 30 employees, Avg: $82,000
// HR:      15 employees, Avg: $65,000

// — EF Core (DB-side GroupBy) —————
// Translates to SQL: GROUP BY Department
var result = await _context.Employees
    .GroupBy(e => e.Department)
    .Select(g => new {
        Department = g.Key,
        Count      = g.Count(),
        AvgSalary  = g.Average(e => e.Salary)
    })
    .ToListAsync();

// Generated SQL:
// SELECT Department, COUNT(*) AS Count, AVG(Salary) AS AvgSalary
// FROM Employees
// GROUP BY Department
```

```
// — LINQ query syntax (alternative) —————
var query = from e in employees
             group e by e.Department into g
             select new {
                 Department = g.Key,
                 Count      = g.Count()
             };
```

Q14. Race Condition: What is a race condition? How do you prevent it?

✓ Answer:

A race condition occurs when two or more threads access shared data concurrently and the outcome depends on the order/timing of execution — producing unpredictable, incorrect results.

```
// — Example of race condition —————
int counter = 0;

// 100 threads all incrementing counter simultaneously
Parallel.For(0, 100, i =>
{
    counter++; // NOT thread-safe! Read-Increment-Write is 3 steps
              // Two threads can read same value before either writes
});
Console.WriteLine(counter); // Could be 87, 93, 99 – not always 100!

// — Prevention Method 1: Interlocked (atomic operations) —————
int counter2 = 0;
Parallel.For(0, 100, i =>
{
    Interlocked.Increment(ref counter2); // atomic – always 100
});
Console.WriteLine(counter2); // Always 100

// — Prevention Method 2: lock (Monitor) —————
private static readonly object _lockObj = new object();
private int _balance = 1000;

public void Withdraw(int amount)
{
    lock (_lockObj) // only one thread enters at a time
    {
        if (_balance >= amount)
            _balance -= amount;
    }
}

// — Prevention Method 3: SemaphoreSlim (async-safe) —————
private readonly SemaphoreSlim _semaphore = new SemaphoreSlim(1, 1);

public async Task WithdrawAsync(int amount)
{
    await _semaphore.WaitAsync(); // async-friendly lock
    try
    {
        if (_balance >= amount)
            _balance -= amount;
    }
    finally
    {
        _semaphore.Release();
    }
}
```

```
// — Prevention Method 4: ConcurrentCollections —————
// Thread-safe by design — no manual locking needed
var safeDict    = new ConcurrentDictionary<int, string>();
var safeQueue   = new ConcurrentQueue<string>();
var safeBag     = new ConcurrentBag<int>();

// — Prevention Method 5: Optimistic Concurrency in EF Core —
// Add rowversion / concurrency token — DB rejects stale updates
public class BankAccount
{
    public int    Id        { get; set; }
    public decimal Balance { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; } // EF throws DbUpdateConcurrencyException
                                           // if another user changed row since read
}
```

Solution	Best For	Async Safe?
Interlocked	Simple counter increment/decrement	Yes
lock (Monitor)	Critical sections with complex logic	No — use SemaphoreSlim for async
SemaphoreSlim	Async-safe critical sections	Yes
ConcurrentDictionary	Thread-safe key-value store	Yes
Optimistic Concurrency	DB-level conflict detection (EF Core)	Yes

Section E: Unit Testing Fundamentals (Q15)

Q15. Unit Testing Fundamentals: What is the most fundamental thing in unit testing?

✓ Answer:

The most fundamental principle of unit testing is ISOLATION — a unit test must test ONE unit (class/method) in complete isolation from external dependencies (DB, file system, HTTP, other classes).

The 3 pillars:

- Isolation — mock/stub all dependencies so only the unit under test is exercised
- AAA Pattern — Arrange (setup), Act (execute), Assert (verify)
- Deterministic — same input always produces same output, no randomness or time-dependency

```
// — Unit Test with xUnit + Moq —————
// Install: dotnet add package xunit
//           dotnet add package Moq
//           dotnet add package FluentAssertions

public class EmployeeServiceTests
{
    private readonly Mock<IEmployeeRepository> _mockRepo;
    private readonly Mock<ILogger<EmployeeService>> _mockLogger;
    private readonly EmployeeService _sut; // System Under Test

    public EmployeeServiceTests()
    {
        _mockRepo = new Mock<IEmployeeRepository>();
    }
}
```

```

        _mockLogger = new Mock<ILogger<EmployeeService>>();
        _sut = new EmployeeService(_mockRepo.Object, _mockLogger.Object);
        // ^ EmployeeService is created with MOCKED dependencies
        // No real DB, no real logger – complete isolation
    }

    [Fact]
    public async Task GetEmployeeAsync_WhenExists_ReturnsEmployee()
    {
        // — ARRANGE —
        var expected = new Employee { Id = 5, Name = "Alice", Salary = 75000 };
        _mockRepo.Setup(r => r.GetByIdAsync(5))
            .ReturnsAsync(expected);

        // — ACT —
        var result = await _sut.GetEmployeeAsync(5);

        // — ASSERT —
        result.Should().NotBeNull();
        result!.Name.Should().Be("Alice");
        result.Salary.Should().Be(75000);
        _mockRepo.Verify(r => r.GetByIdAsync(5), Times.Once);
    }

    [Fact]
    public async Task GetEmployeeAsync_WhenNotFound_ReturnsNull()
    {
        // ARRANGE
        _mockRepo.Setup(r => r.GetByIdAsync(99)).ReturnsAsync((Employee?)null);

        // ACT
        var result = await _sut.GetEmployeeAsync(99);

        // ASSERT
        result.Should().BeNull();
    }

    [Theory]
    [InlineData(-1)]
    [InlineData(0)]
    public async Task GetEmployeeAsync_WhenInvalidId_ThrowsArgumentException(int id)
    {
        // ACT + ASSERT – test boundary conditions
        await Assert.ThrowsAsync<ArgumentException>(
            () => _sut.GetEmployeeAsync(id));
    }
}

```

Testing Principle	Explanation
ISOLATION	Mock all external dependencies — test ONLY the unit's logic
AAA Pattern	Arrange (prepare data), Act (call method), Assert (verify result)
Single Assertion	One logical assertion per test — clear failure message
Fast	Unit tests must run in milliseconds — no I/O, no DB, no HTTP
Deterministic	Same test = same result every run, any machine, any time

Naming	MethodName_Scenario_ExpectedBehavior — self-documenting tests
Test boundary cases	Happy path + null inputs + invalid inputs + edge values

💡 *The most fundamental rule: if your test touches a real database, file system, or external API — it is NOT a unit test. It is an integration test.*

DBIZ.AI | Interview Q&A — Factory Pattern & DI Deep Dive

15 Questions · SignalR · Factory vs DI · EF Core · Race Conditions · LINQ · Unit Testing · Full Code Examples

.NET Full Stack Developer

Interview Preparation Guide

Wipro | Infosys | LTI Mindtree — Detailed Q&A with Coding Examples

1. C# / .NET Core & ASP.NET Core

Q: What is the difference between an abstract class and an interface? Give a real-time scenario.

Answer:

Feature	Abstract Class	Interface
Constructor	Can have constructors	Cannot have constructors
Fields	Can have fields/state	Only properties (C# 8+ default impl.)
Access Modifiers	Any modifier allowed	Public by default
Inheritance	Single inheritance only	Multiple interfaces allowed
Use Case	Shared base behavior	Define a contract / capability

Real-time scenario in CTK project: Payment processing uses an abstract class for shared logic:

```
// Abstract class - shared base for all payment types
public abstract class PaymentBase
{
    public abstract decimal ProcessPayment(decimal amount);
    protected void LogTransaction(string message)
        => Console.WriteLine($"[LOG] {message}");
}

// Interface - defines a contract for refundable payments
public interface IRefundable
{
    bool ProcessRefund(decimal amount);
}

public class CreditCardPayment : PaymentBase, IRefundable
{
    public override decimal ProcessPayment(decimal amount)
    {
        LogTransaction($"Credit card: {amount}");
        return amount;
    }
    public bool ProcessRefund(decimal amount) => true;
}
```


Q: Have you used delegates? Can you provide a real-time scenario?

Answer:

A delegate is a type-safe function pointer. It holds a reference to a method and allows methods to be passed as parameters.

```
// Define delegate
public delegate void NotifyHandler(string message);

public class AlertService
{
    public void SendEmail(string msg) => Console.WriteLine($"Email: {msg}");
    public void SendSMS(string msg)   => Console.WriteLine($"SMS: {msg}");
}

// Usage with multicast delegate
var svc = new AlertService();
NotifyHandler notify = svc.SendEmail;
notify += svc.SendSMS;    // multicast
notify("Booking confirmed!");

// Modern approach - using Action<T> / Func<T>
Action<string> log = msg => Console.WriteLine(msg);
Func<int, int, int> add = (a, b) => a + b;
```

Q: What is the difference between First, FirstOrDefault, Single, and SingleOrDefault in LINQ?

Answer:

Method	Returns	When No Match	When Multiple Match
First()	First element	Throws exception	Returns first
FirstOrDefault()	First or default(T)	Returns null/0	Returns first
Single()	Exactly one element	Throws exception	Throws exception
SingleOrDefault()	One or default(T)	Returns null/0	Throws exception

```
var employees = new List<Employee> { ... };

// First - use when at least 1 record is guaranteed
var emp = employees.First(e => e.Id == 1);

// FirstOrDefault - safe when record might not exist
var emp = employees.FirstOrDefault(e => e.Id == 99); // null if not found

// Single - use when EXACTLY one record must exist (e.g., fetch by unique Id)
var emp = employees.Single(e => e.Id == 1);

// SingleOrDefault - safe version of Single
var emp = employees.SingleOrDefault(e => e.Email == "x@y.com");
```

Q: What is the difference between readonly, const, and static?

Answer:

Keyword	Set When	Per Instance?	Runtime/Compile?
const	Declared only	No — same for all	Compile-time
readonly	Declared or in constructor	Yes — per instance possible	Runtime
static	Once per class	No — class-level	Runtime

```
public class Config
{
    public const string AppName = "CTK";           // compile-time, inlined
    public readonly string ConnectionString;       // set in constructor
    public static int InstanceCount = 0;           // shared across instances

    public Config(string connStr)
    {
        ConnectionString = connStr;               // readonly set here
        InstanceCount++;
    }
}
```

Q: What is the difference between String and StringBuilder?

Answer:

Feature	string	StringBuilder
Mutability	Immutable	Mutable
Performance	Poor in loops (allocates new obj)	Excellent for concatenation in loops
Thread Safety	Thread safe (immutable)	Not thread safe by default
Use Case	Fixed, read-only text	Building dynamic strings in loops

```
// BAD — creates N new string objects in loop
string result = "";
for (int i = 0; i < 1000; i++) result += i;

// GOOD — StringBuilder modifies in-place
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.Append(i);
string result = sb.ToString();
```

Q: What is middleware? How do you create custom middleware in .NET Core?

Answer:

Middleware are components in the ASP.NET Core request pipeline. Each middleware can process the request, optionally call the next middleware, and process the response.

```
// Custom middleware class
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(RequestDelegate next,
        ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next; _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _logger.LogInformation($"Request: {context.Request.Path}");
        await _next(context); // call next middleware
        _logger.LogInformation($"Response: {context.Response.StatusCode}");
    }
}

// Register in Program.cs
app.UseMiddleware<RequestLoggingMiddleware>();
```

Q: What is the difference between Transient, Scoped, and Singleton lifetimes in DI?

Answer:

Lifetime	Created	Destroyed	Best For
Transient	Every time requested	After each use	Lightweight, stateless services
Scoped	Once per HTTP request	End of request	DB context, per-request state
Singleton	Once at app start	App shutdown	Config, caching, shared state

```
// Program.cs / Startup.cs
builder.Services.AddTransient<IEmailService, EmailService>();
builder.Services.AddScoped<IBookingRepository, BookingRepository>();
builder.Services.AddSingleton<ICacheService, CacheService>();

// WARNING: Never inject Scoped into Singleton!
// Scoped service would be captured and kept alive for app lifetime.
// Use IServiceScopeFactory instead:
public class MySingleton
{
    private readonly IServiceScopeFactory _scopeFactory;
    public MySingleton(IServiceScopeFactory f) => _scopeFactory = f;
    public void DoWork()
    {
    }
```

```

        using var scope = _scopeFactory.CreateScope();
        var repo = scope.ServiceProvider.GetRequiredService<IMyRepo>();
    }
}

```

Q: How do you generate a JWT (JSON Web Token) in Web API?

Answer:

```

// Generate JWT in AuthController
public string GenerateToken(User user)
{
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var claims = new[]
    {
        new Claim(ClaimTypes.Name, user.Username),
        new Claim(ClaimTypes.Role, user.Role),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };
    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddHours(1),
        signingCredentials: creds);
    return new JwtSecurityTokenHandler().WriteToken(token);
}

// Register in Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidIssuer = config["Jwt:Issuer"],
            ValidAudience = config["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(config["Jwt:Key"]))
        };
    });
}

```

Q: Payment system design — SOLID / Strategy Pattern (Mall Billing Scenario)

Answer:

Use the Strategy Pattern + Open/Closed Principle. Each payment method is a separate class implementing `IPaymentStrategy`. Adding new methods requires zero changes to existing code.

```

// Step 1: Define the contract (Interface)
public interface IPaymentStrategy
{

```

```

        void Pay(decimal amount);
    }

    // Step 2: Concrete strategies (one class per payment type)
    public class CreditCardPayment : IPaymentStrategy
    {
        public void Pay(decimal amount)
            => Console.WriteLine($"Paid {amount} via Credit Card");
    }
    public class VoucherPayment : IPaymentStrategy
    {
        public void Pay(decimal amount)
            => Console.WriteLine($"Paid {amount} via Voucher");
    }
    // Add new payment – just add a new class, no existing code changes!
    public class CashPayment : IPaymentStrategy
    {
        public void Pay(decimal amount)
            => Console.WriteLine($"Paid {amount} in Cash");
    }

    // Step 3: Context class – depends on abstraction, not concrete class
    public class BillingCounter
    {
        private readonly IPaymentStrategy _strategy;
        public BillingCounter(IPaymentStrategy strategy)
            => _strategy = strategy;
        public void ProcessPayment(decimal amount)
            => _strategy.Pay(amount);
    }

    // Usage
    var counter = new BillingCounter(new CreditCardPayment());
    counter.ProcessPayment(500m);

```

Q: What is the difference between IEnumerable and IQueryable?

Answer:

Feature	IEnumerable	IQueryable
Namespace	System.Collections.Generic	System.Linq
Execution	In-memory (LINQ to Objects)	Database-side (LINQ to SQL/EF)
Filtering	Loads all data, then filters	Translates to SQL WHERE clause
Performance	Poor for large datasets	Optimal — only fetches needed rows
Use Case	Already loaded collections	EF Core / database queries

```

// IQueryable – SQL WHERE runs on DB server
IQueryable<Employee> query = _context.Employees
    .Where(e => e.Salary > 50000); // Translates to: SELECT ... WHERE Salary >
50000

```

```
// IEnumerable - all rows loaded into memory first!
IEnumerable<Employee> list = _context.Employees.ToList()
    .Where(e => e.Salary > 50000); // Filters 1 million rows in C# memory
```

Q: Explain Async and Await. How does it work?

Answer:

async/await is C#'s mechanism for asynchronous programming. It allows threads to be freed while waiting for I/O operations (network, DB), improving throughput without blocking.

```
// Synchronous - thread blocked while DB query runs
public Employee GetEmployee(int id)
    => _context.Employees.FirstOrDefault(e => e.Id == id);

// Asynchronous - thread released back to pool while awaiting
public async Task<Employee> GetEmployeeAsync(int id)
    => await _context.Employees.FirstOrDefaultAsync(e => e.Id == id);

// Controller action
[HttpGet("{id}")]
public async Task<IActionResult> Get(int id)
{
    var emp = await _service.GetEmployeeAsync(id);
    if (emp == null) return NotFound();
    return Ok(emp);
}
```

Q: What is Managed Code and Unmanaged Code?

Answer:

Aspect	Managed Code	Unmanaged Code
Runtime	Runs under CLR (Common Language Runtime)	Runs directly on OS
Memory	Garbage collected automatically	Manual memory management (malloc/free)
Languages	C#, VB.NET, F#	C, C++, COM components
Safety	Type-safe, memory-safe	Prone to memory leaks/access violations

Q: Explain Caching — Memory Cache vs Redis

Answer:

Caching stores frequently accessed data in fast storage to reduce DB hits and improve response times.

```
// In-Memory Cache (single server)
```

```

builder.Services.AddMemoryCache();

public class ProductService
{
    private readonly IMemoryCache _cache;
    public ProductService(IMemoryCache cache) => _cache = cache;

    public async Task<Product> GetProduct(int id)
    {
        string key = $"product_{id}";
        if (!_cache.TryGetValue(key, out Product product))
        {
            product = await _db.Products.FindAsync(id);
            _cache.Set(key, product, TimeSpan.FromMinutes(10));
        }
        return product;
    }
}

// Distributed Cache - Redis (multi-server / cloud)
builder.Services.AddStackExchangeRedisCache(options => {
    options.Configuration = config["Redis:Connection"];
});

public class RedisService
{
    private readonly IDistributedCache _cache;
    public RedisService(IDistributedCache cache) => _cache = cache;

    public async Task SetAsync<T>(string key, T value)
    {
        var json = JsonSerializer.Serialize(value);
        await _cache.SetStringAsync(key, json,
            new DistributedCacheEntryOptions {
                AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
            });
    }
}

```

Q: Explain Microservices Architecture

Answer:

Microservices is an architectural style where an application is composed of small, independent services that communicate via APIs. Each service owns its data and can be deployed independently.

```

// Service-to-service communication using HttpClient
builder.Services.AddHttpClient<IOrderService, OrderService>(client => {
    client.BaseAddress = new Uri("https://order-service/api/");
});

public class OrderService : IOrderService
{
    private readonly HttpClient _client;
    public OrderService(HttpClient client) => _client = client;
}

```

```
public async Task<Order> GetOrder(int id)
{
    var response = await _client.GetAsync($"orders/{id}");
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadFromJsonAsync<Order>();
}
}
```


2. SQL Server & Databases

Q: What is the difference between Clustered and Non-Clustered Index?

Answer:

Feature	Clustered Index	Non-Clustered Index
Storage	Data rows physically sorted by key	Separate structure with row pointers
Count per table	Only ONE allowed	Up to 999 allowed
Speed (Range Queries)	Faster — sequential reads	Slower — extra lookup step
Auto-created on	PRIMARY KEY by default	Manually on frequently queried cols

Q: What is a Covering Index?

Answer:

A covering index includes all columns needed by a query in the index itself, eliminating key lookups back to the base table (avoids 'bookmark lookup').

```
-- Query: frequent search on Email, returning Name and Salary
SELECT Name, Salary FROM Employees WHERE Email = 'a@b.com';

-- Covering index - includes all columns the query needs
CREATE NONCLUSTERED INDEX IX_Emp_Email_Covering
ON Employees (Email)
INCLUDE (Name, Salary); -- INCLUDE avoids table lookup
```

Q: What is a CTE (Common Table Expression)? Have you used a CTE inside a stored procedure?

Answer:

A CTE is a named temporary result set defined with the WITH keyword, valid only for the duration of the query. It improves readability, supports recursion, and can be referenced multiple times.

```
-- Basic CTE in a stored procedure
CREATE PROCEDURE GetTopEarnersByDept
AS
BEGIN
    WITH DeptSalaries AS
    (
        SELECT DepartmentID,
               MAX(Salary) AS MaxSalary
        FROM   Employees
        GROUP BY DepartmentID
    )
    SELECT e.Name, e.Salary, d.DepartmentName
    FROM   Employees e
    JOIN   DeptSalaries ds ON e.DepartmentID = ds.DepartmentID
                        AND e.Salary = ds.MaxSalary
    JOIN   Departments d  ON d.DepartmentID = e.DepartmentID;
```

```
END
```

Q: Write a query to find the second highest salary department-wise (Coding Challenge)

Answer:

```
WITH RankedSalaries AS
(
    SELECT e.EmployeeID,
           e.Name,
           e.Salary,
           e.DepartmentID,
           DENSE_RANK() OVER
               (PARTITION BY e.DepartmentID ORDER BY e.Salary DESC)
               AS SalaryRank
    FROM Employee e
)
SELECT r.Name,
       r.Salary,
       d.DepartmentName
FROM RankedSalaries r
JOIN Department d ON d.DepartmentID = r.DepartmentID
WHERE r.SalaryRank = 2;
```

Why DENSE_RANK instead of ROW_NUMBER? DENSE_RANK handles ties correctly — if two employees share the highest salary, ROW_NUMBER would skip rank 2, while DENSE_RANK gives both rank 1 and still labels the next unique salary as rank 2.

Q: Write a query to remove duplicate records from a User table using CTE and PARTITION BY

Answer:

```
-- User table: Email, UserID, UserName (no primary key)
WITH DuplicateCTE AS
(
    SELECT *,
           ROW_NUMBER() OVER
               (PARTITION BY Email ORDER BY UserID) AS RowNum
    FROM Users
)
DELETE FROM DuplicateCTE
WHERE RowNum > 1; -- keeps row 1 (lowest UserID), deletes rest
```

Q: Difference between Temporary Table (#temp) and Table Variable (@table)?

Answer:

Feature	#Temp Table	@Table Variable
Scope	Session / stored procedure	Current batch only

Feature	#Temp Table	@Table Variable
Indexes	Can create explicit indexes	Only inline index on PK
Performance	Better for large datasets	Better for small datasets
Transaction aware	Yes	No — not rolled back
Best for	1M+ rows, complex ops	< 100 rows, temp calculations

Q: ROW_NUMBER() vs DENSE_RANK() — difference?

Answer:

```
-- Salary data: 100, 100, 90, 80
SELECT Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum,
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRnk,
       RANK() OVER (ORDER BY Salary DESC) AS Rnk
FROM Employees;

-- Output:
-- Salary  RowNum  DenseRnk  Rnk
-- 100      1       1         1
-- 100      2       1         1  <-- RANK skips 2, ROW_NUMBER doesn't, DENSE_RANK
keeps 1
-- 90       3       2         3
-- 80       4       3         4
```

Q: CTE vs Global Temp Table for 1M records stored procedure — which to use and why?

Answer:

Use a Global Temporary Table (##temp) when:

- You need to create indexes on the intermediate result set
- The computation is complex and referenced multiple times
- The intermediate result is large (100K+ rows)

Use a CTE when:

- The query is simpler, read-once, and the dataset is moderate
- Readability is the goal — CTE is not materialized by default

```
-- Use ##temp for large transactional table (1M+ rows)
CREATE PROCEDURE ProcessSalaryReport
AS
BEGIN
    -- Materialize once with indexes
    SELECT EmployeeID, Salary, DepartmentID
    INTO    ##SalaryStage
    FROM    Transactions
    WHERE   Year = 2025;

    CREATE INDEX IX_Stage_Dept ON ##SalaryStage (DepartmentID);
```

```
-- Now run complex calcs against indexed temp table
SELECT d.Name, SUM(s.Salary) AS TotalCost
FROM    ##SalaryStage s
JOIN    Departments d ON d.ID = s.DepartmentID
GROUP BY d.Name;

DROP TABLE ##SalaryStage;

END
```

3. Entity Framework Core

Q: Difference between Eager Loading, Explicit Loading, and Lazy Loading?

Answer:

Type	How	SQL Generated	Use When
Eager Loading	Include()	JOIN in single query	You always need related data
Lazy Loading	Virtual nav properties	Extra SQL on property access	Related data accessed rarely
Explicit Loading	Load() called manually	Separate SQL on demand	Conditional loading needed

```
// Eager Loading - single SQL with JOIN
var orders = await _context.Orders
    .Include(o => o.Customer)
    .Include(o => o.OrderItems)
    .ToListAsync();

// Explicit Loading - load related data on demand
var order = await _context.Orders.FindAsync(1);
await _context.Entry(order).Reference(o => o.Customer).LoadAsync();
await _context.Entry(order).Collection(o => o.OrderItems).LoadAsync();

// Lazy Loading - requires virtual keyword + proxy package
// services.AddDbContext<AppDbContext>(opts =>
//     opts.UseLazyLoadingProxies().UseSqlServer(conn));
public class Order {
    public virtual Customer Customer { get; set; } // lazy-loaded on access
}
```

Q: What is the N+1 query problem? Does it happen in eager or lazy loading?

Answer:

N+1 occurs with Lazy Loading: 1 query fetches N orders, then N additional queries fetch each order's customer — total N+1 queries. Eager Loading with Include() prevents this.

```
// N+1 Problem (Lazy Loading)
var orders = await _context.Orders.ToListAsync(); // 1 query
foreach (var order in orders)
{
    // This fires a separate SQL for EACH order! (N queries)
    Console.WriteLine(order.Customer.Name);
}
// Total SQL hits: 1 + N

// Fix with Eager Loading
var orders = await _context.Orders
    .Include(o => o.Customer) // Single JOIN query
    .ToListAsync();
// Total SQL hits: 1
```

Q: EF Core Migrations Commands

Answer:

```
# Add a new migration
dotnet ef migrations add AddEmployeeTable

# Apply pending migrations to database
dotnet ef database update

# Rollback to specific migration
dotnet ef database update PreviousMigrationName

# Remove the last migration (if not applied)
dotnet ef migrations remove

# Script all migrations to SQL file
dotnet ef migrations script
```

Q: LINQ Challenge: Find the 3rd highest salaried employee. Handle ties.

Answer:

```
// Using LINQ - handles ties with Distinct() + OrderByDescending
var thirdHighest = employees
    .Select(e => e.Salary)
    .Distinct()                // remove duplicate salaries
    .OrderByDescending(s => s)  // rank unique salaries
    .Skip(2)                   // skip 1st and 2nd highest
    .FirstOrDefault();         // take 3rd

var result = employees
    .Where(e => e.Salary == thirdHighest)
    .ToList();                 // all employees at that salary level

// Why not just Skip(2).FirstOrDefault() on employees?
// If salaries are: 100, 100, 90, 80
// Without Distinct: Skip(2) gives 90 (correct here by luck)
// But: 100, 90, 90, 80 - Skip(2) gives 90, not 3rd distinct = 80
// Distinct() ensures correct behavior for all tie scenarios
```

4. Angular (Client-Side)

Q: How do you transfer data between components with no parent-child relationship?

Answer:

Use a shared Angular Service with a BehaviorSubject as the communication channel.

```
// shared.service.ts
@Injectable({ providedIn: 'root' })
export class SharedService {
  private userSource = new BehaviorSubject<User>(null);
  currentUser$ = this.userSource.asObservable();

  updateUser(user: User) {
    this.userSource.next(user); // broadcast to all subscribers
  }
}

// component-a.ts - emits data
constructor(private shared: SharedService) {}
onLogin(user: User) { this.shared.updateUser(user); }

// component-b.ts - receives data
constructor(private shared: SharedService) {}
ngOnInit() {
  this.shared.currentUser$.subscribe(user => this.currentUser = user);
}
```

Q: What is the difference between Subject, BehaviorSubject, and ReplaySubject?

Answer:

Type	Initial Value	Replays to Late Subscriber	Use Case
Subject	None	No — only future values	Event broadcasting
BehaviorSubject	Required (seed value)	Yes — replays last 1 value	Shared state (user, theme)
ReplaySubject(N)	None	Yes — replays last N values	Recent history needed

Q: What is an Angular Interceptor and why do we use it?

Answer:

Interceptors intercept HTTP requests/responses globally. Common uses: adding auth headers, logging, error handling, and token refresh.

```
// auth.interceptor.ts
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
```

```

constructor(private authService: AuthService) {}

intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
  const token = this.authService.getToken();
  const cloned = token
    ? req.clone({ headers: req.headers.set('Authorization', `Bearer ${token}`) })
    : req;
  return next.handle(cloned);
}
}

// app.module.ts
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]

```

Q: How do you refresh an expired auth token using HTTP Interceptor (401 handling)?

Answer:

```

intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
  return next.handle(this.addToken(req)).pipe(
    catchError((error: HttpResponse) => {
      if (error.status === 401) {
        return this.authService.refreshToken().pipe(
          switchMap((newToken: string) => {
            this.authService.storeToken(newToken);
            return next.handle(this.addToken(req)); // retry original request
          }),
          catchError(err => {
            this.authService.logout(); // refresh failed - force logout
            return throwError(() => err);
          })
        );
      }
      return throwError(() => error);
    })
  );
}

private addToken(req: HttpRequest<any>): HttpRequest<any> {
  return req.clone({
    headers: req.headers.set('Authorization',
      `Bearer ${this.authService.getToken()}`)
  });
}

```

Q: Observable vs Promise — key differences?

Answer:

Feature	Observable (RxJS)	Promise
Values	Multiple values over time	Single future value

Feature	Observable (RxJS)	Promise
Lazy/Eager	Lazy — executes on subscribe	Eager — executes immediately
Cancellable	Yes — unsubscribe()	No
Operators	map, filter, switchMap, etc.	None (only .then/.catch)
Use in Angular	HttpClient, EventEmitter	One-off async ops

Q: What is @Injectable() and providedIn: 'root'?

Answer:

```
@Injectable({
  providedIn: 'root' // registers as Singleton in root injector
                    // same instance shared across entire application
})
export class UserService {
  private users: User[] = [];

  getUsers() { return this.users; }
  addUser(user: User) { this.users.push(user); }
}

// 'root'          → app-wide singleton
// 'any'           → one instance per lazy-loaded module
// SomeModule      → scoped to that module
// No providedIn   → must be listed in module providers[]
```

5. Azure Cloud Services

Q: HTTP Trigger vs Timer Trigger in Azure Functions?

Answer:

Trigger	When it fires	Use Case
HTTP Trigger	On incoming HTTP request	REST APIs, webhooks, CRUD endpoints
Timer Trigger	On CRON schedule	Batch jobs, reports, cleanup tasks

```
// HTTP Trigger - Add Customer
public class CustomerFunction
{
    private readonly ICustomerService _service;
    public CustomerFunction(ICustomerService service) => _service = service;

    [Function("AddCustomer")]
    public async Task<IActionResult> Run(
        [HttpTrigger(AuthorizationLevel.Function, "post", Route = "customers")]
        HttpRequest req)
    {
        var body = await new StreamReader(req.Body).ReadToEndAsync();
        var customer = JsonSerializer.Deserialize<Customer>(body);
        await _service.AddAsync(customer);
        return new OkObjectResult(new { message = "Customer added", data = customer
    });
    }
}

// Timer Trigger - runs every day at midnight
[Function("DailyCleanup")]
public void Run([TimerTrigger("0 0 0 * * *")] TimerInfo timer)
{
    _logger.LogInformation($"Daily cleanup ran at: {DateTime.UtcNow}");
    // perform cleanup logic
}
```

Q: Azure Key Vault integration with .NET API

Answer:

```
// Program.cs
var keyVaultUri = new
Uri($"https://{builder.Configuration["KeyVault:Name"]}.vault.azure.net/");
builder.Configuration.AddAzureKeyVault(keyVaultUri, new DefaultAzureCredential());

// Then access secrets exactly like appsettings.json
var connStr = builder.Configuration["SqlConnectionString"];

// Or inject IConfiguration anywhere
public class MyService
{
    private readonly IConfiguration _config;
```

```

public MyService(IConfiguration config) => _config = config;
public void DoWork()
{
    var secret = _config["MySecretKey"]; // sourced from Key Vault
}
}

```

Q: Clean Architecture vs Traditional Three-Layer Architecture

Answer:

Aspect	Three-Layer (Traditional)	Clean Architecture
Layers	Presentation → Business → Data	Domain, Application, Infrastructure, UI
Dependency direction	Top-down — UI depends on BL, BL depends on DAL	Inward — all layers depend on Domain
Testability	Hard — business logic coupled to DAL	Easy — domain/app has no infrastructure deps
DB coupling	Business layer may know DB details	Infrastructure details hidden behind interfaces

Q: Design Patterns — Factory Pattern

Answer:

The Factory Pattern creates objects without specifying the exact class. It centralizes creation logic.

```

// Payment factory — creates correct payment processor
public interface IPaymentProcessor { void Process(decimal amount); }

public class PaymentFactory
{
    public static IPaymentProcessor Create(string type) => type switch
    {
        "CreditCard" => new CreditCardProcessor(),
        "PayPal"      => new PayPalProcessor(),
        "Voucher"     => new VoucherProcessor(),
        _ => throw new ArgumentException($"Unknown payment: {type}")
    };
}

// Usage
var processor = PaymentFactory.Create("CreditCard");
processor.Process(250m);

```

6. OOP Concepts, Memory & Performance

Q: Stack vs Heap memory — what is the difference?

Answer:

Feature	Stack	Heap
Stores	Value types, method frames, local vars	Reference types (objects, strings)
Size	Fixed, small (~1MB per thread)	Large (GBs, managed by GC)
Allocation	Automatic (push/pop)	Manual (new) — GC handles dealloc
Speed	Very fast	Slower due to GC
Lifetime	Until method returns	Until GC collects

```
void Example()
{
    int x = 5;           // value type - stored on STACK
    string s = "hello";  // reference type - object on HEAP, reference on stack
    var emp = new Employee(); // Employee object on HEAP
} // x popped from stack; emp eligible for GC
```

Q: What is the Garbage Collector? How does it work?

Answer:

The GC automatically manages heap memory. It works in 3 generations (Gen 0, 1, 2). Short-lived objects (Gen 0) are collected most frequently. Long-lived objects survive to Gen 2.

```
// IDisposable - for unmanaged resources (DB connections, file handles)
public class FileProcessor : IDisposable
{
    private FileStream _stream;
    private bool _disposed = false;

    public FileProcessor(string path)
        => _stream = new FileStream(path, FileMode.Open);

    public void Dispose()
    {
        if (!_disposed)
        {
            _stream?.Dispose();
            _disposed = true;
        }
        GC.SuppressFinalize(this);
    }
}

// Always use 'using' to ensure Dispose is called
using var fp = new FileProcessor("data.txt");
```

Q: Task vs Thread — what is the difference? When to use each?

Answer:

Feature	Thread	Task
Level	OS-level thread	Higher-level abstraction over threads
Pool	New thread each time	Uses ThreadPool (efficient)
Return value	Cannot return a value	Task<T> returns a value
Cancellation	Manual flags	Built-in CancellationToken
Use when	Long-running CPU work	I/O ops, async work (99% of cases)

```
// Task — preferred for async I/O
public async Task<string> FetchDataAsync()
{
    await Task.Delay(1000); // non-blocking wait
    return "data";
}

// Task.Run — for CPU-bound work on background thread
var result = await Task.Run(() => ComputeHeavyCalculation());

// Parallel tasks
var t1 = FetchDataAsync();
var t2 = FetchOtherAsync();
await Task.WhenAll(t1, t2); // run both concurrently
```

Q: Hashtable vs Dictionary vs HashSet

Answer:

Type	Key/Value?	Type Safety	Duplicates	Thread Safe
Hashtable	Yes (object, object)	No — boxing/unboxing	No duplicate keys	Yes (synchronized)
Dictionary<K,V>	Yes (typed)	Yes — generic	No duplicate keys	No
HashSet<T>	Values only	Yes — generic	No duplicates allowed	No

Q: ref vs out parameter keywords in C#

Answer:

```
// ref — variable must be initialized BEFORE passing
void Double(ref int value) => value *= 2;
int x = 5;
Double(ref x); // x is now 10
```

```
// out - variable need NOT be initialized; MUST be assigned inside method
bool TryParse(string s, out int result)
{
    result = 0;    // must assign before returning
    return int.TryParse(s, out result);
}
if (TryParse("42", out int val))
    Console.WriteLine(val);    // 42
```

Q: Method Overloading vs Method Overriding

Answer:

Feature	Overloading	Overriding
Definition	Same name, different parameters	Redefine base class virtual method
Binding	Compile-time (static)	Runtime (dynamic dispatch)
Keyword	None needed	override (child), virtual (parent)
Polymorphism	Compile-time polymorphism	Runtime polymorphism

```
// Overloading
public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;    // same name, diff params

// Overriding
public class Animal { public virtual void Speak() => Console.WriteLine("..."); }
public class Dog : Animal { public override void Speak() =>
    Console.WriteLine("Woof!"); }

Animal a = new Dog();
a.Speak();    // "Woof!" - runtime binding
```

Q: Partial class — what is it and when to use it?

Answer:

A partial class lets you split a class definition across multiple files. All parts are combined at compile time into one class. Partial classes CANNOT span different projects.

```
// Employee.cs
public partial class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
}

// Employee.Validation.cs
public partial class Employee
{
```

```
    public bool IsValid() => !string.IsNullOrEmpty(Name) && Id > 0;
}

// Use case: Code generators (EF scaffolding generates one partial,
// developer adds logic in another partial – both survive regeneration)
```

Q: Constructor Chaining in C#

Answer:

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Department { get; set; }

    public Employee(int id) : this(id, "Unknown") { }

    public Employee(int id, string name) : this(id, name, "General") { }

    public Employee(int id, string name, string dept)
    {
        Id = id; Name = name; Department = dept;
    }
}

var e1 = new Employee(1);           // Id=1, Name=Unknown, Dept=General
var e2 = new Employee(2, "Alice"); // Id=2, Name=Alice, Dept=General
```

7. Web API, REST & HTTP

Q: Common HTTP Status Codes — how many do you know?

Answer:

Code	Name	When to use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE (nothing to return)
400	Bad Request	Validation errors, bad input
401	Unauthorized	Not authenticated (missing/invalid token)
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate/state conflict (e.g., duplicate email)
422	Unprocessable Entity	Semantically invalid input
500	Internal Server Error	Unexpected server exception

Q: API Versioning — how do you implement it in .NET?

Answer:

```
// Install: Microsoft.AspNetCore.Mvc.Versioning
builder.Services.AddApiVersioning(options => {
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
});

[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/employees")]
public class EmployeesV1Controller : ControllerBase
{
    [HttpGet] public IActionResult Get() => Ok("v1 employees");
}

[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/employees")]
public class EmployeesV2Controller : ControllerBase
{
    [HttpGet] public IActionResult Get() => Ok("v2 employees with pagination");
}
```

Q: Exception Handling in .NET Core Web API — global approach

Answer:

```
// Global exception handling middleware
public class GlobalExceptionHandler
```



```

{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;

    public GlobalExceptionMiddleware(RequestDelegate next,
        ILogger<GlobalExceptionMiddleware> logger)
    { _next = next; _logger = logger; }

    public async Task InvokeAsync(HttpContext context)
    {
        try { await _next(context); }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");
            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";
            await context.Response.WriteAsJsonAsync(new
            {
                error = "An error occurred",
                detail = ex.Message
            });
        }
    }
}

// Program.cs
app.UseMiddleware<GlobalExceptionMiddleware>();

```

Q: Model Validation in ASP.NET Core

Answer:

```

public class CreateEmployeeDto
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Range(18, 65, ErrorMessage = "Age must be 18-65")]
    public int Age { get; set; }

    [EmailAddress]
    public string Email { get; set; }
}

// Controller - ModelState checked automatically with [ApiController]
[HttpPost]
public IActionResult Create([FromBody] CreateEmployeeDto dto)
{
    // With [ApiController], 400 is returned automatically if invalid
    var emp = _mapper.Map<Employee>(dto);
    _service.Add(emp);
    return CreatedAtAction(nameof(Get), new { id = emp.Id }, emp);
}

```

8. AI Developer Roadmap (Emerging Skills)

The path forward: Becoming an AI Developer — RAG, MCP, Semantic Kernel, Azure AI, Guardrails, Embeddings.

Chunking, Embeddings, and Vectorization

These are the three core pillars of building RAG (Retrieval-Augmented Generation) systems:

```
// 1. CHUNKING — split documents into smaller pieces
//   Why: LLMs have token limits; search needs focused context
//   Strategies: fixed-size (512 tokens), sentence-based, paragraph-based

// 2. EMBEDDINGS — convert text to numerical vectors
//   Why: enables semantic similarity search (not just keyword matching)
//   Model: text-embedding-ada-002 (Azure OpenAI)

// 3. VECTORIZATION — store embeddings in a vector database
//   Options: Azure AI Search, Pinecone, Qdrant, Weaviate

// C# example using Semantic Kernel
var kernel = Kernel.CreateBuilder()
    .AddAzureOpenAIChatCompletion("gpt-4", endpoint, apiKey)
    .AddAzureOpenAITextEmbeddingGeneration("text-embedding-ada-002", endpoint,
    apiKey)
    .Build();
```

RAG — Retrieval-Augmented Generation in ASP.NET

```
// RAG Pipeline in C#
public class RagService
{
    private readonly Kernel _kernel;
    private readonly IVectorStore _vectorStore;

    public async Task<string> AskAsync(string question)
    {
        // Step 1: Embed the user's question
        var embedding = await
        _kernel.GetRequiredService<ITextEmbeddingGenerationService>()
            .GenerateEmbeddingAsync(question);

        // Step 2: Retrieve top-K relevant chunks
        var docs = await _vectorStore.SearchAsync(embedding, topK: 5);

        // Step 3: Build grounded prompt
        var context = string.Join("\n", docs.Select(d => d.Content));
        var prompt = $"""
        Answer based on context only:
        Context: {context}
        Question: {question}
        """;
```

```

        // Step 4: Call LLM with grounded context
        var result = await _kernel.InvokePromptAsync(prompt);
        return result.ToString();
    }
}

```

MCP Server using C#

```

// Model Context Protocol (MCP) server in C#
// Install: dotnet add package ModelContextProtocol

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddMcpServer()
    .WithHttpTransport()
    .WithToolsFromAssembly();

var app = builder.Build();
app.MapMcp();
app.Run();

// Define an MCP Tool
[McpServerToolType]
public class FlightTools
{
    [McpServerTool, Description("Search available flights")]
    public async Task<FlightResult[]> SearchFlights(
        [Description("Origin airport code")] string origin,
        [Description("Destination code")] string destination,
        [Description("Travel date (yyyy-MM-dd)")] string date)
    {
        // call flight search API
        return await _flightService.SearchAsync(origin, destination, date);
    }
}

```

Semantic Kernel — Agentic AI

```

// Agentic pattern with Semantic Kernel
var kernel = Kernel.CreateBuilder()
    .AddAzureOpenAIChatCompletion(model, endpoint, key)
    .Build();

// Add native C# function as a plugin
kernel.Plugins.AddFromType<FlightSearchPlugin>("FlightSearch");

// Auto function calling — kernel decides when to call plugins
var settings = new AzureOpenAIPromptExecutionSettings
{
    FunctionChoiceBehavior = FunctionChoiceBehavior.Auto()
};

var result = await kernel.InvokePromptAsync(
    "Find me a direct flight from DOH to LHR next Monday",

```

```
new KernelArguments(settings));  
  
Console.WriteLine(result); // AI automatically invoked FlightSearch plugin
```

Interview Q&A – Company-wise

Coding Assessment

Q1: Product Service – Design a service with GET /products and POST /products endpoints

```
// ASP.NET Core Web API  
  
[ApiController]  
[Route("api/[controller]")]  
public class ProductsController : ControllerBase  
{  
    private static List<Product> _products = new();  
  
    [HttpGet]  
    public IActionResult GetProducts() => Ok(_products);  
  
    [HttpPost]  
    public IActionResult AddProduct([FromBody] Product product)  
    {  
        product.Id = _products.Count + 1;  
        _products.Add(product);  
        return CreatedAtAction(nameof(GetProducts), new { id = product.Id },  
product);  
    }  
}  
  
public class Product  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public decimal Price { get; set; }  
}
```

Q2: Dividend ÷ Divider Problem – Compute integer division without using *, /, or % operators

```
public int Divide(int dividend, int divisor)  
{  
    if (divisor == 0) throw new DivideByZeroException();  
    int sign = (dividend < 0) ^ (divisor < 0) ? -1 : 1;  
    long a = Math.Abs((long)dividend);  
    long b = Math.Abs((long)divisor);
```

```

long result = 0;

while (a >= b)
{
    long temp = b, multiple = 1;
    while (a >= (temp << 1))
    {
        temp <<= 1;
        multiple <<= 1;
    }
    a -= temp;
    result += multiple;
}
return (int)(sign * result);
}
// Uses bit shifting (<<) to double divisor until it exceeds dividend

```

.NET Core Topics

Q1: What are the core fundamentals of .NET Core?

.NET Core is a cross-platform, open-source framework. Key fundamentals: (1) Dependency Injection is built-in via IServiceCollection. (2) Middleware pipeline processes HTTP requests. (3) Configuration system supports appsettings.json, environment variables, secrets. (4) Hosting via IHost and WebApplication builder. (5) Kestrel is the default cross-platform web server. (6) Everything is modular — you add only what you need via NuGet packages.

Q2: How does JWT authentication work in .NET Core?

JWT flow: (1) Client sends credentials to /auth/login. (2) Server validates and returns a signed JWT token (Header.Payload.Signature). (3) Client includes token in subsequent requests as Authorization: Bearer <token>. (4) Server validates signature using the secret key configured in AddAuthentication().AddJwtBearer(). Setup in Program.cs:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(options => { options.TokenValidationParameters = new TokenValidationParameters { ValidateIssuer = true, ValidateAudience = true, ValidateLifetime = true, ValidateIssuerSigningKey = true, IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret)) }; });
```

Then use [Authorize] attribute on controllers.

Q3: How do you implement API versioning in .NET Core?

Install: Microsoft.AspNetCore.Mvc.Versioning

```

builder.Services.AddApiVersioning(o => {
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.ReportApiVersions = true;
});
// URL segment versioning: /api/v1/products
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/{controller}")]
// Header versioning: api-version: 2.0 in request headers
// Query string: /api/products?api-version=2.0

```

Q4: How do you secure APIs in .NET Core?

Multiple layers: (1) JWT Bearer Authentication — validate tokens on every request. (2) Authorization Policies — `[Authorize(Policy="AdminOnly")]` with role/claim checks. (3) HTTPS enforcement — `UseHttpsRedirection()` + HSTS. (4) CORS — restrict to known origins. (5) Rate Limiting — `AspNetCoreRateLimit` package. (6) Input Validation — `FluentValidation` or `DataAnnotations`. (7) Store secrets in Azure Key Vault, not `appsettings.json`. (8) Use `[ApiController]` for automatic model validation. (9) Add API Gateway (APIM) for centralized security.

Azure Logic Apps

Q1: How do you implement try-catch error handling in Azure Logic Apps, and how do you ensure only valid errors are handled in the catch block?

In Logic Apps, wrap actions in a Scope. Add a second Scope configured as the catch block by setting 'Run After' to 'has failed', 'has timed out', or 'has been skipped'. To handle only specific errors, add a Condition inside the catch scope: `check result('ScopeName')?[0]?['error']?['code'] equals the expected error code` (e.g., 'BadRequest', '404'). This way, the catch block logic runs only for targeted error types, not all failures.

Q2: How do you handle transient errors in Azure Logic Apps?

Logic Apps has built-in retry policies on each action. Configure under Settings > Retry Policy: choose Exponential interval (recommended for transient errors) or Fixed interval. Set retry count (e.g., 4) and interval. For custom control, use a Until loop with a condition checking the response status. For HTTP actions, the runtime automatically retries on 429 (throttling) and 5xx errors. You can also enable Run After on failure to route to a fallback path.

Q3: Why did you choose Logic App Standard over Consumption Plan in your project?

Logic App Standard runs on dedicated infrastructure (App Service Plan), giving: (1) VNet integration — connect to private resources without exposing them publicly. (2) Stateful + Stateless workflows in one app. (3) Better performance with local execution. (4) More control over scaling. (5) Supports multiple workflows in one resource. Consumption plan is per-execution billed and fully managed but lacks VNet support and has cold-start latency — making Standard the right choice for enterprise integrations needing private connectivity.

Azure API Management (APIM)

Q1: What is an APIM policy, and where have you used it?

APIM policies are XML-based rules applied at inbound, backend, outbound, or on-error pipeline stages. Common uses: (1) `rate-limit-by-key` — throttle calls per subscription. (2) `set-header` — inject headers before forwarding to backend. (3) `rewrite-uri` — transform incoming URL to backend URL. (4) `validate-jwt` — verify JWT token at the gateway before hitting the API. (5) `cors` — handle cross-origin at gateway

level. I used validate-jwt to enforce AAD token validation and set-backend-service to route to different backends based on environment.

Q2: How do you implement OAuth 2.0 authentication in APIM? Walk through the steps.

Step-by-step: (1) Register an App in Azure AD — define app roles and expose an API scope (e.g., api://myapp/read). (2) In APIM > OAuth 2.0 + OpenID Connect, add an OAuth 2.0 server pointing to the AAD authorization and token endpoints. (3) Link the OAuth server to the API under API Settings. (4) Add a validate-jwt inbound policy on the API: specify the AAD OpenID config URL, required audience (client ID), and required claims. (5) Client app acquires token from AAD and sends it as Bearer in the Authorization header. APIM validates it before forwarding to backend.

Azure Functions

Q1: What is Dependency Injection in Azure Functions, and why is it important?

DI in Azure Functions allows services (DbContext, HttpClient, custom services) to be injected into function classes rather than instantiated manually. This enables testability (mock dependencies), reuse of singleton/scoped instances, and cleaner separation of concerns. Without DI, you'd create new instances per invocation, causing connection pool exhaustion (especially with DbContext/HttpClient).

Q2: How do you register and configure Dependency Injection in Azure Functions?

```
// Install: Microsoft.Azure.Functions.Extensions
// Create Startup.cs:
[assembly: FunctionsStartup(typeof(MyApp.Startup))]
public class Startup : FunctionsStartup
{
    public override void Configure(IFunctionsHostBuilder builder)
    {
        builder.Services.AddSingleton<MyService, MyService>();
        builder.Services.AddDbContext<AppDbContext>(options =>
            options.UseSqlServer(Environment.GetEnvironmentVariable("SqlConnection")));
        builder.Services.AddHttpClient();
    }
}
// Inject in function:
public class MyFunction
{
    private readonly IMyService _service;
    public MyFunction(IMyService service) { _service = service; }
    [FunctionName("MyFunc")]
    public async Task<IActionResult> Run([HttpTrigger] HttpRequest req) {
        ... }
}
```

Azure Service Bus

Q1: What are Filters and Routing in Azure Service Bus Topics and Subscriptions?

Service Bus Topics allow multiple subscriptions, each with filter rules that decide which messages it receives. Filter types: (1) SQL Filter — filter by message properties: color = 'red' AND priority > 2. (2) Correlation Filter — match CorrelationId, SessionId, Label, or custom properties (most performant). (3)

True Filter — subscription receives all messages. (4) False Filter — subscription receives no messages. Routing: different subscriptions (e.g., OrdersIndia, OrdersUS) subscribe to the same topic but with different country filters, enabling message routing without multiple queues.

Q2: How do you handle messages in the Dead Letter Queue (DLQ) of Azure Service Bus?

Messages land in DLQ when: max delivery count is exceeded, TTL expires, or explicitly dead-lettered by consumer. Handling: (1) Monitor DLQ using Azure Monitor alerts or Application Insights. (2) Create a separate processor/function that reads from the DLQ path: entityPath/\$DeadLetterQueue. (3) Inspect the DeadLetterReason and DeadLetterErrorDescription properties to understand failure. (4) Fix the underlying issue and resubmit to the main queue, or log and discard. In production, I set up an Azure Function on a timer to drain and reprocess DLQ messages after the root cause is resolved.

Transformations & Design Patterns

Q1: What is the difference between XSLT Transformation and Liquid Transformation, and when would you use each?

XSLT (Extensible Stylesheet Language Transformations): Used to transform XML to XML (or HTML/text). Powerful for complex XML restructuring, looping, conditionals. Used in Logic Apps via the Transform XML action with an Integration Account map. Liquid Transformation: A template engine (based on Shopify Liquid) used to transform JSON to JSON, JSON to text, or JSON to HTML. Used in Logic Apps via the Transform JSON to JSON action. Rule of thumb: XML-to-XML use XSLT; JSON-to-JSON use Liquid. For XML-to-JSON, convert XML to JSON first, then apply Liquid.

Q2: What are Cloud Integration Design Patterns, and which have you implemented?

Key patterns: (1) Message Broker — decouple producer/consumer via Service Bus (implemented in CTK project for async processing). (2) Saga Pattern — manage distributed transactions across microservices with compensating transactions. (3) Competing Consumers — multiple function instances process from the same queue for scale. (4) Claim Check — store large payloads in Blob Storage, pass a reference token in the message. (5) Retry & Circuit Breaker — handle transient failures gracefully. (6) API Gateway — centralized entry point via APIM for auth, throttling, routing. (7) Pub/Sub — Service Bus Topics + Subscriptions for broadcast scenarios.

C# / OOP Questions

Q1: Difference between Abstract Class and Interface

Abstract Class: can have implemented methods, fields, constructors; supports single inheritance; use when sharing code among related classes. Interface: only method signatures (from C# 8, can have default implementations); supports multiple implementation; use to define a contract. Key rule: if classes share behavior → Abstract Class. If unrelated classes need a common contract → Interface. Real example: Shape (abstract class with Area() method) vs ISerializable (interface any class can implement).

Q2: Dependency Injection and Service Lifetimes (Transient, Scoped, Singleton)

Transient: new instance every time it's requested. Use for lightweight, stateless services. Scoped: one instance per HTTP request. Use for DbContext — ensures same context within a request. Singleton: one instance for the app's lifetime. Use for config, caching, logging. Danger: never inject Scoped into Singleton — the scoped service won't be disposed correctly (captive dependency problem). Register: services.AddTransient<>(), services.AddScoped<>(), services.AddSingleton<>().

Q3: Async/Await Internals

async/await is syntactic sugar over the Task-based Asynchronous Pattern. When you await a Task, the compiler generates a state machine that suspends the current method and releases the thread back to the thread pool. When the awaited operation completes, execution resumes (possibly on a different thread). Key points: await doesn't block threads — it frees them. ConfigureAwait(false) avoids capturing the SynchronizationContext (important in libraries). async void is only for event handlers — use async Task everywhere else. Exceptions in async methods are wrapped in the returned Task.

Q4: Task vs Thread

Thread: OS-level, heavyweight (1MB stack by default), managed by you. Task: abstraction over the thread pool, lightweight, supports async/await, continuations, cancellation, and return values. Use Task always for async operations. Use Thread only if you need fine-grained control (priority, foreground/background). Task.Run() schedules work on the thread pool; new Thread() creates a dedicated thread.

Q5: IEnumerable vs IQueryable

IEnumerable: executes query in memory (LINQ to Objects). Data is pulled into memory first, then filtered. IQueryable: defers execution to the data source (LINQ to SQL/EF Core). The expression tree is translated to SQL and filtering happens in the database. Rule: always use IQueryable in repository methods to let callers add .Where()/Skip()/Take() before the query hits the DB. Use IEnumerable when data is already in memory.

Q6: String vs StringBuilder

string is immutable — every concatenation creates a new string object in memory. StringBuilder is mutable — it maintains a buffer and appends in-place. Use string for simple, infrequent concatenations. Use StringBuilder in loops or when building large strings dynamically. Example: building a CSV row in a loop with 10,000 records → StringBuilder avoids 10,000 string allocations.

Q7: Dispose vs Finalize

Dispose: deterministic cleanup, called explicitly by code or via using statement. Implement IDisposable. Called immediately when the using block exits. Finalize (~ClassName): non-deterministic, called by the GC before collecting the object. Use only for unmanaged resource cleanup as a safety net. Best practice: implement both (Dispose Pattern) — Dispose releases resources immediately; Finalize is the fallback if Dispose wasn't called.

Q8: Value Type vs Reference Type

Value Types (int, struct, enum, bool): stored on the stack, copied on assignment, no heap allocation. Reference Types (class, string, array, delegate): stored on the heap, assignment copies the reference, GC-managed. Key implication: modifying a struct parameter inside a method doesn't affect the caller's copy (pass by value). Use ref or out to pass value types by reference.

Q9: Extension Methods

Extension methods add new methods to existing types without modifying them.

```
// Must be in a static class, method must be static, first param prefixed with 'this'
public static class StringExtensions
{
```

```

public static bool IsNullOrEmpty(this string value)
    => string.IsNullOrEmpty(value);
public static string ToTitleCase(this string value)
    => CultureInfo.CurrentCulture.TextInfo.ToTitleCase(value.ToLower());
}
// Usage: "hello world".ToTitleCase();
Used in LINQ — all LINQ methods (Where, Select, OrderBy) are extension methods on
IEnumerable<T>.

```

Q10: SOLID Principles (SRP, OCP, LSP)

SRP (Single Responsibility): a class should have only one reason to change. Don't mix business logic with data access in the same class. OCP (Open/Closed): open for extension, closed for modification. Add new behavior via new classes/interfaces, not by editing existing code. Use strategy pattern. LSP (Liskov Substitution): derived classes must be substitutable for their base class without breaking behavior. If Square extends Rectangle and overrides Width/Height setters in a way that violates the area formula, it breaks LSP. ISP: don't force clients to depend on interfaces they don't use — split fat interfaces. DIP: depend on abstractions (interfaces), not concrete implementations — the foundation of DI.

C# & .NET Questions

Q1: Collections (Generic & Non-Generic)

Non-Generic (System.Collections): ArrayList, Hashtable, Queue, Stack — store objects, require boxing/unboxing, not type-safe. Generic (System.Collections.Generic): List<T>, Dictionary<K,V>, Queue<T>, Stack<T>, HashSet<T> — type-safe, better performance, no boxing. Always prefer generic collections. Use Dictionary for key-value lookup O(1), List for ordered collection, HashSet for unique items.

Q2: Boxing & Unboxing

Boxing: converting a value type to object (reference type) — allocates on heap. `int x = 5; object o = x; // boxing`. Unboxing: extracting value type back from object — requires explicit cast. `int y = (int)o; // unboxing`. Both have performance costs. Avoid in performance-critical loops. This is why generic collections were introduced — List<int> avoids boxing that ArrayList<int> caused.

Q3: Ref vs Out

ref: variable must be initialized before passing; method can read and write it. out: variable doesn't need to be initialized; method must assign a value before returning. Use ref when the method needs the current value AND will update it (e.g., swap). Use out when the method's purpose is to produce a value (e.g., `int.TryParse(str, out int result)`).

Q4: Write Custom Middleware – Global Exception Handling Example

```

public class GlobalExceptionMiddleware

```

```

{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;

    public GlobalExceptionMiddleware(RequestDelegate next, ILogger<GlobalExceptionMiddleware>
logger)
    {
        _next = next; _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try { await _next(context); }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");
            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";
            await context.Response.WriteAsync(
                JsonSerializer.Serialize(new { error = ex.Message }));
        }
    }
}
// Register: app.UseMiddleware<GlobalExceptionMiddleware>();

```

Q5: Create two objects and swap their values (Code)

```

// Using temp variable
var a = new Product { Id = 1, Name = "A" };
var b = new Product { Id = 2, Name = "B" };

// Swap reference types using tuple deconstruction
(a, b) = (b, a);
Console.WriteLine(a.Name); // B
Console.WriteLine(b.Name); // A

// For value types, classic swap:
int x = 10, y = 20;
(x, y) = (y, x); // C# 7+ tuple swap

```

Q6: Explain 4 Pillars of OOPs as used in your project

Encapsulation: CTK entities like FamTrip have private setters; business rules enforced inside the class, not exposed. Abstraction: IRepository<T> hides the EF Core implementation; controllers only know the interface. Inheritance: BaseEntity class (Id, CreatedAt, ModifiedAt) inherited by all domain entities — avoids repetition. Polymorphism: INotificationService has EmailNotification and SMSNotification implementations; the correct one is injected at runtime via DI.

Q7: How do you test/debug your backend API endpoints?

During development: Swagger UI (auto-generated via Swashbuckle) for quick testing. Postman for complex scenarios — save collections per environment. Visual Studio debugger — set breakpoints, inspect variables. For integration testing: xUnit + WebApplicationFactory to spin up in-memory test server. For production issues: Application Insights for traces and exceptions, Serilog structured logs, Azure Log Stream for real-time output.

Q8: How do you document your API endpoints?

Swagger/OpenAPI via Swashbuckle: auto-generates interactive docs from code. Enhance with XML comments: [summary], [param], [response] tags on controller methods. Add ProducesResponseType attributes to document response types. For enterprise: use API Management (APIM) developer portal — import OpenAPI spec and publish polished documentation. Keep Postman collections updated and share with team.

Q9: Code First vs DB First in EF Core — which is more efficient and why?

Code First: define C# entity classes → EF generates DB schema via migrations. Gives full version control of schema, fits agile development. DB First: generate C# models from existing DB using Scaffold-DbContext. Use when legacy DB exists and team owns the schema. Code First is generally preferred for new projects: schema is tracked in Git, migrations are reversible, no manual sync between code and DB. DB First is efficient when the DBA controls the schema.

Q10: How do you integrate API endpoints in Angular? (Example code)

```
// product.service.ts
@Injectable({ providedIn: 'root' })
export class ProductService {
  private apiUrl = 'https://api.example.com/api/products';
  constructor(private http: HttpClient) {}

  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>(this.apiUrl);
  }
  addProduct(product: Product): Observable<Product> {
    return this.http.post<Product>(this.apiUrl, product);
  }
}
// component.ts
products$ = this.productService.getProducts();
// template
<div *ngFor="let p of products$ | async">{{ p.name }}</div>
```

Q11: How do you secure API endpoints — some public, some protected?

```
// Mark protected endpoints:
[Authorize]
[HttpGet("secure-data")]
public IActionResult SecureData() { ... }

// Mark public endpoints:
[AllowAnonymous]
[HttpGet("public-data")]
public IActionResult PublicData() { ... }

// Configure JWT in Program.cs:
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => { /* token validation params */ });
app.UseAuthentication();
app.UseAuthorization();
```

```
// Role-based:  
[Authorize(Roles = "Admin")]  
// Policy-based:  
[Authorize(Policy = "PremiumOnly")]
```

Q12: What are the 4 data binding techniques in Angular?

1. Interpolation `{{ value }}` – component → template, display data. 2. Property Binding `[src]='imageUrl'` – component → template, bind DOM properties. 3. Event Binding `(click)='handleClick()'` – template → component, handle events. 4. Two-Way Binding `[(ngModel)]='name'` – both directions simultaneously, requires `FormsModule`. Two-way binding is syntactic sugar for `[ngModel] + (ngModelChange)`.

Q13: How do you identify and resolve performance issues?

Identify: Application Insights slow request traces, SQL Profiler for slow queries, browser DevTools Network tab for Angular. Backend: use execution plans in SSMS to find missing indexes. EF Core: enable logging to see generated SQL, catch N+1 queries. Angular: Chrome Performance tab, `ChangeDetectionStrategy.OnPush` to reduce re-renders. Resolve: add indexes, use `AsNoTracking()`, paginate results, implement caching (`IMemoryCache/Redis`), lazy load Angular modules.

Q14: What is indexing and different types of indexing strategies?

Clustered Index: defines physical sort order of table rows. One per table, usually on PK. Non-Clustered Index: separate structure with pointers to rows. Multiple allowed. Covering Index: includes all columns needed by a query to avoid table lookup (`INCLUDE` columns). Composite Index: index on multiple columns — column order matters (most selective first). Filtered Index: index with a `WHERE` clause — efficient for partial data. Full-Text Index: for text search. Strategy: index columns in `WHERE`, `JOIN`, `ORDER BY`. Avoid over-indexing — hurts `INSERT/UPDATE` performance.

Q15: How do you check execution plan in SQL and identify performance issues?

In SSMS: `SET STATISTICS IO ON + SET STATISTICS TIME ON` before query. Or click 'Include Actual Execution Plan' (`Ctrl+M`). Look for: Table Scan (no index used — add index), Clustered Index Scan (partial — add covering index), Key Lookup (non-clustered hit but needs columns from table — use `INCLUDE`), Thick arrows (large row estimates — stats may be stale, run `UPDATE STATISTICS`). Sort operator with high cost — add an index to avoid sort.

Q16: Have you ever used SQL Profiler?

Yes, SQL Server Profiler captures real-time query activity. Used it to identify slow queries in production by filtering on `Duration > 1000ms` and `CPU > 500`. This helped pinpoint an N+1 issue from EF Core generating 500+ `SELECT` calls per page load. Modern alternative: Extended Events (lighter weight, less server overhead). Also used `sp_WhoIsActive` for live query monitoring.

Q17: Have you been involved in production issue triages? How do you resolve them?

Yes. Process: (1) Identify — alert from Application Insights or Azure Monitor. (2) Reproduce — check logs, correlate request traces. (3) Isolate — is it DB, API, or frontend? (4) Hotfix — push a targeted fix via CI/CD pipeline with expedited review. (5) Verify — monitor metrics post-deploy. (6) Post-mortem — document root cause and preventive action. In CTK, we had a Service Bus message processing outage caused by a DLQ overflow — resolved by deploying a DLQ reprocessor function and adding a circuit breaker.

ASP.NET Core

Q1: Temporary tables vs Global temporary tables in SQL Server?

#TempTable: local to the current session/connection, dropped automatically when session ends.
##GlobalTempTable: visible to all sessions, dropped when last session using it closes. Both are stored in tempdb. Table Variable (@TableVar): scoped to the batch/stored procedure, faster for small datasets, no statistics, can't be used with certain index types. Use #Temp for intermediate results in complex queries. Use @TableVar for small, simple result sets.

Q2: What are Action Filters in ASP.NET Core?

Action Filters run before/after a controller action executes. Implement `IActionFilter` or `IAsyncActionFilter`.

```
// Example: Logging filter
public class LoggingFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
        => Console.WriteLine($"Action {context.ActionDescriptor.DisplayName}
starting");

    public void OnActionExecuted(ActionExecutedContext context)
        => Console.WriteLine($"Action completed");
}
// Register globally:
builder.Services.AddControllers(o => o.Filters.Add<LoggingFilter>());
// Or per controller: [ServiceFilter(typeof(LoggingFilter))]
```

Other built-in filters: Authorization, Resource, Exception, Result filters.

Q3: Explain INNER JOIN with example

INNER JOIN returns only rows where there is a match in both tables.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
```

```
INNER JOIN Departments d ON e.DepartmentId = d.Id
```

-- Returns only employees who belong to an existing department.

-- Employees with NULL DepartmentId are excluded.

-- Used when you need related data from two tables and only care about matched records.

Q4: Code review: Identify issues in async method

Common async code issues to look for:

1. `async void` — should be `async Task` (exceptions are unobservable)
2. `.Result` or `.Wait()` — causes deadlocks in ASP.NET Core
3. Missing `await` — fire-and-forget bug, exception swallowed
4. Missing `ConfigureAwait(false)` in library code
5. Not using `CancellationToken` — method can't be cancelled

6. Blocking inside async (Thread.Sleep vs await Task.Delay)
7. Creating new HttpClient inside async method — should be injected as singleton

Q5: Difference between IActionResult and ActionResult<T>?

IActionResult: non-generic, can return any result type (Ok, NotFound, BadRequest). Flexible but no type info for Swagger. ActionResult<T>: generic, tells Swagger the expected return type AND allows returning IActionResult for error cases. Recommended for Web API controllers — gives better OpenAPI documentation. Example: ActionResult<Product> can return Ok(product) or return NotFound().

Q6: Extension methods — Why must the extension class be static?

The class must be static so the compiler can find the extension methods without instantiating the class. Extension methods themselves must be static because they don't operate on an instance of the containing class — they operate on the first parameter (the extended type). The static class is just a namespace/container for organizing extension methods.

Q7: Partial classes and partial methods

Partial class: a class definition split across multiple files, combined at compile time. Used by code generators (EF Core, WinForms designer) to separate generated code from user code. Partial method: declared in one partial class part, optionally implemented in another. If not implemented, the compiler removes the call entirely — zero overhead. Must be private and return void (C# 9+ relaxes this for implemented partials).

Q8: What are delegates? Why do we use them? Are methods first-class functions?

A delegate is a type-safe function pointer — it holds a reference to a method. Yes, in C#, methods are first-class via delegates — you can pass them as parameters, store in variables, return from methods. Use delegates for: callbacks, event handlers, strategy pattern. Built-in delegates: Action (void return), Func<T, TResult> (returns value), Predicate<T> (returns bool). Example: List<T>.Sort(Comparison<T>) — you pass a delegate defining sort logic.

Q9: What is a request delegate? What is HttpContext?

Request delegate: RequestDelegate is a function (Func<HttpContext, Task>) that handles an HTTP request. It's the building block of the ASP.NET Core middleware pipeline. Each middleware receives the request, does work, and optionally calls the next delegate. HttpContext: encapsulates all HTTP request/response information — Request (headers, body, query), Response (status, headers, body), User (ClaimsPrincipal), Session, Connection, and Services (IServiceProvider for request scope).

Q10: How to avoid background services hanging — CancellationToken

```
public class MyBackgroundService : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await DoWorkAsync(stoppingToken);
            await Task.Delay(TimeSpan.FromSeconds(30), stoppingToken);
        }
    }
    private async Task DoWorkAsync(CancellationToken ct)
    {

```



```

        // Pass ct to all async calls: await dbContext.SaveChangesAsync(ct);
        ct.ThrowIfCancellationRequested(); // Check in long CPU loops
    }
}
// stoppingToken is signalled on app shutdown - service exits gracefully.

```

Q11: Flow of ASP.NET Core Web API request

Request arrives → Kestrel (HTTP server) → Middleware pipeline (Routing, Authentication, Authorization, CORS, Custom middleware) → Controller action selected → Model binding (deserialize request) → Action Filters (OnActionExecuting) → Action method executes → Result Filters → Response serialized → Middleware pipeline (outbound) → Kestrel sends response.

Q12: .NET Framework vs .NET Core

.NET Framework: Windows-only, large monolithic install, older APIs, supports WCF/WebForms fully.
.NET Core (now .NET 5+): cross-platform (Windows/Linux/macOS), lightweight, modular, open-source, better performance, side-by-side versioning. Choose .NET Core for all new development. Framework only if maintaining legacy WCF services or Windows-specific APIs not yet ported.

Q13: What is Kestrel and Reverse Proxy?

Kestrel: cross-platform, high-performance HTTP server built into ASP.NET Core. It's the default server that listens for requests. Reverse Proxy (IIS/Nginx/Azure App Service): sits in front of Kestrel to handle SSL termination, load balancing, static file serving, and security. In production, Kestrel is almost always behind a reverse proxy — it's exposed only to the proxy, not the internet directly.

Q14: Why do we need the Repository Pattern?

Repository Pattern abstracts data access behind an interface. Benefits: (1) Swap data sources without changing business logic (SQL → MongoDB). (2) Unit test services by mocking IRepository<T>. (3) Centralize query logic — no scattered LINQ in controllers. (4) Works well with Unit of Work for transaction management. In CTK, IFamTripRepository.GetPendingApprovals() encapsulates the complex EF query; the service layer just calls it.

Q15: Facade Design Pattern (GoF)

Facade provides a simplified interface to a complex subsystem.

```

// Complex subsystems:
class HotelBooking { public void Book() { ... } }
class FlightBooking { public void Book() { ... } }
class CarRental { public void Reserve() { ... } }

// Facade:
class TravelFacade
{
    private HotelBooking _hotel = new();
    private FlightBooking _flight = new();
    private CarRental _car = new();

    public void BookTrip()
    {
        _hotel.Book();
        _flight.Book();
        _car.Reserve();
    }
}

```



```

    }
}
// Client calls TravelFacade.BookTrip() instead of 3 separate classes.
Used in the CTK project: a FamTripFacade orchestrates eligibility check, booking, and notification services.

```

Q16: What is AsNoTracking(), Skip(), Take()?

AsNoTracking(): tells EF Core not to track returned entities in the ChangeTracker — significantly faster for read-only queries (no snapshot overhead). Use for GET endpoints. Skip(n): offset — skip first n records. Take(n): limit — return next n records. Used together for pagination:

```
dbContext.Products.AsNoTracking().OrderBy(p => p.Id).Skip((page-1)*pageSize).Take(pageSize).ToListAsync().
```

SQL Server

Q1: HAVING vs GROUP BY

GROUP BY groups rows with the same values into summary rows. HAVING filters groups after aggregation (WHERE filters rows before grouping). Example: SELECT DepartmentId, COUNT(*) as EmpCount FROM Employees GROUP BY DepartmentId HAVING COUNT(*) > 5 — returns only departments with more than 5 employees. Rule: WHERE = filter rows, HAVING = filter groups.

Q2: Write a query to find count of each employee per department

```

SELECT d.DepartmentName, COUNT(e.Id) AS EmployeeCount
FROM Departments d
LEFT JOIN Employees e ON e.DepartmentId = d.Id
GROUP BY d.DepartmentName
ORDER BY EmployeeCount DESC;
-- LEFT JOIN ensures departments with 0 employees are also shown.

```

Q3: Table Variable vs Temp Table — difference and when to use which

@TableVariable: scoped to batch/SP, no transaction log, no statistics, can't be used with EXEC sp (can't pass across dynamic SQL). Best for small datasets (<1000 rows). #TempTable: scoped to session, stored in tempdb with statistics, supports indexes, participates in transactions, survives across multiple batches. Best for larger datasets needing indexes or when joining with other tables repeatedly. Choose temp table when you need indexes or when the data volume is large.

Q4: What are Stored Procedures (SP)?

Stored Procedures are precompiled SQL code stored in the database. Benefits: (1) Performance — execution plan cached after first run. (2) Security — grant EXECUTE permission without exposing tables. (3) Reuse — call from multiple applications. (4) Reduce network traffic — one call executes multiple statements. Drawbacks: business logic spread across app and DB, harder to version control, tightly coupled to SQL Server.

Q5: Performance optimization in SQL Server

Key techniques: (1) Proper indexing — clustered on PK, non-clustered on WHERE/JOIN columns, covering indexes with INCLUDE. (2) Avoid SELECT * — retrieve only needed columns. (3) Use EXISTS instead of IN for subqueries. (4) Avoid functions on indexed columns in WHERE —

SARGability. (5) UPDATE STATISTICS regularly. (6) Query hints sparingly (NOLOCK for dirty reads on reporting). (7) Partition large tables. (8) Use READ COMMITTED SNAPSHOT ISOLATION to reduce blocking. (9) Identify and resolve deadlocks via Extended Events.

Q6: Query execution flow in SQL Server

1. Parse — syntax checked, parse tree created. 2. Algebrize — semantic check, object resolution (table/column existence). 3. Optimize — Query Optimizer generates candidate execution plans, chooses lowest-cost plan based on statistics. 4. Plan Cache — plan stored; reused on next execution if parameters match (parameterized queries). 5. Execute — Storage Engine retrieves data using the plan. 6. Results returned to client.

Angular

Q1: Count occurrence of each word (JavaScript/TypeScript)

```
function wordCount(str: string): Record<string, number> {  
    return str.toLowerCase().split(/s+/).reduce((acc, word) => {  
        acc[word] = (acc[word] || 0) + 1;  
        return acc;  
    }, {} as Record<string, number>);  
}  
  
console.log(wordCount("hello world hello"));  
// { hello: 2, world: 1 }
```

HCL – Round 1 (CleverHire AI Bot)

Q1: Walk me through the specific steps you took — joins, indexes, keys — what exact query, procedure, or production issue did you personally diagnose, and what was the before/after impact?

In the Qatar Airways CTK project, we had a stored procedure GetAgentIncentiveSummary that was running ~8 seconds per call on the report dashboard. It was joining five tables: Agents, FamTrips, Bookings, IncentiveRules, and Approvals, with no indexes on the foreign key columns used in the JOIN conditions.

My specific contribution:

1. Ran the query in SSMS with "Include Actual Execution Plan" (Ctrl+M).
2. Identified a Clustered Index Scan on the Bookings table (200K+ rows) and a Key Lookup on Agents.
3. Added a non-clustered index on Bookings(AgentId, TripId) INCLUDE (Status, BookingDate).
4. Added a filtered index on Approvals(Status) WHERE Status = 'Pending'.

Before: ~8.2 seconds average, 45,000 logical reads on Bookings.

After: ~0.9 seconds, 320 logical reads — verified using SET STATISTICS IO ON before and after.

Q2: You mentioned an N+1 issue and reducing repeated execution. What was the actual query pattern, and what change did you make to prove the improvement in production?

The N+1 was from EF Core. The dashboard page loaded a list of FamTrips, and for each trip the code called `trip.Bookings.Count` — triggering a separate SELECT per trip:

```
// BAD: N+1
var trips = dbContext.FamTrips.ToList(); // 1 query
foreach(var t in trips)
    var count = t.Bookings.Count; // 1 query × N trips

// FIX: Eager load with projection
var trips = dbContext.FamTrips
    .Select(t => new { t.Id, t.Name, BookingCount = t.Bookings.Count() })
    .ToList(); // 1 query total
```

Proof: Enabled EF Core query logging (`LogLevel.Information`) in dev. Saw 87 SELECT statements per page load before, 1 after. In production, confirmed via Application Insights dependency tracking — avg duration dropped from 4.1s to 0.3s for that page.

Q3: What was the latency or CPU before you changed it, what was it after, and how did you measure that?

Measurement approach: Application Insights custom metrics + SQL Server DMVs.

Before fix:

- API endpoint P95 latency: 7.8 seconds (Application Insights)
- SQL CPU time: ~3200ms (from SET STATISTICS TIME ON)
- Logical reads: 45,000 (from SET STATISTICS IO ON)

After fix (indexes + eager loading):

- API endpoint P95 latency: 0.7 seconds
- SQL CPU time: ~85ms
- Logical reads: 320

Also queried `sys.dm_exec_query_stats` DMV before and after to confirm plan cache had picked up the new index. In production, Azure App Insights "Performance" blade showed the operation moved from the red (>5s) bucket to green (<1s) post-deployment.

Q4: What exact plan pattern were you looking for in the execution plan, and what did it tell you about the slow part?

The key patterns I look for:

1. Table Scan / Clustered Index Scan on large tables — means no usable index. Action: add non-clustered index on the WHERE/JOIN columns.
2. Key Lookup — non-clustered index was used but needed extra columns from the base table. Action: add those columns to INCLUDE clause of the index.
3. Thick arrows between operators — indicates large row estimate; often means stale statistics. Action: `UPDATE STATISTICS TableName`.
4. Sort operator with high cost — missing index to pre-sort. Action: add index matching ORDER BY.
5. Nested Loops on large outer input — can signal a missing index on the inner table join column.

In the CTK case, I saw a Clustered Index Scan (not seek) on Bookings with an arrow width proportional to 180K rows, followed by a Hash Match join — telling me the optimizer couldn't use an index seek and resorted to a full scan + hash join, which is expensive on large tables.

Q5: In your last SQL Server system, what was the hardest stored procedure you had to tune, and what exact change did you make first?

The hardest was usp_GenerateMonthlyIncentiveReport — a 200-line SP joining 6 tables with 3 levels of nested subqueries, running for 45+ seconds on month-end.

First change: replaced the deeply nested subqueries with CTEs for readability and to allow the optimizer to materialize intermediate results:

```
WITH EligibleAgents AS (  
    SELECT AgentId FROM Agents WHERE Status = 'Active' AND JoinDate <  
    DATEADD(MONTH, -3, GETDATE())  
)  
TripBookings AS (  
    SELECT b.AgentId, COUNT(*) AS TripCount, SUM(b.Amount) AS TotalAmount  
    FROM Bookings b  
    INNER JOIN EligibleAgents e ON b.AgentId = e.AgentId  
    WHERE b.BookingDate BETWEEN @StartDate AND @EndDate  
    GROUP BY b.AgentId  
)  
SELECT ... FROM TripBookings ...
```

This alone cut runtime from 45s to 22s. Second change: added a composite index on Bookings(AgentId, BookingDate) INCLUDE(Amount) which brought it to 3.8s. Verified by running the old and new SP back-to-back on the same date range in a non-prod environment and comparing STATISTICS IO output.

Q6: You mentioned replacing repeated SELECTs with a CTE. What was your specific contribution, and how did you verify it did not change the result set?

My contribution: identified 4 places in the SP where the same subquery (agent eligibility check) was copy-pasted. Extracted it into a single CTE at the top.

Verification approach (no data integrity risk):

1. Ran OLD SP with OUTPUT INTO #OldResults on a test dataset of 3 months.
2. Ran NEW SP with OUTPUT INTO #NewResults on the same dataset.
3. Compared with:
SELECT * FROM #OldResults EXCEPT SELECT * FROM #NewResults -- should return 0 rows
SELECT * FROM #NewResults EXCEPT SELECT * FROM #OldResults -- should return 0 rows
4. Also compared COUNT(*) and SUM(Amount) totals between both result sets.
5. Did this across 3 different date ranges (current month, last quarter, year-to-date).

Zero differences confirmed. Deployed to UAT, had QA sign off on report output parity, then promoted to production.

Q1: You have three months, two teams disagree on whether to split a service now or keep it monolithic, and a hard launch date is fixed. How do you decide and ship on time?

Framework I use: Strangler Fig + risk-adjusted decomposition.

Decision criteria (3 questions):

1. Does this capability have an independent deployment need right now? If not, keep it monolithic.
2. Does the team boundary align with the service boundary? If the same team owns both sides, there is no coordination gain from splitting.
3. Can we isolate it behind an interface (anti-corruption layer) today and physically split it post-launch?

With a fixed launch date, I would advocate for a "logical split, physical monolith" approach:

- Extract the capability into its own namespace/project within the solution.
- Define the interface (INotificationService) and wire it via DI.
- Deploy as one unit for launch.
- Post-launch, physically separate into its own service with minimal risk since the boundary is already clean.

This gives both teams what they want: the "split now" team gets a clean code boundary; the "keep monolithic" team gets no deployment risk at launch.

Q2: What was the hardest decomposition decision you have made before, and what dependency or failure mode made that choice difficult?

In CTK, the Incentive Calculation logic was deeply embedded in the Trip Management module. Both read/wrote to the same database tables and shared domain objects.

The hard part: Incentive calculation depended on real-time trip status — if a trip was cancelled mid-calculation, the incentive had to be voided. Splitting into two services meant this transactional consistency was gone (can't do a DB transaction across two services).

How we resolved it:

- Kept the core trip + incentive calculation in the same bounded context (same service, same DB transaction).
- Extracted only the Notification and Reporting concerns as separate services, since those were eventually-consistent by nature.
- Used the Outbox Pattern: write notification events to an OutboxEvents table inside the same DB transaction, then a background job published them to Service Bus. This gave us consistency without distributed transactions.

Q3: If you had to split one business capability from a monolith under a fixed launch, what would be the first boundary you would pick, and what would you keep inside the monolith?

First boundary to split: Notification Service.

Reason: notifications are fire-and-forget, eventually consistent, and have no write-back to core domain data. They are the safest to extract because a notification failure doesn't break a business transaction.

Keep inside monolith:

- Core booking/trip management (transactional, needs ACID guarantees)
- Incentive calculation (tightly coupled to trip status)
- User authentication (security boundary changes are high-risk)

The rule I use: split services that are naturally async and failure-tolerant first. Keep services that require distributed transaction coordination in the monolith until you have a clear saga/outbox strategy.

Q1: In Azure, if a deploy causes cascading failures across three dependent services, and rollback is not safe because the DB schema already moved forward, what do you do first?

First action: stop the bleeding — prevent further traffic from hitting the broken code while keeping the forward schema.

Immediate steps (first 2 minutes):

1. Azure App Service: Deployment Slots — swap back to previous slot (this rolls back the app code, not the DB). The slot swap is instant and schema-safe.
2. If slots are not configured: scale down the new deployment to 0 instances via Azure App Service scaling rules and route traffic to the previous version via Azure Traffic Manager or APIM routing policies.
3. Engage the DB: if the new schema added columns/tables (additive), the old code runs fine against it. Only destructive schema changes (dropped columns) cause true incompatibility — which is why we enforce expand/contract migration strategy.

The key: decouple code rollback from schema rollback. Code can roll back instantly; schema needs a separate migration to revert.

Q2: Once cascading failures are already happening in production, what exact steps would you take in the first 10 minutes to stabilize it without rolling back the schema?

Minute 0-2: Reduce blast radius

- Azure App Service: enable "auto-swap back" or manually swap deployment slot to previous code version.
- Azure API Management: apply a throttling policy or circuit breaker policy to failing endpoints to prevent downstream services from being overwhelmed.

Minute 2-5: Identify the cascade origin

- Application Insights: open Live Metrics Stream — look for the first service showing failure rate spike (this is the root cause, not the downstream victims).
- Azure Service Health + Resource Health blade: check for platform-level issues (not your code).

Minute 5-8: Isolate the failing service

- If the failing service is non-critical: disable its downstream calls via feature flag (Azure App Configuration) — let the system degrade gracefully.
- If Service Bus messages are piling up: pause the consumer (disable the Azure Function trigger) to prevent DLQ overflow while you stabilize.

Minute 8-10: Verify stabilization

- Watch Application Insights failure rate drop below threshold.
- Confirm dependent services return to green in Azure Monitor.
- Communicate status to stakeholders via incident channel.

Root cause analysis happens after stabilization, not during.

Q3: Tell me one concrete Azure service you would inspect first in that outage, what signal you would look for, and what action you would take next.

Service: Azure Application Insights — specifically the "Failures" blade + End-to-End Transaction trace.

Signal I look for:

- Operation failure rate: which operation name (controller action or function) shows the first spike in the timeline.
- Dependency failures tab: is the failure originating in the app code OR in a downstream dependency (SQL, Service Bus, external API)?
- Exception type and stack trace on the first failed request — this tells me whether it is a `NullReferenceException` (code bug), a `SqlException` (schema mismatch), or a timeout (infrastructure).

Concrete example from CTK:

After a deploy, App Insights showed HTTP 500 on POST /api/trips. Dependency trace showed `SqlException: "Invalid column name 'IncentiveType'"` — the new code was referencing a column the migration hadn't run yet on prod (migration ran on staging, was blocked on prod by a permissions issue).

Action taken:

1. Swapped App Service slot back to previous code version (30 seconds).
2. Ran the migration manually via Azure SQL: `ALTER TABLE Trips ADD IncentiveType NVARCHAR(50) NULL.`
3. Swapped back to new code.
4. Total downtime: 4 minutes.

Post-incident: added migration validation step to the CI/CD pipeline (check pending migrations before deploy completes).